Artificial Intelligence: A Mathematical and Engineering Perspective

March 2026

Chapter 1

Learning as Function Approximation

1.1 Introduction

Machine learning is often introduced through lists of tasks: predicting house prices, recognizing objects in images, translating between languages, recommending movies, or generating text. Although these applications look very different on the surface, they share a common mathematical core. In each case, we are given observations and asked to construct a rule that maps inputs to outputs in a useful way. This rule may predict a number, assign a category, estimate a probability distribution, or produce a structured object such as a sentence or image. From a sufficiently abstract viewpoint, learning is the problem of discovering an unknown function from data.

This chapter develops that viewpoint carefully. The idea of *learning as function approximation* is not the whole story of machine learning, but it is one of its most important organizing principles. It gives a common language for regression, classification, sequence prediction, density estimation, and representation learning. It also clarifies why machine learning is difficult. We never observe the unknown rule directly. Instead, we only see finitely many examples, often corrupted by noise, bias, missing information, or measurement error. The learner must infer a useful approximation from incomplete evidence.

Thinking in terms of function approximation immediately raises fundamental questions. What is the input space? What is the output space? What class of functions are we allowed to search over? How do we decide whether one approximation is better than another? When do finitely many examples contain enough information to identify a useful rule? Why do some models generalize well beyond the training data while others fail? These questions will guide not only this chapter but the rest of the book.

The aim of this chapter is threefold. First, we build a general mathematical formulation of learning problems. Second, we show how a wide range of machine

learning tasks fit into that formulation. Third, we explain why approximation alone is not enough: learning also requires data, structure, inductive bias, and criteria for selecting among many possible approximations. Later chapters will develop these ideas more rigorously, but the conceptual framework begins here.

1.2 What Does It Mean to Learn?

At an intuitive level, learning means improving performance on a task through experience. In machine learning, the "experience" is data, and the "performance" is measured by some loss or utility. The learner observes examples and uses them to construct a model whose behavior is expected to be useful on future examples.

A mathematical way to express this is to imagine an unknown target rule

$$f^\star : \mathcal{X} \to \mathcal{Y},$$

where $\mathcal{X}$ is an input space and $\mathcal{Y}$ is an output space. The goal of the learner is to construct a function

$$f : \mathcal{X} \to \mathcal{Y}$$

that approximates $f^\star$ well on inputs of interest.

Here the word "function" should be interpreted broadly. In the simplest settings, f maps a vector of numbers to a real number or a class label. In more advanced settings, f may map an image to a sentence, a sequence of tokens to the next token distribution, or a noisy signal to a denoised one. The core idea is unchanged: the learner seeks a useful mapping between spaces.

[Learning Problem] A learning problem consists of

- an input space $\mathcal{X}$,
- an output space $\mathcal{Y}$,
- a collection of observed examples,
- a class $\mathcal{H}$ of candidate functions $f : \mathcal{X} \to \mathcal{Y}$,
- and a criterion for judging the quality of a candidate function.

The goal is to choose a function $f \in \mathcal{H}$ that performs well on the task of interest.

This definition is deliberately broad. It covers classical regression and classification, but it also includes many modern tasks in deep learning. What changes from one setting to another is the nature of $\mathcal{X}$, $\mathcal{Y}$, the hypothesis class $\mathcal{H}$, and the performance criterion.

A useful first distinction is between the unknown target rule and the model we actually learn. The target rule $f^\star$ is an idealization. In many practical situations, no deterministic function perfectly describes the relationship between input and output. Real data may contain randomness, ambiguity, or multiple plausible answers. Even then, the function viewpoint remains valuable. One can

interpret $f^\star$ as the best possible predictor under some loss, or more generally as a conditional distribution or decision rule. Later chapters will make this probabilistic refinement precise.

1.2.1 Experience as Data

The learner does not have direct access to $f^\star$. Instead, it sees a finite sample of input–output pairs,

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n).$$

These examples provide partial information about the unknown rule. From them, the learner must extrapolate to future inputs.

This is the central difficulty of machine learning. Infinitely many functions can agree with finitely many observations. Without additional assumptions, the data alone do not determine a unique solution. Thus learning is never just approximation; it is approximation under uncertainty from finite information.

1.2.2 A First Approximation View

Suppose the target rule were known exactly. Then the problem would reduce to approximation theory: find a simpler or more computationally tractable function f that stays close to $f^\star$ under an appropriate notion of error. Machine learning adds another layer: the target rule is hidden, and we only observe samples generated from it. So the learner must solve two problems simultaneously:

1. infer a useful target relationship from limited data;
2. approximate that relationship within a chosen model class.

This dual role explains why machine learning sits at an intersection of statistics, optimization, approximation theory, information theory, and computer science.

1.3 Inputs, Outputs, and Representations

A learning problem begins with a choice of input and output spaces. While this may appear straightforward, it is often one of the most consequential modeling decisions.

1.3.1 Input Space

The input space $\mathcal{X}$ is the set of objects on which the learner operates. Examples include:

- vectors in $\mathbb{R}^d$ representing tabular features,
- images represented as arrays in $\mathbb{R}^{H \times W \times C}$,

- sequences of tokens,
- graphs,
- audio signals,
- time series,
- sets or point clouds.

In practice, the raw object of interest is often not immediately available in a numerical form suitable for learning. A real-world entity such as a person, document, or molecule must be represented in a way that the model can process. This transformation is called a *representation*. Choosing a representation is itself a modeling step, and later chapters will show that much of modern deep learning can be interpreted as *learning representations* automatically from data.

1.3.2 Output Space

The output space $\mathcal{Y}$ depends on the task. Common examples are:

- $\mathbb{R}$ for scalar regression,
- $\{1, \ldots, K\}$ for multiclass classification,
- $\mathbb{R}^m$ for vector-valued prediction,
- a set of sequences for machine translation or text generation,
- a space of probability distributions for probabilistic forecasting.

The structure of $\mathcal{Y}$ strongly influences the form of the model and the loss function. Predicting a continuous real value is different from choosing among discrete categories, and both differ from generating a structured object such as a sentence or image.

1.3.3 Representations Matter

Two learning problems may be equivalent in principle but dramatically different in difficulty depending on representation. For instance, classifying handwritten digits from raw pixels is harder than classifying them from well-chosen geometric features. A major theme in machine learning is that *good representations make learning easier*.

This observation leads to a useful decomposition:

$$\text{raw object} \longrightarrow \text{representation} \longrightarrow \text{prediction}.$$

Classical machine learning often relied heavily on human-designed representations. Deep learning shifted much of the burden to the model itself, allowing representations to be learned jointly with the predictor.

[House Price Prediction] Suppose we want to predict the price of a house. The input might include area, number of bedrooms, age, location, and other features. Thus $\mathcal{X}$ could be a subset of $\mathbb{R}^d$, and $\mathcal{Y} = \mathbb{R}$. The learner seeks a function

$$f : \mathbb{R}^d \to \mathbb{R}$$

that maps features to a price estimate.

[Image Classification] Suppose we want to classify an image into one of K categories. Then the input space may be

$$\mathcal{X} \subseteq \mathbb{R}^{H \times W \times C},$$

and the output space is $\mathcal{Y} = \{1, \ldots, K\}$. The learner seeks a function that maps pixel arrays to labels.

[Language Modeling] In next-token prediction, the input is a sequence of previously observed tokens, and the output is a probability distribution over the next token. Here the model may be viewed as

$$f : \mathcal{V}^* \to \Delta(\mathcal{V}),$$

where $\mathcal{V}$ is a vocabulary, $\mathcal{V}^*$ denotes finite sequences over $\mathcal{V}$, and $\Delta(\mathcal{V})$ is the simplex of probability distributions on $\mathcal{V}$.

These examples differ greatly in data type and architecture, but they share a common mathematical skeleton: learning a map between spaces from observed examples.

1.4 Approximation, Error, and Loss

To say that one function approximates another requires a notion of closeness. In machine learning, this is usually encoded through a *loss function*.

[Loss Function] A loss function is a map

$$\ell : \mathcal{Y} \times \mathcal{Y} \to [0, \infty)$$

where $\ell(\hat{y}, y)$ measures the penalty incurred when predicting $\hat{y}$ while the true target is y.

The choice of loss depends on the task:

- squared loss: $\ell(\hat{y}, y) = (\hat{y} - y)^2$,
- absolute loss: $\ell(\hat{y}, y) = |\hat{y} - y|$,
- zero-one loss for classification:

$$\ell(\hat{y}, y) = \begin{cases} 0, & \hat{y} = y, \\ 1, & \hat{y} \neq y, \end{cases}$$

- cross-entropy or log loss for probabilistic classification.

The loss quantifies the quality of a prediction at a single point. To evaluate a function over a dataset or distribution, one aggregates the loss.

1.4.1 Empirical Error

Given training data $(x_i, y_i)_{i=1}^n$, the empirical error of a predictor f is

$$\widehat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

This is the average loss on the observed sample. It is computable from data and therefore central to training.

1.4.2 Population Error

What we ultimately care about, however, is not just performance on the training data but performance on future examples. If data are generated from some underlying distribution P on $\mathcal{X} \times \mathcal{Y}$, then the population risk is

$$R(f) = \mathbb{E}_{(X,Y)\sim P}\big[\ell(f(X), Y)\big].$$

This quantity measures the expected loss on fresh draws from the data-generating process.

The distinction between empirical and population error is fundamental. A model may fit the training sample perfectly while performing poorly on new data. This phenomenon is called *overfitting*. Much of machine learning is about finding predictors whose empirical success transfers to population success.

1.4.3 Approximation Error and Estimation Error

The approximation viewpoint suggests an important decomposition. Suppose the true target rule lies outside the chosen model class $\mathcal{H}$. Then even the best possible function in $\mathcal{H}$ will not match it perfectly. This mismatch is called *approximation error*. In addition, because we only observe finitely many examples, the learner may fail to identify even the best function within $\mathcal{H}$. This is *estimation error*.

A rough schematic decomposition is

$$\text{total prediction error} = \text{approximation error} + \text{estimation error} + \text{optimization error}.$$

The exact form depends on context, but the message is universal:

- if the model class is too simple, approximation error dominates;
- if the model class is too flexible relative to the data, estimation error may dominate;
- if the training algorithm is poor, optimization error may dominate.

This decomposition helps explain why model selection is delicate. A larger hypothesis class can represent more functions, reducing approximation error, but it may be harder to train and more prone to overfitting.

1.5 Supervised Learning as Approximation from Examples

The most direct instance of the function-approximation viewpoint is supervised learning.

[Supervised Learning] In supervised learning, one observes labeled examples

$$(x_1, y_1), \ldots, (x_n, y_n)$$

and seeks a function $f : \mathcal{X} \to \mathcal{Y}$ that predicts the output y from the input x.

The learner typically chooses f from a hypothesis class $\mathcal{H}$ by minimizing or approximately minimizing empirical loss:

$$\min_{f \in \mathcal{H}} \widehat{R}_n(f) = \min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

This is the basic form of *empirical risk minimization*. It will appear repeatedly throughout the book.

1.5.1 Regression

In regression, the output space is continuous, often $\mathcal{Y} = \mathbb{R}$ or $\mathbb{R}^m$. The objective is to predict a numerical quantity. Typical examples include forecasting temperature, estimating demand, or predicting a physical parameter from sensor data.

If squared loss is used, one seeks a function that makes predictions close to the target in Euclidean distance. Linear regression, polynomial regression, and neural network regression all fit within this framework.

1.5.2 Classification

In classification, the output space is discrete. The function assigns an input to one of finitely many categories. Examples include spam detection, disease diagnosis, sentiment analysis, and object recognition.

Although classification is often introduced as label prediction, many successful models instead produce scores or probabilities from which labels are derived. Thus even in classification, it is often useful to think of the model as approximating a richer target such as a conditional probability distribution.

1.5.3 Structured Prediction

Some tasks require outputs with internal structure: sequences, trees, graphs, or segmentations. Machine translation, speech recognition, image captioning, and semantic segmentation fall into this category. Here the output space is more complicated, but the same conceptual picture applies: learn a function from inputs to structured outputs from examples.

[Regression vs Classification] Suppose a hospital uses medical measurements to assist clinicians. Predicting a patient's blood glucose level is a regression problem because the output is numerical. Predicting whether a patient has a disease is a classification problem because the output is a category. In both cases, the learner maps feature vectors to outputs, but the output structure and loss functions differ.

1.6 Unsupervised and Self-Supervised Learning

The function-approximation viewpoint extends beyond supervised learning, but it requires some care.

In unsupervised learning, we are often given inputs without explicit labels:

$$x_1, \ldots, x_n.$$

At first this seems to fall outside the input-to-output picture. However, many unsupervised tasks can still be formulated in terms of learning functions. The key is that the target or supervision is implicit rather than externally provided.

1.6.1 Representation Learning

A common unsupervised goal is to learn a representation map

$$\phi : \mathcal{X} \to \mathcal{Z},$$

where $\mathcal{Z}$ is a latent or feature space capturing useful structure in the data. The representation may then support downstream tasks such as clustering, retrieval, or prediction.

Examples include principal component analysis, autoencoders, contrastive learning, and masked prediction methods. In each case, the learner constructs a function that transforms data into a more informative or compressed representation.

1.6.2 Density Modeling

Another unsupervised goal is to model the probability distribution of the data itself. A generative model may be viewed as learning a function that assigns likelihoods or produces samples. Later chapters will show that this often involves learning either a density

$$p_\theta(x),$$

a latent-variable decoder, a score function, or a conditional generator. Although the mathematical form is different from direct prediction, the central challenge remains the same: infer structure from data and represent it with a tractable model.

1.6.3 Self-Supervision

Modern deep learning frequently uses *self-supervised* objectives, where labels are generated automatically from the data. For instance:

- predict a masked word from its context,
- predict the next token in a sequence,
- predict one augmented view of an image from another.

In such cases, the model still learns a function from inputs to targets, but the targets are constructed from the data rather than provided by human annotation.

This is a useful reminder that "supervision" need not mean "human labels." What matters is that the learner has a training signal that defines what it means to approximate well.

1.7 Learning from Finite Data

Why is learning difficult? Because data are finite and functions are many.

Suppose we observe n examples. Unless the hypothesis class is extremely restricted, there will generally be many functions that fit the observed sample perfectly:

$$f(x_i) = y_i, \qquad i = 1, \ldots, n.$$

Some of these functions may behave sensibly on unseen inputs; others may be wildly unstable outside the training sample. The data alone do not tell us which one to choose.

This simple observation is one of the deepest in machine learning. It implies that successful learning requires more than matching the data. It requires assumptions, preferences, or structural constraints that guide the choice among many data-consistent solutions.

1.7.1 Interpolation Is Not Enough

A polynomial of sufficiently high degree can interpolate finitely many points exactly. Yet exact interpolation does not guarantee meaningful prediction between or beyond those points. Similarly, a highly flexible machine learning model can fit training data perfectly while generalizing poorly.

Thus the goal is not merely to *memorize* the sample but to discover regularities that persist beyond it.

1.7.2 Generalization as Extrapolation from Evidence

The learner is asked to make predictions on future inputs, often drawn from the same environment as the training data. This requires *generalization*: good performance on unseen examples. Generalization is possible only if the training

data encode information about persistent structure and if the learner is biased toward hypotheses that capture that structure.

The tension between fitting observed data and generalizing to unseen data will be one of the central themes of the book.

[Finite Data Do Not Uniquely Determine a Predictor] Let $\{(x_i, y_i)\}_{i=1}^n$ be a finite dataset with distinct inputs $x_i \in \mathcal{X}$. If the hypothesis class $\mathcal{H}$ is sufficiently rich, then there may exist many distinct functions $f \in \mathcal{H}$ such that

$$f(x_i) = y_i \quad \text{for all } i = 1, \ldots, n.$$

Hence fitting the training data alone does not determine a unique predictor.

Proof sketch. A finite dataset imposes only finitely many constraints on a candidate function. If the function class has many degrees of freedom, then these constraints may leave infinitely many possibilities open away from the observed inputs. Such functions agree on the training sample but differ elsewhere. Therefore data fitting alone is insufficient for selecting a unique predictor. □

This proposition may look obvious, but its consequences are profound. It explains why notions such as simplicity, regularity, smoothness, margin, sparsity, and architecture matter. They are not cosmetic additions; they are what make learning possible.

1.8 Hypothesis Classes and the Need for Structure

Since finitely many examples do not determine a predictor uniquely, we must restrict attention to a set of plausible candidate functions.

[Hypothesis Class] A hypothesis class $\mathcal{H}$ is a set of candidate functions

$$f : \mathcal{X} \to \mathcal{Y}$$

from which the learner chooses.

Examples include:

- linear functions $f(x) = w^\top x + b$,
- polynomial functions up to a fixed degree,
- decision trees of bounded depth,
- neural networks of a specified architecture,
- kernel methods with a chosen feature map.

The choice of $\mathcal{H}$ expresses assumptions about the task. A linear class assumes that the relationship between input and output can be captured by a linear rule in the chosen features. A convolutional neural network assumes spatial locality and weight sharing are useful. A transformer assumes attention-based interactions among tokens are useful. In all cases, the hypothesis class encodes *inductive bias*, the preferences that guide learning beyond the observed data.

1.8.1 Too Small and Too Large Classes

There is a delicate balance in choosing $\mathcal{H}$.

- If $\mathcal{H}$ is too small, it may not contain any function close to the target rule. Then approximation error is large.
- If $\mathcal{H}$ is too large, it may contain many functions that fit the sample but generalize poorly. Then estimation becomes difficult.

This tension is one of the foundational tradeoffs of machine learning. Much of model design is about choosing a hypothesis class expressive enough to capture important structure but constrained enough to learn reliably from available data.

1.8.2 Architecture as Structured Approximation

Modern deep learning can be seen as a sophisticated way of designing rich yet structured hypothesis classes. Different architectures correspond to different assumptions:

- multilayer perceptrons favor generic composition,
- convolutional networks favor locality and translation structure,
- recurrent networks favor sequential state evolution,
- transformers favor content-based interactions across positions.

These are not arbitrary engineering choices. They are forms of prior structure that shape what kinds of functions the model can represent efficiently and what patterns it can learn from finite data.

1.9 Worked Examples of the Approximation View

We now examine several tasks through the lens of function approximation.

1.9.1 Example 1: Polynomial Curve Fitting

Suppose we observe data points (x_i, y_i) with $x_i \in \mathbb{R}$ and want to model the dependence of y on x. One possible hypothesis class is the set of polynomials of degree at most m:

$$\mathcal{H}_m = \{f(x) = a_0 + a_1 x + \cdots + a_m x^m\}.$$

If m is small, the class may be too rigid and fail to capture the trend. If m is very large, the fitted polynomial may oscillate sharply between observed points. Thus increasing expressive power helps approximation but may hurt generalization.

This example illustrates several recurring themes:

- choosing a function class,
- fitting parameters to data,
- trading flexibility against stability,
- distinguishing interpolation from useful prediction.

1.9.2 Example 2: Image Classification as a Function on High-Dimensional Space

An image with height H, width W, and C channels may be represented as a vector in $\mathbb{R}^{HWC}$. A classifier is then a function

$$f : \mathbb{R}^{HWC} \to \{1, \ldots, K\}.$$

At this level of abstraction, image classification is just another function approximation problem. However, the ambient dimension is huge, and naive function classes are hopelessly inefficient. The success of convolutional networks comes from imposing structure adapted to images: locality, translation equivariance, and hierarchical feature extraction.

Thus the approximation viewpoint is not invalidated by modern deep learning; rather, it becomes richer. The main question is no longer only *whether* we approximate functions, but *which kinds of function classes make the approximation statistically and computationally feasible.*

1.9.3 Example 3: Sequence Prediction

Let $x = (x_1, \ldots, x_t)$ be a token sequence. In next-token prediction, the model outputs a distribution over possible continuations:

$$f(x_1, \ldots, x_t) \approx P(X_{t+1} \mid X_1 = x_1, \ldots, X_t = x_t).$$

This is still a function approximation problem, but the target is probabilistic and the domain contains variable-length sequences. The hypothesis class must respect this structure. Recurrent networks and transformers are different ways of constructing such classes.

1.10 Beyond Deterministic Functions

The function-approximation viewpoint is extremely useful, but it has limitations if interpreted too literally.

1.10.1 Noise and Ambiguity

In many real problems, the same input may correspond to multiple plausible outputs. A single deterministic function may not capture this well. For instance:

- two patients with similar features may have different outcomes,
- a sentence may admit multiple valid translations,
- a text prompt may correspond to many plausible images.

In such cases, it is often better to model conditional distributions rather than point predictions. Later chapters will generalize the approximation perspective from deterministic functions $f(x)$ to probabilistic models such as $p(y \mid x)$ or generative models over x itself.

1.10.2 Decision Making vs Prediction

Some applications require decisions rather than predictions. A medical system might recommend treatment, not just estimate disease probability. A recommendation engine may optimize engagement under constraints. A robot must choose actions. In these settings, the learned object may be a policy or decision rule rather than a pure predictor.

Still, the basic mathematical spirit survives: construct a mapping from observations to outputs that performs well under a task-specific objective.

1.10.3 Why the Function View Still Matters

Even when machine learning extends beyond deterministic prediction, the function viewpoint remains foundational because it gives:

- a common abstraction across tasks,
- a language for approximation and error,
- a bridge to optimization,
- a way to formalize representation and architecture,
- a basis for understanding generalization.

For this reason, it is one of the best conceptual entry points into the field.

1.11 A Unifying Perspective

Machine learning can be viewed as the art and science of constructing useful mappings from data. Whether we predict real values, classify images, translate sentences, generate text, or learn latent representations, we are searching for structured rules that transform inputs into outputs in a way that extends beyond the observed sample.

The approximation view unifies these tasks, but it also reveals the fundamental challenges:

- the target rule is unknown,

- the data are finite,
- many candidate functions fit the observations,
- different choices of representation and hypothesis class lead to different behaviors,
- success depends on both expressivity and structure.

Thus learning is not just approximation. It is approximation under uncertainty, guided by data and constrained by inductive bias.

A helpful slogan is:

$$\text{learning} = \text{approximation} + \text{data} + \text{structure}.$$

Approximation alone explains how models represent relationships. Data explain how these relationships are inferred. Structure explains why good generalization is possible at all.

1.12 Summary

In this chapter we introduced one of the central organizing ideas of machine learning: learning as function approximation. We formulated a learning problem in terms of input space, output space, hypothesis class, data, and performance criterion. We saw how regression, classification, structured prediction, representation learning, and sequence modeling all fit into this perspective.

We emphasized that the learner does not observe the target rule directly, but only finite samples. This makes learning fundamentally underdetermined unless one imposes structure through a hypothesis class and inductive bias. We distinguished empirical performance from population performance and introduced the rough decomposition of total error into approximation, estimation, and optimization components.

This chapter sets the stage for the rest of the book. The next chapter will add a probabilistic view of data and prediction, making precise how uncertainty enters learning and how losses and expectations formalize the notion of a good predictor.

1.13 Exercises

1. For each of the following tasks, specify a plausible input space $\mathcal{X}$ and output space $\mathcal{Y}$:
   (a) predicting a student's exam score from study habits,
   (b) classifying an email as spam or not spam,
   (c) translating an English sentence into French,
   (d) predicting tomorrow's temperature from weather measurements.

2. Explain in your own words the difference between a target rule $f^\star$ and a learned model f.

3. Give an example of two different functions that agree on a finite set of training points but differ elsewhere. What does this illustrate about learning from finite data?

4. Consider the hypothesis class of all linear functions $f(x) = w^\top x + b$ on $\mathbb{R}^d$. Describe one situation in which this class may be too restrictive and one in which it may be appropriate.

5. Why is interpolation of the training data not enough to guarantee good prediction on unseen examples?

6. Let $\ell(\hat{y}, y) = (\hat{y} - y)^2$. Write the empirical risk for a dataset $(x_i, y_i)_{i=1}^n$ and a predictor f.

7. Give an example of a machine learning task where the output should be viewed as a probability distribution rather than a single deterministic label.

8. In image classification, why might the choice of representation or architecture matter even if both models are universal approximators in principle?

9. Compare the following two goals:

   (a) learning a function from house features to price,

   (b) learning a representation of houses useful for later prediction tasks.

   How are they similar, and how are they different?

10. Suppose a model class is made larger and more expressive. What effect might this have on approximation error? What effect might it have on estimation error?

11. Explain the difference between supervised, unsupervised, and self-supervised learning using the function-approximation viewpoint.

12. Give a real-world example where the same input may correspond to multiple plausible outputs. Why does this suggest that a probabilistic model may be preferable to a deterministic one?

13. Consider polynomial fitting with degree m.

    (a) What may happen if m is too small?

    (b) What may happen if m is too large?

    Relate your answer to approximation and generalization.

14. Let $\mathcal{H}$ be a hypothesis class and suppose $f_1, f_2 \in \mathcal{H}$ both satisfy

$$f_1(x_i) = f_2(x_i) = y_i \quad \text{for } i = 1, \ldots, n.$$

Why might it still matter which of the two functions is chosen?

15. Write a short paragraph explaining the slogan

$$\text{learning} = \text{approximation} + \text{data} + \text{structure}.$$

Historical Note

The idea that learning can be viewed as approximation has roots in several mathematical traditions: approximation theory, statistics, control, and pattern recognition. Classical regression already embodies the search for a function that fits observed data. Modern machine learning broadened this picture by considering richer function classes, large-scale optimization, learned representations, and highly structured architectures. Deep learning did not abandon the approximation viewpoint; rather, it made it far more expressive by learning complicated compositional maps from data at scale.

Chapter 2

A Probabilistic View of Learning

2.1 Introduction

In Chapter 1, we introduced learning as the problem of constructing a useful mapping from inputs to outputs based on data. That viewpoint is broad and powerful, but it leaves an important question unresolved: where do the data come from? Why should performance on a finite sample tell us anything about future performance? Why do some prediction rules make sense under uncertainty while others do not?

Probability provides the language in which these questions can be made precise. In machine learning, data are not usually treated as isolated observations. Rather, they are modeled as realizations of random variables generated by an unknown process. A learning algorithm receives a sample from this process and attempts to infer a rule that performs well on future observations drawn in a similar way. This probabilistic viewpoint allows us to define prediction error mathematically, distinguish training performance from true performance, and characterize optimal predictors under different loss functions.

This chapter develops the statistical learning setup carefully. We begin by treating data as samples from an unknown distribution on input–output pairs. We then define loss functions, expected risk, and empirical risk. These concepts formalize the idea that learning is not just about fitting past data, but about minimizing expected future error. We next study conditional distributions and derive the form of optimal predictors for common losses. This leads naturally to the notions of conditional expectation, conditional median, and Bayes decision rules. We also explain the role of noise, ambiguity, and irreducible uncertainty, which show why perfect prediction is often impossible even in principle.

Finally, we discuss two important interpretive frameworks: the frequentist and Bayesian viewpoints. These are not merely philosophical labels. They reflect different ways of representing uncertainty and different ways of under-

standing what a model parameter means. Both perspectives play an important role in modern machine learning.

The main goal of this chapter is to move from the informal language of "predicting from data" to a mathematically precise statistical framework. This framework will serve as the foundation for nearly everything that follows: generalization theory, empirical risk minimization, probabilistic modeling, neural network training, and generative modeling.

2.2 Data as Samples from an Unknown Distribution

A central assumption in statistical learning is that the data we observe are generated according to some underlying probability distribution. This assumption does not mean that the world is simple or that the distribution is known. On the contrary, the distribution is usually highly complex and entirely unknown. The point is that probability gives us a way to model variability, uncertainty, and repeated sampling.

2.2.1 Random Variables and Observations

Let X be a random variable taking values in an input space $\mathcal{X}$, and let Y be a random variable taking values in an output space $\mathcal{Y}$. The pair (X, Y) is assumed to have some joint distribution P on $\mathcal{X} \times \mathcal{Y}$.

A dataset of size n is modeled as a sample

$$(X_1, Y_1), (X_2, Y_2), \ldots, (X_n, Y_n),$$

typically assumed to be independent and identically distributed (i.i.d.) according to P:

$$(X_i, Y_i) \sim P, \qquad i = 1, \ldots, n.$$

The i.i.d. assumption is not universally valid, but it is the basic starting point for much of statistical learning theory.

[Statistical Learning Setup] A statistical learning problem consists of a pair of random variables (X, Y) with unknown joint distribution P on $\mathcal{X} \times \mathcal{Y}$, together with a hypothesis class $\mathcal{H}$ of functions $f : \mathcal{X} \to \mathcal{A}$, where $\mathcal{A}$ is the prediction space, and a loss function

$$\ell : \mathcal{A} \times \mathcal{Y} \to [0, \infty).$$

The goal is to choose a predictor $f \in \mathcal{H}$ that performs well under the distribution P.

Here $\mathcal{A}$ is the set of possible predictions. In regression, one often has $\mathcal{A} = \mathbb{R}$. In classification, $\mathcal{A}$ may be a finite label set or a probability simplex over labels.

2.2.2 Why Distributions Matter

The probabilistic setup allows us to formalize a crucial idea: the learner is not judged only on the finite training sample, but on future examples generated from the same underlying process. That is why the distribution P matters. It determines what kinds of inputs are likely, how outputs relate to inputs, and what uncertainty is intrinsic to the problem.

A single finite sample provides only partial information about P. Learning, therefore, is the problem of drawing reliable conclusions about future performance from incomplete evidence.

2.2.3 Marginals, Conditionals, and the Joint Law

The joint distribution P can be understood through:

- the marginal distribution of X,
- the marginal distribution of Y,
- the conditional distribution of Y given X.

In supervised learning, the conditional law of Y given X is especially important. It describes how the output behaves for a given input. In many tasks, the ideal predictor is determined entirely by this conditional distribution.

For example:

- in regression with squared loss, the optimal predictor is the conditional mean,
- in regression with absolute loss, the optimal predictor is the conditional median,
- in classification with zero-one loss, the optimal predictor is the most probable class.

These facts reveal that prediction is fundamentally about conditional probability.

[Medical Prediction] Suppose X represents patient features and Y represents a diagnosis or outcome. Then the learning problem is governed not only by the overall frequency of the disease, but by the conditional distribution of Y given X. Two patients may have different risks because their feature vectors correspond to different conditional distributions.

2.3 Loss Functions and Risk

To judge the quality of a prediction, we need a quantitative notion of error. This is supplied by a loss function.

[Loss Function] A loss function is a map

$$\ell : \mathcal{A} \times \mathcal{Y} \to [0, \infty)$$

such that $\ell(a, y)$ measures the penalty incurred by predicting a when the true output is y.

The choice of loss depends on the task and on what kinds of mistakes matter.

2.3.1 Common Loss Functions

Some standard examples are:

- **Squared loss:**
$$\ell(a, y) = (a - y)^2.$$

- **Absolute loss:**
$$\ell(a, y) = |a - y|.$$

- **Zero-one loss for classification:**
$$\ell(a, y) = \begin{cases} 0, & a = y, \\ 1, & a \neq y. \end{cases}$$

- **Log loss / cross-entropy:** if a is a probability distribution over classes and y is the true class, then
$$\ell(a, y) = -\log a(y).$$

Different loss functions lead to different notions of optimality. A predictor that is optimal under squared loss need not be optimal under absolute loss or zero-one loss.

2.3.2 Population Risk

The most important quantity in statistical learning is expected loss under the data-generating distribution.

[Population Risk] Let $(X, Y) \sim P$, and let $f : \mathcal{X} \to \mathcal{A}$ be a predictor. The population risk of f is

$$R(f) = \mathbb{E}[\ell(f(X), Y)].$$

This quantity represents the average loss the predictor would incur on future data drawn from the same distribution.

Population risk is the true performance criterion. It is what we actually care about. However, it is usually not directly computable because the distribution P is unknown.

2.3.3 Empirical Risk

Given a sample $(X_i, Y_i)_{i=1}^n$, the empirical risk of f is

$$\widehat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(X_i), Y_i).$$

Unlike population risk, this quantity is observable. Learning algorithms often minimize empirical risk or a regularized version of it.

The distinction between these two risks is fundamental:

- *population risk* measures true expected future performance;
- *empirical risk* measures performance on the observed sample.

One of the main questions of learning theory is when small empirical risk implies small population risk.

2.3.4 Risk Minimization

In an ideal world, if we knew the distribution P, we would seek a predictor minimizing population risk:

$$f^\star \in \arg\min_f R(f).$$

Such a predictor is often called a *Bayes predictor* or *Bayes decision rule*, depending on the context.

In practice, we usually restrict attention to a hypothesis class $\mathcal{H}$ and instead try to solve

$$\min_{f \in \mathcal{H}} R(f),$$

or more realistically, we minimize empirical risk:

$$\min_{f \in \mathcal{H}} \widehat{R}_n(f).$$

Thus statistical learning can be viewed as the attempt to approximate the population risk minimizer using only a finite sample.

2.4 Conditional Distributions and Optimal Prediction

The conditional distribution of Y given $X = x$ encodes the predictive structure of the problem. It tells us, for each input x, how the output behaves probabilistically. Many optimal prediction rules can be derived by minimizing conditional expected loss pointwise in x.

2.4.1 Conditional Risk

Let f be a predictor. Then its population risk can be decomposed as

$$R(f) = \mathbb{E}\big[\ell(f(X), Y)\big] = \mathbb{E}\Big[\mathbb{E}\big[\ell(f(X), Y) \mid X\big]\Big].$$

This is an instance of the tower property of conditional expectation.

For a fixed input value x, define the conditional risk of predicting $a \in \mathcal{A}$ by

$$r_x(a) = \mathbb{E}[\ell(a, Y) \mid X = x].$$

Then the overall risk of a predictor f is

$$R(f) = \mathbb{E}[r_X(f(X))].$$

This leads to a fundamental principle:

[Pointwise Risk Minimization] Suppose for each $x \in \mathcal{X}$ we choose a prediction $f^\star(x)$ satisfying

$$f^\star(x) \in \arg\min_{a \in \mathcal{A}} \mathbb{E}[\ell(a, Y) \mid X = x].$$

Then $f^\star$ minimizes the population risk over all measurable predictors.

Proof sketch. For any predictor f, one has

$$\mathbb{E}[\ell(f(X), Y) \mid X = x] \geq \inf_{a \in \mathcal{A}} \mathbb{E}[\ell(a, Y) \mid X = x]$$

for each x. Integrating over the distribution of X yields

$$R(f) \geq R(f^\star).$$

Thus minimizing conditional risk pointwise also minimizes total population risk. □

This proposition is conceptually central. It shows that the Bayes-optimal predictor is determined by the conditional distribution of Y given X.

2.5 Conditional Expectation and Optimal Prediction Under Squared Loss

We now derive one of the most important results in statistics and machine learning: under squared loss, the optimal predictor is the conditional mean.

[Bayes Optimal Predictor Under Squared Loss] Suppose $\mathcal{Y} \subseteq \mathbb{R}$ and the loss is

$$\ell(a, y) = (a - y)^2.$$

Then the predictor that minimizes population risk over all measurable functions is

$$f^\star(x) = \mathbb{E}[Y \mid X = x].$$

Proof. Fix x and consider the conditional risk

$$r_x(a) = \mathbb{E}\big[(a - Y)^2 \mid X = x\big].$$

Let

$$m(x) = \mathbb{E}[Y \mid X = x].$$

We expand:

$$(a - Y)^2 = (a - m(x) + m(x) - Y)^2.$$

Thus

$$(a - Y)^2 = (a - m(x))^2 + 2(a - m(x))(m(x) - Y) + (m(x) - Y)^2.$$

Taking conditional expectation given $X = x$ yields

$$r_x(a) = (a - m(x))^2 + 2(a - m(x))\mathbb{E}[m(x) - Y \mid X = x] + \mathbb{E}[(m(x) - Y)^2 \mid X = x].$$

Since $m(x) = \mathbb{E}[Y \mid X = x]$, we have

$$\mathbb{E}[m(x) - Y \mid X = x] = m(x) - \mathbb{E}[Y \mid X = x] = 0.$$

Therefore

$$r_x(a) = (a - m(x))^2 + \mathbb{E}[(m(x) - Y)^2 \mid X = x].$$

The second term does not depend on a, so $r_x(a)$ is minimized when

$$a = m(x) = \mathbb{E}[Y \mid X = x].$$

By pointwise risk minimization, the optimal predictor is

$$f^\star(x) = \mathbb{E}[Y \mid X = x].$$

□

2.5.1 Interpretation

This theorem explains why regression models trained with squared loss aim to estimate a conditional mean. Even when the data are noisy, the best squared-error predictor is not the noiseless underlying mechanism in any absolute sense; it is the average response conditioned on the input.

This also clarifies an important modeling issue. If the conditional distribution of Y given $X = x$ is multimodal, then the conditional mean may not be a typical outcome. For example, if a text prompt admits several very different plausible images, the mean of all such images may look blurry and unrealistic. This is one reason why mean prediction is not always a good generative strategy, even though it is optimal under squared loss.

2.5.2 Bias–Variance Style Decomposition

The proof above also gives a decomposition:

$$\mathbb{E}[(a - Y)^2 \mid X = x] = (a - \mathbb{E}[Y \mid X = x])^2 + \mathrm{Var}(Y \mid X = x).$$

This shows that the conditional variance is the unavoidable residual error at input x. Even the Bayes-optimal predictor cannot reduce this part.

2.6 Conditional Median and Optimal Prediction Under Absolute Loss

The optimal predictor changes if we change the loss function. Under absolute loss, the correct target is no longer the conditional mean but the conditional median.

[Conditional Median] A value $m(x)$ is called a conditional median of Y given $X = x$ if

$$\mathbb{P}(Y \leq m(x) \mid X = x) \geq \frac{1}{2} \qquad \text{and} \qquad \mathbb{P}(Y \geq m(x) \mid X = x) \geq \frac{1}{2}.$$

[Bayes Optimal Predictor Under Absolute Loss] Suppose $\mathcal{Y} \subseteq \mathbb{R}$ and the loss is

$$\ell(a, y) = |a - y|.$$

Then any conditional median of Y given $X = x$ minimizes the conditional expected loss

$$\mathbb{E}[|a - Y| \mid X = x].$$

Hence a Bayes-optimal predictor under absolute loss is any conditional median.

Proof sketch. Fix x and define

$$r_x(a) = \mathbb{E}[|a - Y| \mid X = x].$$

The function $r_x(a)$ is convex in a. When differentiable, its derivative is

$$\frac{d}{da} r_x(a) = \mathbb{P}(Y \leq a \mid X = x) - \mathbb{P}(Y \geq a \mid X = x),$$

up to technicalities at atoms. A minimizer therefore occurs when the probability mass on either side of a is at least one-half, which is exactly the definition of a median. Thus any conditional median minimizes conditional expected absolute deviation. □

2.6.1 Interpretation

The conditional median is more robust to outliers than the conditional mean. Squared loss penalizes large deviations heavily, so it is sensitive to rare extreme values. Absolute loss grows only linearly, so it is less strongly affected by outliers.

This is a recurring theme: the loss function determines what feature of the conditional distribution is being estimated.

[Mean vs Median] Suppose that given a certain input x, the output Y equals 0 with probability 0.9 and equals 100 with probability 0.1. Then

$$\mathbb{E}[Y \mid X = x] = 10,$$

while the conditional median is 0. The mean is optimal under squared loss, but the median is optimal under absolute loss.

2.7 Bayes Classification Under Zero-One Loss

Classification provides another important example of Bayes-optimal prediction. Here the output space is finite, say

$$\mathcal{Y} = \{1, 2, \ldots, K\},$$

and the loss is zero-one loss:

$$\ell(a, y) = \begin{cases} 0, & a = y, \\ 1, & a \neq y. \end{cases}$$

[Bayes Classifier Under Zero-One Loss] Under zero-one loss, a predictor $f^\star$ minimizing population risk is given by

$$f^\star(x) \in \arg\max_{k \in \{1,\ldots,K\}} \mathbb{P}(Y = k \mid X = x).$$

That is, the Bayes classifier chooses the most probable class given the input.

Proof. Fix x and consider predicting class a. The conditional risk is

$$r_x(a) = \mathbb{E}[\ell(a, Y) \mid X = x].$$

Since zero-one loss is 1 unless $a = Y$, we have

$$r_x(a) = 1 - \mathbb{P}(Y = a \mid X = x).$$

Thus minimizing $r_x(a)$ is equivalent to maximizing $\mathbb{P}(Y = a \mid X = x)$. Therefore any most probable class is conditionally optimal. By pointwise risk minimization, this yields a population risk minimizer. □

2.7.1 Bayes Error Rate

Even the Bayes classifier may make errors, because the label may be intrinsically uncertain given the input. The minimum achievable classification error is called the *Bayes error rate*. At input x, the conditional Bayes error is

$$1 - \max_k \mathbb{P}(Y = k \mid X = x).$$

Averaging over X yields the total Bayes error rate.

This quantity represents irreducible uncertainty in the classification problem. If two classes overlap in feature space, then no classifier can separate them perfectly.

2.8 Noise, Uncertainty, and Irreducible Error

One of the most important lessons of the probabilistic viewpoint is that imperfect prediction may be unavoidable even with infinite data and unlimited computation.

2.8.1 Aleatoric Uncertainty

Some uncertainty is intrinsic to the data-generating process. This is often called *aleatoric uncertainty*. It reflects randomness, ambiguity, or unobserved factors. For example:

- two patients with the same measured features may have different outcomes,
- stock prices may fluctuate due to unpredictable events,
- multiple translations of a sentence may all be valid.

Aleatoric uncertainty cannot be eliminated by collecting more data about the same variables alone. It is built into the conditional distribution of Y given X.

2.8.2 Irreducible Error in Regression

Under squared loss, the Bayes-optimal predictor is the conditional mean, and the minimum conditional risk is

$$\mathbb{E}\Big[\big(Y - \mathbb{E}[Y \mid X]\big)^2 \,\Big|\, X\Big] = \mathrm{Var}(Y \mid X).$$

Averaging over X, the minimum achievable population risk is

$$\mathbb{E}[\mathrm{Var}(Y \mid X)].$$

This term represents the irreducible error.

2.8.3 Irreducible Error in Classification

Under zero-one loss, the irreducible error is the Bayes error rate:

$$R^\star = \mathbb{E}\Big[1 - \max_k \mathbb{P}(Y = k \mid X)\Big].$$

Again, this is the error of the best possible classifier, not just the best classifier in a restricted hypothesis class.

2.8.4 Epistemic Uncertainty

In addition to aleatoric uncertainty, there is uncertainty due to limited knowledge. This is often called *epistemic uncertainty*. It arises because the learner does not know the true distribution P and only sees finitely many examples. Unlike aleatoric uncertainty, epistemic uncertainty can in principle be reduced by more data, better models, or stronger prior knowledge.

This distinction is useful:

- aleatoric uncertainty is inherent in the problem;
- epistemic uncertainty reflects incomplete knowledge of the problem.

[Noisy Regression Model] Suppose

$$Y = g(X) + \varepsilon,$$

where ε is random noise with

$$\mathbb{E}[\varepsilon \mid X] = 0.$$

Then

$$\mathbb{E}[Y \mid X] = g(X),$$

so the conditional mean recovers the signal. But unless $\varepsilon = 0$ almost surely, even the best predictor incurs nonzero error because of the noise term.

2.9 Deterministic and Probabilistic Prediction Rules

The results above show that optimal point predictions depend on the choice of loss. But there are situations where a single deterministic prediction is not enough. In such cases, it is often more informative to predict a full conditional distribution.

2.9.1 Point Prediction

A deterministic predictor outputs a single value

$$f(x) \in \mathcal{A}.$$

Examples:

- a predicted house price,
- a single label,
- a single next token chosen greedily.

Point prediction is appropriate when the task requires a single action or estimate, or when the uncertainty is mild.

2.9.2 Probabilistic Prediction

A probabilistic predictor outputs a distribution

$$q(\cdot \mid x)$$

over possible outputs. This can express uncertainty, multimodality, and confidence.

Examples:

- class probabilities in classification,
- predictive intervals in regression,
- next-token distributions in language modeling,
- distributions over future trajectories in forecasting.

Probabilistic prediction is often strictly more informative than deterministic prediction. A deterministic rule can be obtained from a probabilistic predictor by taking a mean, median, mode, or decision that minimizes expected loss.

2.9.3 Why Probabilities Matter

A model that outputs only a class label may hide the difference between:

- a case where one class has probability 0.99,
- a case where it has probability 0.51.

The predicted label is the same, but the uncertainty is very different.

In high-stakes domains such as medicine, finance, and scientific prediction, this distinction is often crucial.

2.10 Bayesian and Frequentist Interpretations

The probabilistic language of learning admits multiple interpretations. Two of the most important are the frequentist and Bayesian viewpoints.

2.10.1 Frequentist View

In the frequentist interpretation:

- the data are random,
- the parameters of the model are fixed but unknown,
- probability describes long-run variability of sampling.

For example, in linear regression one may write

$$Y = X^\top \theta + \varepsilon,$$

where θ is an unknown fixed parameter vector. Estimation seeks to infer θ from random data.

From this perspective, uncertainty statements concern the behavior of estimators across repeated samples. Confidence intervals and sampling distributions are frequentist tools.

2.10.2 Bayesian View

In the Bayesian interpretation:

- unknown quantities such as parameters are themselves treated probabilistically,
- prior beliefs are encoded in a prior distribution,
- observed data update the prior to a posterior distribution.

If θ denotes a model parameter, Bayes' rule gives

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)\, p(\theta),$$

where $\mathcal{D}$ is the dataset.

From the posterior one can derive predictive distributions:

$$p(y \mid x, \mathcal{D}) = \int p(y \mid x, \theta)\, p(\theta \mid \mathcal{D})\, d\theta.$$

This approach naturally quantifies uncertainty due to limited data and allows prior knowledge to be incorporated in a principled way.

2.10.3 Comparing the Two Views

The frequentist and Bayesian viewpoints often lead to similar practical procedures, but they differ conceptually.

Frequentist emphasis. How does an estimator behave across repeated sampling from the population?

Bayesian emphasis. Given the observed data, what should we believe about unknown quantities?

Neither framework exhausts all of machine learning, but both are fundamental. Classical statistics developed many frequentist tools, while modern probabilistic machine learning often incorporates Bayesian ideas, especially in uncertainty quantification, latent-variable modeling, and probabilistic inference.

2.10.4 A Common Ground

Despite their differences, both perspectives share the same basic statistical learning setup:

- data arise from a probabilistic mechanism,
- prediction quality is judged by expected loss,
- future performance depends on the underlying data-generating process.

Thus the probabilistic view of learning is broader than the Bayesian/frequentist divide. It is the common framework within which both can be understood.

2.11 Worked Derivations and Illustrative Examples

2.11.1 Why the Conditional Mean Minimizes Squared Loss

Let a be a candidate prediction at input x. Then

$$\mathbb{E}[(Y-a)^2 \mid X=x]$$

is minimized at

$$a = \mathbb{E}[Y \mid X=x].$$

The proof was given earlier, but its meaning is worth repeating: squared loss asks the predictor to aim for the center of mass of the conditional distribution.

2.11.2 Why the Conditional Median Minimizes Absolute Loss

Similarly,

$$\mathbb{E}[|Y - a| \mid X = x]$$

is minimized at a conditional median. Absolute loss asks the predictor to balance the probability mass on either side of the prediction.

2.11.3 Classification as Choosing the Most Likely Class

Under zero-one loss, the expected conditional error of predicting class k is

$$1 - \mathbb{P}(Y = k \mid X = x),$$

so the best class is simply the one with highest conditional probability.

[Three Different Optimal Predictors] Suppose that for a fixed input x, the conditional distribution of Y is:

$$Y = \begin{cases} 0, & \text{with probability } 0.4, \\ 1, & \text{with probability } 0.2, \\ 10, & \text{with probability } 0.4. \end{cases}$$

Then:

- the conditional mean is

  $$\mathbb{E}[Y \mid X = x] = 0(0.4) + 1(0.2) + 10(0.4) = 4.2,$$

  so this is optimal under squared loss;
- a conditional median can be any value in $[1, 10]$ only if the cumulative distribution permits it, but here the smallest median is 1 and the set of medians collapses to values where cumulative mass reaches one-half; in this discrete case, 1 is a median;
- if the outputs were treated as class labels, then the most probable class would be tied between 0 and 10.

Thus the optimal prediction depends fundamentally on the loss and on how the problem is formulated.

2.12 A Unifying Perspective

The probabilistic view of learning adds a crucial layer to the function-approximation viewpoint developed in Chapter 1. Instead of thinking of data as arbitrary examples, we model them as realizations from an unknown distribution. Instead of measuring performance only on a sample, we evaluate predictors by expected loss under that distribution. Instead of speaking vaguely about "best prediction," we derive optimal rules from conditional distributions and loss functions.

This framework leads to several powerful ideas:

- learning is about minimizing expected risk, not just fitting observed data;
- optimal prediction depends on the loss function;
- conditional expectation, conditional median, and class probabilities are not arbitrary quantities but Bayes-optimal targets under different losses;
- some uncertainty is irreducible, because the output may remain random even after conditioning on the input;
- probabilistic modeling provides the bridge between observed data and principled prediction.

A useful slogan is:

$$\text{prediction} = \text{decision under uncertainty}.$$

The role of probability is to describe that uncertainty; the role of the loss function is to specify what counts as a good decision.

2.13 Summary

In this chapter we introduced the statistical learning framework. We modeled data as samples from an unknown joint distribution of random variables (X, Y). We defined loss functions, population risk, and empirical risk, and emphasized that population risk is the true target of learning.

We then showed how the conditional distribution of Y given X determines optimal prediction. Under squared loss, the Bayes-optimal predictor is the conditional mean. Under absolute loss, it is a conditional median. Under zero-one loss, the Bayes classifier chooses the most probable class. These results illustrate that "optimal prediction" depends on the chosen loss and on the underlying uncertainty structure.

We also discussed noise and irreducible error, distinguishing uncertainty inherent in the problem from uncertainty due to limited knowledge. Finally, we compared Bayesian and frequentist interpretations of the probabilistic setup.

This chapter provides the probabilistic backbone for the rest of the book. The next chapter will examine hypothesis spaces and inductive bias, explaining why data alone do not determine a predictor and why structural assumptions are essential for generalization.

2.14 Exercises

1. Let (X, Y) be a random pair with loss function ℓ. Write the definition of population risk and explain in words what it means.
2. Let $(X_i, Y_i)_{i=1}^n$ be a dataset. Write the empirical risk of a predictor f and compare it with the population risk.

3. Suppose $Y \in \mathbb{R}$ and the loss is squared loss. Show that for a fixed random variable Y, the quantity
$$\mathbb{E}[(Y-a)^2]$$
is minimized over $a \in \mathbb{R}$ at $a = \mathbb{E}[Y]$.

4. Extend the previous exercise to the conditional setting and show that
$$f^\star(x) = \mathbb{E}[Y \mid X = x]$$
minimizes the conditional expected squared loss.

5. Suppose the loss is absolute loss. Prove, or give a careful argument, that a median minimizes expected absolute deviation.

6. For each of the following losses, identify the Bayes-optimal predictor:
   (a) squared loss,
   (b) absolute loss,
   (c) zero-one loss.

7. Let $\mathcal{Y} = \{1, 2, 3\}$. Suppose that for some fixed x,
$$\mathbb{P}(Y = 1 \mid X = x) = 0.2, \qquad \mathbb{P}(Y = 2 \mid X = x) = 0.5, \qquad \mathbb{P}(Y = 3 \mid X = x) = 0.3.$$
What is the Bayes classifier at x under zero-one loss? What is the conditional Bayes error at x?

8. Give an example where the conditional mean and conditional median are different. Explain why this happens.

9. Suppose
$$Y = g(X) + \varepsilon$$
with $\mathbb{E}[\varepsilon \mid X] = 0$. Show that
$$\mathbb{E}[Y \mid X] = g(X).$$

10. Show that under squared loss, the minimum achievable conditional risk at input x is
$$\mathrm{Var}(Y \mid X = x).$$

11. Explain the difference between aleatoric and epistemic uncertainty. Give one example of each.

12. Consider a classifier that outputs only the predicted class label and another classifier that outputs a full vector of class probabilities. Why might the second be more informative?

13. In your own words, explain why the optimal predictor depends on the loss function.

14. Let Y be a Bernoulli random variable with

$$\mathbb{P}(Y = 1 \mid X = x) = p(x).$$

Under zero-one loss, what is the optimal classifier? Under squared loss for predicting $Y \in \{0, 1\}$ numerically, what is the optimal predictor?

15. Compare the frequentist and Bayesian interpretations of the statement "the parameter θ is uncertain." What does this mean in each framework?

16. A model predicts the same class label for two different inputs, but in one case its estimated probability is 0.99 and in the other case 0.55. Why might these two predictions be treated differently in practice?

17. Suppose that for a fixed input x, the conditional distribution of Y is symmetric around a value m. Show that m is both the conditional mean and a conditional median.

18. Let $f^\star$ be a Bayes-optimal predictor over all measurable functions, and let $\mathcal{H}$ be a restricted hypothesis class. Explain why the best predictor in $\mathcal{H}$ may still have larger risk than $f^\star$.

Historical Note

The probabilistic formulation of learning draws from classical statistics, decision theory, and probability. The idea of evaluating predictors by expected loss is central to statistical decision theory, while concepts such as conditional expectation and Bayes rules are foundational in probability and mathematical statistics. Modern machine learning inherits these ideas and extends them to high-dimensional models, large-scale optimization, and complex predictive tasks. Although the models have changed dramatically, the underlying probabilistic logic remains one of the deepest unifying principles of the field.

Chapter 3

Hypothesis Spaces and Inductive Bias

3.1 Introduction

In the previous two chapters, we developed two foundational perspectives on machine learning. Chapter 1 presented learning as the construction of a mapping from inputs to outputs, while Chapter 2 placed this process in a probabilistic setting by modeling data as samples from an unknown distribution and evaluating predictors through expected loss. These viewpoints clarify what learning is and how prediction error should be measured. But they also expose a deep problem: the data alone do not determine what should be learned.

If we are given only a finite sample, then many different functions may agree perfectly with the observations. Some of these functions may capture meaningful regularities in the underlying phenomenon, while others may behave arbitrarily away from the training points. Thus learning cannot consist merely in fitting the sample. It must also involve deciding which of the many data-consistent explanations should be preferred. This is the central role of *hypothesis spaces* and *inductive bias*.

A hypothesis space is the set of candidate functions the learner is allowed to consider. Inductive bias is the collection of assumptions, preferences, structural constraints, or algorithmic tendencies that guide the learner toward some hypotheses rather than others. These ideas are not secondary implementation details. They are what make learning possible in the first place. Without restricting the search space or preferring some solutions over others, generalization from finite data would be hopeless.

This chapter develops these ideas in detail. We begin by explaining why unrestricted learning is impossible in any meaningful sense. We then introduce hypothesis classes formally and discuss how they define the search space of learning. From there, we distinguish between explicit and implicit forms of inductive bias, and study common structural assumptions such as sparsity,

smoothness, locality, and compositionality. We then examine how bias appears in both classical models and modern deep architectures. Throughout, we emphasize that model design is not simply about expressive power; it is about choosing a structured class of functions whose biases align with the problem at hand.

Conceptually, this chapter is one of the anchors of the book. It explains why architecture matters, why optimization affects generalization, why representation choices are consequential, and why the same dataset can yield very different models depending on the assumptions imposed. Later chapters on regularization, kernels, convolutional networks, transformers, and generalization theory all build on the ideas introduced here.

3.2 Why Unrestricted Learning Is Impossible

A natural first thought is that a sufficiently powerful learner should simply search over all possible functions and choose the one that best matches the data. This idea is appealing because it seems to avoid restrictive modeling assumptions. But in fact it fails for a fundamental reason: with finite data, unrestricted function learning is underdetermined.

3.2.1 Finite Data Impose Finite Constraints

Suppose we observe a supervised dataset

$$(x_1, y_1), \ldots, (x_n, y_n),$$

where the inputs x_i are distinct. If the space of candidate functions is very large, then there will generally be many functions f such that

$$f(x_i) = y_i \qquad \text{for all } i = 1, \ldots, n.$$

These functions agree on the observed data but may differ dramatically on unseen inputs. Nothing in the sample itself determines which continuation beyond the sample is preferable.

This is not a rare pathology. It is the generic situation whenever the function class has more flexibility than the finite sample can constrain.

[Finite Samples Do Not Determine a Unique Rule] Let $\{(x_i, y_i)\}_{i=1}^n$ be a finite dataset with distinct inputs. If the hypothesis space contains all functions from $\mathcal{X}$ to $\mathcal{Y}$, then in general there exist many distinct functions $f : \mathcal{X} \to \mathcal{Y}$ such that

$$f(x_i) = y_i \qquad \text{for all } i = 1, \ldots, n.$$

Therefore the training data alone do not determine a unique predictor.

Proof sketch. A finite sample constrains the value of a candidate function only on finitely many points. On all other inputs, the function may be specified arbitrarily. Thus one may construct many different functions that agree on the sample and differ elsewhere. □

This proposition seems elementary, but it captures one of the most important facts in machine learning: data fitting alone is insufficient for learning.

3.2.2 Interpolation Is Easy; Generalization Is Hard

If unrestricted memorization were the goal, learning would often be trivial. A sufficiently rich system can simply store the training examples or interpolate them exactly. The real challenge is to produce a predictor that works on new data.

This distinction is essential. A learner is useful not because it reproduces the past, but because it extracts patterns that persist beyond the observed sample. Generalization therefore requires assumptions about the structure of the target relationship. Without such assumptions, extrapolation is not justified.

3.2.3 No-Free-Lunch Intuition

The impossibility of unrestricted learning is closely related to the so-called *no-free-lunch* intuition. Informally, if one averages over all possible target functions with equal weight, then no learner can outperform any other in a universal sense. Success on some problems must come at the price of being better suited to some functions than others.

[No-Free-Lunch Intuition] A learning algorithm can generalize only by favoring some target functions over others. If all possible target functions are treated as equally plausible, then no algorithm has a universal advantage across all problems.

This is not yet a formal theorem in the strongest sense, but it expresses an indispensable principle: every successful learner embodies preferences. These preferences are exactly what we call inductive bias.

3.2.4 Why This Is Good News Rather Than Bad News

At first, the impossibility of unbiased unrestricted learning may seem discouraging. In fact, it is the reason model design matters. It means that the choice of hypothesis class, architecture, prior, regularizer, and training procedure is not arbitrary. These choices encode beliefs about what kinds of functions are likely to arise in the world. When those beliefs align with the task, learning becomes possible.

3.3 Function Classes and Search Spaces

Since unrestricted learning is impossible, we must restrict the set of candidate predictors. This leads to the notion of a hypothesis class.

[Hypothesis Class] A hypothesis class is a set

$$\mathcal{H} \subseteq \{f : \mathcal{X} \to \mathcal{Y}\}$$

of candidate functions from the input space $\mathcal{X}$ to the output space $\mathcal{Y}$.

The learner chooses a predictor from $\mathcal{H}$ rather than from the space of all conceivable functions.

3.3.1 Hypothesis Classes as Search Spaces

One can think of the hypothesis class as the *search space* of learning. It determines what kinds of solutions are even expressible. If a function is not in $\mathcal{H}$, then no amount of data or optimization can recover it exactly. Conversely, if $\mathcal{H}$ is extremely large, then the learning problem may become statistically or computationally difficult.

Thus the choice of $\mathcal{H}$ is a balance:

- too small, and the class may miss important structure;
- too large, and the class may fit noise or require too much data to identify a good solution.

This is one of the most fundamental modeling tradeoffs in all of machine learning.

3.3.2 Examples of Hypothesis Classes

Different models correspond to different hypothesis classes.

Linear models. A linear hypothesis class on $\mathbb{R}^d$ is

$$\mathcal{H}_{\text{lin}} = \{x \mapsto w^\top x + b : w \in \mathbb{R}^d,\ b \in \mathbb{R}\}.$$

This class assumes that prediction is a linear function of the chosen features.

Polynomial models. A polynomial hypothesis class of degree at most m consists of functions with bounded polynomial complexity. This class is more expressive than linear functions but still highly structured.

Kernel methods. Kernel methods define hypothesis spaces of functions that are linear in a possibly high-dimensional feature space. Although these spaces may be infinite-dimensional, they are structured by the chosen kernel.

Neural networks. A neural network architecture specifies a family of functions formed by compositions of affine maps and nonlinearities. The architecture determines what kinds of interactions and representations are easy or efficient to express.

3.3.3 Expressivity Is Not the Whole Story

It is tempting to compare models only by expressive power. Certainly, a richer class can represent more complicated functions. But expressivity alone does not determine learning quality. Two hypothesis classes may both contain an excellent predictor, yet one may be easier to learn from finite data because its internal structure better matches the problem.

This means that the question

"Can the class represent the target?"

is important, but incomplete. We must also ask:

"Does the class encourage the right kinds of functions?"

3.3.4 Approximation and Search

The hypothesis class also interacts with optimization. A model family may in principle contain excellent solutions, but the parameterization or training procedure may make them difficult to find. Thus the search space has both:

- a *representational* aspect, which determines what functions exist in the class;
- an *algorithmic* aspect, which determines what functions are accessible under training.

This distinction leads naturally to the notion of implicit bias, which we will study shortly.

3.4 Inductive Bias: What It Is and Why It Is Necessary

If learning from finite data requires preferences among hypotheses, then those preferences must be encoded somewhere. This is the role of inductive bias.

[Inductive Bias] An inductive bias is any assumption, preference, restriction, prior, or algorithmic tendency that causes a learning system to favor some hypotheses over others beyond what is logically implied by the training data alone.

Inductive bias tells the learner how to extend observed regularities to unseen cases.

3.4.1 Bias Is Not a Defect

In everyday language, the word "bias" often has a negative connotation. In machine learning, inductive bias is not inherently bad. It is necessary. Without it, a learner could not generalize. The important question is not whether a model has bias, but whether its bias is appropriate for the task.

For example:

- a linear model assumes approximate linearity in the features;
- a smoothness prior assumes nearby inputs should have similar outputs;
- a CNN assumes local patterns and translation structure matter;
- a transformer assumes that token-token interactions can be mediated by attention.

These are all forms of bias. They can help enormously when aligned with the problem and hinder badly when misaligned.

3.4.2 Inductive Bias Statement

A useful abstract statement is the following.

[Inductive Bias Principle] Generalization from finite data is possible only if the learner prefers some hypotheses over others in ways not fully determined by the sample.

This principle is not merely philosophical. It explains why all successful machine learning systems involve constraints, regularities, architectures, or priors.

3.4.3 Bias and Generalization

Suppose two functions fit the training data equally well. Which one should the learner choose? The answer depends on bias. A learner may prefer the smoother function, the lower-norm function, the simpler function, the more compressible function, or the one compatible with architectural symmetries. In each case, the learning system is selecting according to some notion of plausibility beyond the sample.

Later chapters will formalize some of these ideas through regularization, norm control, margins, and complexity measures. At this stage, the key message is conceptual: inductive bias is the bridge from finite evidence to future prediction.

3.5 Explicit and Implicit Inductive Bias

Inductive bias can arise in more than one way. A useful distinction is between *explicit* and *implicit* bias.

3.5.1 Explicit Bias

Explicit inductive bias is built directly into the formulation of the model or objective. It is visible and intentional.

Examples include:

- restricting to a linear hypothesis class,
- adding an ℓ_2 penalty to prefer small weights,

- imposing sparsity through an ℓ_1 penalty,
- choosing a convolutional architecture to encode locality and translation structure,
- choosing a kernel that encodes smoothness or similarity.

These biases are easy to identify because they appear directly in the model class or training objective.

3.5.2 Implicit Bias

Implicit inductive bias arises from the training algorithm, parameterization, initialization, data ordering, optimization dynamics, or numerical properties of the learning system, even when the objective does not explicitly state a preference.

Examples include:

- gradient descent tending toward certain low-complexity solutions,
- optimization favoring large-margin separators in separable settings,
- initialization affecting which minima are found,
- parameterization changing which representations are easy to reach.

Implicit bias is especially important in deep learning. Neural networks are often highly overparameterized, so many solutions fit the data well. Yet the training algorithm does not select arbitrarily among them. It often prefers structured solutions, and these preferences can strongly affect generalization.

3.5.3 Why the Distinction Matters

Two models with identical representational power may behave differently because their optimization procedures impose different implicit biases. Conversely, two models with different explicit structure may end up behaving similarly if trained in a way that induces comparable preferences.

Thus one cannot understand generalization by examining the hypothesis class alone. One must also ask how the training dynamics navigate that class.

3.6 Structural Assumptions in Learning

Many inductive biases can be understood as assumptions about structure in the target function or the data. Some of the most important are sparsity, smoothness, locality, and compositionality.

3.6.1 Sparsity

A sparse model assumes that only a small number of features, parameters, or interactions are truly important. Sparsity is attractive when the data are high-dimensional but the underlying mechanism is simple.

Examples:

- linear regression with only a few relevant variables,
- compressed sensing,
- feature selection through ℓ_1 regularization.

A sparse inductive bias prefers explanations that use few active components. This can improve interpretability and statistical efficiency when the assumption is reasonable.

3.6.2 Smoothness

A smoothness bias assumes that small changes in the input should produce small changes in the output. This is common in regression, physical modeling, and many kernel methods.

Smoothness can be encoded through:

- regularization on derivatives,
- kernels favoring smooth functions,
- neighborhood-based assumptions,
- architectural continuity.

Smoothness is often one of the most natural assumptions for interpolation between nearby observations.

3.6.3 Locality

A locality bias assumes that prediction depends primarily on nearby or local patterns rather than arbitrary global interactions. This is particularly important in images, audio, and spatial data.

Convolutional neural networks encode locality by restricting filters to small receptive fields and reusing them across positions. This bias dramatically reduces the number of parameters and emphasizes local pattern detection.

3.6.4 Compositionality

A compositionality bias assumes that complex patterns can be built from simpler reusable parts. This is one of the guiding ideas behind deep learning. Rather

than representing a task as a monolithic flat computation, deep networks construct layered features in which higher-level representations are composed from lower-level ones.

Examples include:

- edges $\to$ textures $\to$ objects in vision,
- phonemes $\to$ words $\to$ phrases in language,
- subroutines composed into larger algorithms.

Compositional structure is one of the main reasons depth can be so powerful.

3.6.5 Invariance and Equivariance

Another important family of structural assumptions concerns symmetries.

[Invariance] A predictor f is invariant to a transformation T if

$$f(Tx) = f(x)$$

for all relevant inputs x.

[Equivariance] A representation map ϕ is equivariant to a transformation T if there exists a corresponding transformation S such that

$$\phi(Tx) = S\phi(x).$$

Invariance says that a transformation should not change the prediction. Equivariance says that transformations of the input should induce structured transformations in the representation.

Examples:

- image classification often seeks approximate translation invariance,
- convolution layers are translation equivariant,
- permutation-invariant models are useful for sets,
- graph neural networks encode symmetries of graph structure.

These symmetry assumptions can dramatically reduce sample complexity by building known structure directly into the model.

3.7 Simplicity Assumptions and Occam-Type Preferences

A pervasive idea in learning is that among hypotheses that explain the data equally well, simpler ones should often be preferred. This is related to Occam's razor: do not multiply complexity beyond necessity.

3.7.1 What Simplicity Can Mean

In machine learning, simplicity may refer to many different things:

- fewer parameters,
- smaller norm,
- smoother behavior,
- sparser representation,
- shallower or more regular structure,
- shorter description length,
- lower effective complexity.

There is no single universal definition of simplicity, and different mathematical frameworks formalize it differently. But the underlying intuition is that regular explanations are more plausible than arbitrary ones.

3.7.2 Why Simplicity Helps

A function that fits the training data in an extremely irregular way may be exploiting accidental features of the sample. A simpler function is often more likely to reflect a genuine pattern that extends beyond the observations. This is why regularization methods, norm penalties, smooth kernels, and architectural constraints often improve generalization.

However, simplicity must be understood relative to the problem. A linear rule may be simple in one representation and highly complicated in another. Thus simplicity is not absolute; it is tied to the language in which the model is expressed.

3.7.3 Simplicity and Representation

This point is worth emphasizing. The same target relation may be simple under one representation and complex under another. For instance, a problem that is not linearly separable in raw input coordinates may become linearly separable after a suitable feature transformation. In this sense, representation learning may be viewed as the search for coordinates in which the task becomes simple.

3.8 Bias in Classical Models

Inductive bias is not unique to deep learning. Classical machine learning models are full of strong structural assumptions.

3.8.1 Linear Models

Linear regression and logistic regression assume that the output depends linearly on the chosen features. This is a major bias. It is powerful when the representation is appropriate and limiting when important nonlinear interactions are present.

The linear bias has several appealing consequences:

- interpretability,
- computational tractability,
- relatively low variance,
- clear geometric structure.

But it also means that success depends heavily on feature design.

3.8.2 Kernel Methods

Kernel methods encode bias through a notion of similarity. A kernel chooses what it means for two inputs to be close or related. For example, the radial basis function kernel favors smooth functions with nearby inputs behaving similarly.

Thus the inductive bias is not only in the predictor but in the geometry imposed on the input space. Kernel methods can be highly expressive, but they remain structured by the chosen similarity measure.

3.8.3 Nearest-Neighbor and Local Methods

Methods based on neighborhoods encode a local smoothness bias: nearby points should have similar outputs. This can be effective in low-dimensional settings or when distance is meaningful, but it may fail in high-dimensional spaces where local neighborhoods become unstable.

3.8.4 Regularized Linear Models

Ridge regression and lasso illustrate how explicit regularization creates explicit bias:

- ridge prefers small coefficients,
- lasso prefers sparse coefficients.

These are not merely numerical stabilizers. They express beliefs about what kinds of predictors are plausible.

3.9 Bias in Deep Models

Deep learning often appears flexible and universal, but it is deeply structured. Each architecture embodies a family of inductive biases.

3.9.1 Multilayer Perceptrons

A multilayer perceptron assumes that the target function can be represented as a composition of affine transformations and nonlinearities. This encodes a compositional bias, but without specialized spatial or sequential structure.

As a result, multilayer perceptrons are general-purpose but may be statistically inefficient when the task has stronger known structure that could be exploited.

3.9.2 Convolutional Neural Networks

Convolutional neural networks embody several powerful biases:

- **locality:** filters operate on local neighborhoods;
- **weight sharing:** the same filter is reused across positions;
- **translation equivariance:** shifting the input shifts intermediate feature maps in a structured way;
- **hierarchical composition:** local features combine into larger patterns.

These biases make CNNs highly effective for images and other spatially structured data. They dramatically reduce the number of parameters compared with fully connected models and align well with the structure of visual signals.

[CNN Bias] In image classification, an edge detector that is useful in the top-left corner is usually also useful in the bottom-right corner. Convolution exploits this by reusing the same filter everywhere. This is an explicit translation-related inductive bias.

3.9.3 Transformers

Transformers encode a different family of assumptions:

- inputs are sequences or sets of tokens,
- relationships among positions are important,
- relevant dependencies may be long-range,
- attention can adaptively select which tokens matter.

Unlike CNNs, transformers do not impose strict locality by default. Instead, they assume that flexible pairwise interactions mediated by attention are useful. Positional encodings add information about order, reflecting the belief that sequence position matters even though attention itself is permutation-equivariant before positional information is added.

[Transformer Bias] In language modeling, the next token may depend strongly on a word that appeared many positions earlier. A transformer is biased toward capturing such long-range contextual interactions through attention.

3.9.4 Recurrent Models

Recurrent neural networks encode a sequential-state bias: information is processed progressively through a hidden state. This bias is natural for time series and streaming problems, though it can struggle with long-range dependencies compared with attention-based models.

3.9.5 Architecture as Inductive Bias

These examples illustrate a central thesis:

[Architecture as Inductive Bias] The architecture of a model determines not only what functions can be represented, but also what regularities are easy to express, parameter-efficient to learn, and statistically favored.

Architecture is therefore not just an implementation choice. It is a mathematical and statistical assumption about structure in the task.

3.10 Priors, Regularization, and Preferences Over Functions

Inductive bias can also be expressed through priors and regularization.

3.10.1 Priors

In Bayesian language, a prior distribution over parameters or functions expresses beliefs about plausible solutions before seeing the data. A prior may prefer:

- small weights,
- smooth functions,
- sparse representations,
- hierarchical structure.

From this viewpoint, learning updates prior beliefs using the observed data.

3.10.2 Regularization as Encoded Preference

In optimization-based learning, regularization terms play a similar role. A training objective of the form

$$\widehat{R}_n(f) + \lambda\,\Omega(f)$$

combines data fit with a penalty that discourages certain forms of complexity. The term $\Omega(f)$ encodes preference. For example:

- $\|w\|_2^2$ prefers smaller weights,
- $\|w\|_1$ prefers sparsity,

- derivative penalties prefer smoothness.

Although Bayesian priors and regularization are not identical in every setting, they are closely related conceptually: both express which hypotheses are regarded as more plausible.

3.10.3 Bias Through the Learning Pipeline

More broadly, inductive bias enters through many parts of the pipeline:

- the representation of inputs,
- the hypothesis class,
- the architecture,
- the prior or regularizer,
- the optimization algorithm,
- the augmentation strategy,
- the evaluation metric.

Thus bias is not located in just one component. It is distributed throughout the learning system.

3.11 Comparing Hypothesis Classes for a Task

A natural question is how to compare two hypothesis classes for a given task. There is no single universal criterion, but several considerations are important.

3.11.1 Alignment with Problem Structure

The first question is whether the class reflects known regularities of the task. For image data, convolution may be more appropriate than a generic fully connected network because the task has spatial structure. For sequence data with long-range dependencies, attention-based models may be better aligned than purely local models.

3.11.2 Expressivity vs Efficiency

A more expressive class can represent more functions, but that does not mean it is always preferable. Sometimes a less expressive but better structured class learns faster, generalizes better, and requires fewer examples.

3.11.3 Statistical and Computational Tradeoffs

A model class should be judged not only by what it can represent but also by:

- how much data it needs,
- how stable it is,
- how hard it is to optimize,
- whether its inductive biases match the domain.

The "best" hypothesis class is therefore task-dependent.

[Two Classes for Image Classification] Consider the task of image classification. Compare:

1. a fully connected multilayer perceptron on raw pixels;
2. a convolutional neural network.

Both may be expressive enough in principle, but the CNN builds in locality and translation structure. This typically makes it more parameter-efficient and better aligned with the geometry of images.

[Two Classes for Tabular Regression] For structured tabular data with a moderate number of interpretable features, compare:

1. linear regression,
2. a deep multilayer perceptron.

If the relationship is approximately linear and interpretability matters, the linear class may be preferable. If there are strong nonlinear interactions and enough data, the neural network may be better.

3.12 A Unifying Perspective

This chapter developed one of the deepest ideas in machine learning: learning is possible only because the learner is not neutral. It searches within a restricted space of hypotheses and brings preferences about which solutions are more plausible. These preferences may appear as model class restrictions, priors, regularizers, optimization dynamics, symmetries, architectural patterns, or representation choices.

A useful way to summarize the situation is:

$$\text{generalization} \quad \text{requires} \quad \text{inductive bias.}$$

Without bias, finite data do not determine how to behave on unseen inputs. With well-chosen bias, a learner can extrapolate in meaningful ways.

The central conceptual shift is this: machine learning is not merely about fitting functions. It is about choosing structured families of functions whose

biases align with the structure of the world. Linear models, kernels, CNNs, recurrent networks, transformers, and regularized estimators all embody this principle in different forms.

Thus the question is never simply

"How powerful is the model?"

but rather

"What kinds of functions does the model prefer, and are those preferences appropriate for the task?"

3.13 Summary

In this chapter we introduced hypothesis spaces as the search spaces of learning and inductive bias as the assumptions or preferences that guide prediction beyond the training data. We explained why unrestricted learning is impossible from finite data and developed the no-free-lunch intuition that successful learning requires favoring some hypotheses over others.

We distinguished between explicit and implicit inductive bias, and studied common structural assumptions such as sparsity, smoothness, locality, compositionality, invariance, and equivariance. We then examined how these ideas appear in both classical and deep models. Linear models favor linear dependence in features; kernels encode similarity and smoothness; CNNs encode locality and translation structure; transformers encode flexible sequence interactions and context dependence.

The main lesson is that architecture, regularization, prior assumptions, and optimization are all forms of inductive bias. They are not incidental details. They are the mechanisms by which learning becomes possible.

The next chapter will build on this discussion by examining learning as optimization, where the hypothesis class must be searched by concrete computational procedures and where the objective function formalizes the tradeoff between data fit and structure.

3.14 Exercises

1. Give the definition of a hypothesis class. Why is a hypothesis class necessary in machine learning?

2. Explain in your own words why unrestricted learning from finite data is impossible.

3. State the no-free-lunch intuition informally. What does it say about generalization?

4. Consider the class of all functions from a finite set $\mathcal{X}$ to $\mathcal{Y} = \{0, 1\}$. Suppose you observe labels on only part of $\mathcal{X}$. Why are there many hypotheses consistent with the observed data?

5. Compare the following two hypothesis classes for regression on $\mathbb{R}^d$:
   (a) linear functions,
   (b) degree-m polynomial functions.

   Which is more expressive? Which may generalize better with little data? Explain.
6. What is inductive bias? Why is it not a flaw but a necessity?
7. Give one example of explicit inductive bias and one example of implicit inductive bias.
8. Explain the difference between explicit regularization and implicit bias from optimization.
9. A model fits the training data perfectly. Why might it still generalize poorly?
10. What is meant by a sparsity assumption? Give a learning problem where sparsity is a reasonable bias.
11. What is meant by smoothness? Why can smoothness be useful for interpolation?
12. Define invariance and equivariance. Give one example of each from machine learning.
13. Why is convolutional structure a sensible inductive bias for images?
14. Why can a transformer be better suited than a purely local model for language modeling?
15. In what sense is architecture a form of inductive bias?
16. Compare the inductive biases of:
    (a) linear regression,
    (b) kernel regression with an RBF kernel,
    (c) a convolutional neural network,
    (d) a transformer.
17. Suppose you are designing a model for the following task: predicting sentiment from short text reviews. Compare two possible hypothesis classes and argue which inductive biases may help.
18. Suppose a task has strong translation symmetry. Why might a fully connected network be less sample-efficient than a convolutional model?
19. Explain how priors and regularizers both encode preferences over functions.

20. Give an example of a task where a linear model is a better choice than a deep neural network, and explain why.

21. Give an example of a task where compositionality is an important structural assumption.

22. Why is simplicity not an absolute notion but representation-dependent?

23. Consider two models that have similar expressive power but are trained with different optimization algorithms. Why might they generalize differently?

24. For each of the following tasks, identify a plausible useful inductive bias:

    (a) object recognition in photographs,

    (b) predicting housing prices from tabular features,

    (c) forecasting a time series,

    (d) language translation,

    (e) classifying a set of unordered points.

25. Write a short paragraph explaining the slogan:

    generalization requires inductive bias.

Historical Note

The modern language of hypothesis spaces and inductive bias emerged from several traditions: logic and philosophy of induction, statistical learning theory, approximation theory, and pattern recognition. Classical statistics long relied on structural modeling assumptions such as linearity, Gaussian noise, and smoothness. Statistical learning theory clarified that generalization depends on restricting complexity or favoring certain hypotheses. In modern deep learning, these ideas reappear in new forms through architecture, optimization, implicit bias, and large-scale representation learning. Although the models have changed, the central question remains the same: what assumptions allow finite data to support reliable prediction?

Chapter 4

Learning as Optimization

4.1 Introduction

Learning is often described as finding a good predictor from data. In most modern formulations, this search is expressed as an optimization problem: choose parameters or functions that minimize a loss on the observed sample, possibly together with a regularization term. This viewpoint is central because it connects statistical modeling with algorithms. A model is not only a family of functions; it is also an objective to be minimized and a landscape to be navigated.

This chapter introduces the optimization view of learning in a mathematically grounded but compact form. We define empirical risk minimization, derive gradient-based updates, discuss convex and nonconvex objectives, and explain the relation between constrained and penalized formulations. We also distinguish optimization error from statistical error, since a model may fail either because the objective was not minimized well or because the data do not support accurate generalization.

4.2 From Learning Objectives to Optimization Problems

Let $\mathcal{H}$ be a hypothesis class and let

$$\ell : \mathcal{A} \times \mathcal{Y} \to [0, \infty)$$

be a loss function. Given data

$$(x_1, y_1), \ldots, (x_n, y_n),$$

the empirical risk of a predictor f is

$$\widehat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

[Empirical Risk Minimization] Empirical risk minimization (ERM) chooses a predictor by solving

$$\min_{f \in \mathcal{H}} \widehat{R}_n(f).$$

In practice, one often parameterizes the hypothesis class by a parameter vector $\theta \in \mathbb{R}^p$, writing f_θ. Then the learning problem becomes

$$\min_{\theta \in \mathbb{R}^p} J(\theta), \qquad J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

Regularization is commonly added:

$$\min_{\theta} \; J(\theta) + \lambda \Omega(\theta),$$

where $\Omega(\theta)$ penalizes complexity, such as large norms or lack of sparsity.

Thus learning typically becomes the minimization of an objective function

$$F(\theta) = \text{data fit} + \text{regularization}.$$

4.3 Gradient Descent as Local Linearization

When the objective is differentiable, the gradient indicates the local direction of steepest increase. To decrease the objective, one moves in the opposite direction.

[Gradient Descent Update] Given a differentiable objective $F : \mathbb{R}^p \to \mathbb{R}$, gradient descent updates parameters by

$$\theta_{t+1} = \theta_t - \eta_t \nabla F(\theta_t),$$

where $\eta_t > 0$ is the step size or learning rate.

Why does this make sense? By first-order Taylor expansion,

$$F(\theta + \Delta) \approx F(\theta) + \nabla F(\theta)^\top \Delta.$$

If we choose $\Delta = -\eta \nabla F(\theta)$, then

$$F(\theta + \Delta) \approx F(\theta) - \eta \|\nabla F(\theta)\|^2,$$

so for small enough η, the objective decreases locally.

This gives a geometric interpretation: gradient descent repeatedly linearizes the objective and moves in the direction that most rapidly decreases this local approximation.

4.4 Worked Examples

4.4.1 Least Squares

For linear regression with prediction

$$f_\theta(x) = x^\top \theta,$$

the least-squares objective is

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(x_i^\top \theta - y_i)^2.$$

If $X \in \mathbb{R}^{n\times p}$ is the design matrix and $y \in \mathbb{R}^n$ the target vector, then

$$J(\theta) = \frac{1}{n}\|X\theta - y\|^2.$$

Its gradient is

$$\nabla J(\theta) = \frac{2}{n}X^\top(X\theta - y).$$

Hence gradient descent takes the form

$$\theta_{t+1} = \theta_t - \eta\frac{2}{n}X^\top(X\theta_t - y).$$

4.4.2 Logistic Loss

For binary classification with labels $y_i \in \{0, 1\}$ and score $z_i = x_i^\top \theta$, the logistic model predicts

$$\sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

The average logistic loss is

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\Big(y_i \log \sigma(z_i) + (1 - y_i)\log(1 - \sigma(z_i))\Big).$$

Its gradient is

$$\nabla J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(\sigma(x_i^\top \theta) - y_i)x_i.$$

This has a natural interpretation: prediction minus target, weighted by the input.

4.4.3 Ridge Regression

Ridge regression adds an ℓ_2 penalty:

$$J_\lambda(\theta) = \frac{1}{n}\|X\theta - y\|^2 + \lambda\|\theta\|^2.$$

Its gradient is

$$\nabla J_\lambda(\theta) = \frac{2}{n}X^\top(X\theta - y) + 2\lambda\theta.$$

Thus the penalty pulls the solution toward smaller parameter norms.

4.5 Convex and Nonconvex Landscapes

Optimization behavior depends strongly on the geometry of the objective.

[Convex Function] A function $F : \mathbb{R}^p \to \mathbb{R}$ is convex if for all $\theta, \phi \in \mathbb{R}^p$ and all $\lambda \in [0, 1]$,

$$F(\lambda\theta + (1 - \lambda)\phi) \leq \lambda F(\theta) + (1 - \lambda)F(\phi).$$

Geometrically, the graph of a convex function lies below the line segment joining any two points on the graph.

[Strongly Convex Function] A differentiable function F is μ-strongly convex if there exists $\mu > 0$ such that for all θ, ϕ,

$$F(\phi) \geq F(\theta) + \nabla F(\theta)^\top(\phi - \theta) + \frac{\mu}{2}\|\phi - \theta\|^2.$$

Strong convexity implies a unique minimizer and gives stronger convergence guarantees for gradient methods.

4.5.1 Why Convexity Matters

For convex objectives:

- every local minimum is a global minimum,
- the landscape is more predictable,
- convergence theory is cleaner.

Least squares and logistic regression are convex in their parameters. Ridge regression is strongly convex when the penalty is present.

4.5.2 Nonconvex Objectives

Deep neural networks typically produce nonconvex objectives. In that case, the landscape may contain:

- multiple local minima,
- saddle points,
- flat and sharp regions,
- symmetries due to parameterization.

Despite this, gradient-based methods often work well in practice. Understanding why is one of the important themes of modern deep learning theory.

4.6 Constraints, Penalties, and Lagrangians

Learning problems can often be written either as constrained optimization problems or as unconstrained penalized problems.

A constrained form is

$$\min_{\theta} J(\theta) \quad \text{subject to} \quad \Omega(\theta) \leq c.$$

A penalized form is

$$\min_{\theta} J(\theta) + \lambda\Omega(\theta).$$

The first says: fit the data as well as possible while staying within a complexity budget. The second says: fit the data, but pay a cost for complexity.

These formulations are closely related. The bridge is given by the Lagrangian

$$\mathcal{L}(\theta, \lambda) = J(\theta) + \lambda(\Omega(\theta) - c), \qquad \lambda \geq 0.$$

Under suitable conditions, solving the constrained problem corresponds to solving a penalized problem for an appropriate value of λ.

4.6.1 Interpretation

This equivalence is conceptually important:

- constraints encode hard limits,
- penalties encode soft preferences,
- both express inductive bias.

Ridge regression is the penalized counterpart of constraining the parameter norm.

4.7 Optimization Error and Statistical Error

When a trained model performs poorly, the reason may not be purely statistical. It may simply be that the optimizer failed to find a good minimizer.

It is useful to separate:

- **optimization error:** the gap between the achieved objective value and the best value within the hypothesis class;
- **estimation/statistical error:** the gap between the best empirical or in-class solution and the true population optimum.

Very roughly,

$$\text{total error} \approx \text{approximation error} + \text{statistical error} + \text{optimization error}.$$

This distinction matters because different remedies address different failures:

- large optimization error suggests better algorithms, initialization, or tuning;
- large statistical error suggests more data, better regularization, or a better-matched hypothesis class.

4.8 A Unifying Perspective

The optimization view of learning turns model fitting into a precise mathematical problem. A hypothesis class gives the search space, a loss function gives the data-fit criterion, and regularization or constraints add structural preference. The resulting objective defines a landscape, and learning algorithms move through that landscape in search of low-risk solutions.

This viewpoint also clarifies an important fact: good learning requires both a sensible statistical formulation and a workable optimization procedure. A model may be theoretically well motivated but computationally hard to train, or easy to optimize but statistically too rigid.

4.9 Summary

In this chapter we formulated learning as optimization. We defined empirical risk minimization, introduced gradient descent as local linearization of the objective, and studied three basic examples: least squares, logistic loss, and ridge regression. We also defined convexity and strong convexity, explained why convex objectives are easier to analyze, and contrasted them with the nonconvex landscapes common in deep learning.

Finally, we compared constrained and penalized formulations through the Lagrangian viewpoint and distinguished optimization error from statistical error. These ideas will recur throughout the book, especially when we study large-scale training methods and deep neural networks.

4.10 Exercises

1. Let
$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(x_i^\top\theta - y_i)^2.$$
Compute $\nabla J(\theta)$.

2. Derive the gradient descent update for least squares.

3. For logistic regression with
$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\Big(y_i \log\sigma(x_i^\top\theta) + (1-y_i)\log(1-\sigma(x_i^\top\theta))\Big),$$

show that

$$\nabla J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(\sigma(x_i^\top \theta) - y_i)x_i.$$

4. Write the gradient descent update for ridge regression.

5. Give the definition of a convex function. Why does convexity simplify optimization?

6. Give the definition of strong convexity. What additional consequence does it have compared with ordinary convexity?

7. Compare the constrained problem

$$\min_\theta J(\theta) \quad \text{subject to } \|\theta\|^2 \le c$$

with the penalized problem

$$\min_\theta J(\theta) + \lambda\|\theta\|^2.$$

Explain the difference in interpretation.

8. What is optimization error? How is it different from statistical error?

9. Explain why gradient descent can be interpreted as minimizing a local linear approximation.

10. Give one example of a convex learning objective and one example of a nonconvex learning objective.

Chapter 5

Generalization and Overfitting

5.1 Introduction

A central goal of machine learning is not merely to fit observed data, but to perform well on new data drawn from the same environment. This is the problem of *generalization*. A model may achieve very small training error yet fail badly on unseen examples. Conversely, a model that does not fit the training data well may be too simple to capture the underlying pattern. These two failures are commonly called *overfitting* and *underfitting*.

This chapter gives a compact but more formal treatment of generalization. We distinguish training, validation, and test performance; define the generalization gap; discuss model complexity and effective capacity; and introduce the basic concentration intuition behind why empirical performance can sometimes reflect population performance. We also examine the train/validation/test protocol and conclude with a brief remark on modern phenomena such as double descent.

5.2 What Generalization Means

Let $(X, Y) \sim P$ be a random input–output pair, and let

$$(X_1, Y_1), \ldots, (X_n, Y_n)$$

be a training sample drawn i.i.d. from P. For a predictor f, recall the population risk

$$R(f) = \mathbb{E}[\ell(f(X), Y)]$$

and the empirical risk

$$\widehat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(X_i), Y_i).$$

[Generalization Gap] The generalization gap of a predictor f is

$$R(f) - \widehat{R}_n(f).$$

A model generalizes well when its empirical performance is close to its population performance, that is, when the generalization gap is small and the population risk itself is low.

The key distinction is:

- **training error** measures fit to observed data;
- **test error** estimates performance on unseen data;
- **generalization** concerns how well training success transfers to future data.

5.3 Overfitting and Underfitting

A model *underfits* when it is too simple to capture the relevant structure in the data. Then both training and test errors are high.

A model *overfits* when it adapts too closely to accidental features of the training sample. Then training error is low but test error is substantially higher.

5.3.1 A Basic Pattern

As model complexity increases, one often sees:

- training error decreases;
- test error first decreases, then may increase if overfitting occurs.

This leads to the classical U-shaped test-error curve as a function of complexity.

5.3.2 Worked Example: Polynomial Regression

Suppose we fit data with polynomials of degree m:

$$f_m(x) = a_0 + a_1 x + \cdots + a_m x^m.$$

If m is too small, the model cannot capture curvature and underfits. If m is too large relative to sample size, it may interpolate noise and overfit. Thus low training error alone is not evidence of good prediction.

5.3.3 Worked Example: Noisy Classification

In classification with label noise, a very flexible classifier may memorize mislabeled points. This can drive training error toward zero while harming test performance. A less flexible classifier may have slightly larger training error but smaller population risk.

5.4 Sources of Generalization Error

Why can a model fail to generalize? Several sources matter.

5.4.1 Approximation Error

If the hypothesis class is too restrictive, even the best function in the class may be far from optimal. This is approximation error.

5.4.2 Estimation Error

Even if the class contains good predictors, finite data may not be enough to identify them reliably. This is estimation error.

5.4.3 Optimization Error

The training algorithm may fail to find a sufficiently good minimizer of the objective. This is optimization error.

Very roughly,

$$\text{prediction error} \approx \text{approximation error} + \text{estimation error} + \text{optimization error}.$$

5.4.4 Bias–Variance Intuition

A classical way to organize part of this tradeoff is through bias and variance.

- **Bias** reflects systematic error from using a model class that is too rigid.
- **Variance** reflects sensitivity to fluctuations in the training sample.

Simple models tend to have high bias and low variance. Flexible models tend to have lower bias but higher variance. Overfitting is closely related to excessive variance.

5.5 Capacity, Complexity, and Effective Complexity

Generalization depends not only on the number of parameters, but more broadly on model capacity: the richness of the functions the learner can effectively realize.

5.5.1 Capacity and Complexity

A highly expressive model can fit many patterns, including noise. A more restricted model may generalize better when data are limited. Complexity can be measured in many ways:

- number of parameters,
- norm size,
- smoothness,
- margin,
- function-class complexity measures.

No single notion works universally, but the core idea is that richer classes require stronger control to generalize well.

5.5.2 Uniform Convergence Intuition

A central idea in learning theory is that empirical risk should approximate population risk *uniformly* over a hypothesis class:

$$\sup_{f \in \mathcal{H}} |R(f) - \widehat{R}_n(f)| \quad \text{is small.}$$

If this holds, then any function that looks good on the sample is likely to be genuinely good on the population.

This is the intuition behind uniform convergence: the larger or more complex the class, the harder it is to guarantee that empirical and population risks stay close simultaneously for all hypotheses.

5.5.3 Concentration Intuition

Why should empirical averages resemble expectations at all? Because averages of many independent observations tend to stabilize. This is the role of concentration. At an informal level:

- for a fixed predictor, empirical risk is an average of random quantities;
- such averages often stay close to their expectations when sample size is large;
- controlling this simultaneously over many predictors becomes harder as the class grows.

This is why both sample size and model complexity matter.

5.5.4 Effective Complexity

In modern learning, raw parameter count is often not the full story. Two models with the same size may generalize differently depending on regularization, optimization, architecture, and data. This motivates the idea of *effective complexity*: the complexity actually expressed by the trained solution, not just the nominal size of the hypothesis class.

5.6 The Train/Validation/Test Protocol

To estimate generalization and choose models responsibly, one separates data into different roles.

5.6.1 Training Set

The training set is used to fit parameters by minimizing empirical risk or a related objective.

5.6.2 Validation Set

The validation set is used for model selection:

- choosing hyperparameters,
- selecting architecture,
- deciding stopping time,
- comparing regularization strengths.

5.6.3 Test Set

The test set is used only at the end to estimate final performance. It should not influence model design. Otherwise, test performance becomes optimistic and no longer represents true unseen accuracy.

5.6.4 Why the Distinction Matters

If the same data are repeatedly used both to fit and to evaluate models, then evaluation becomes contaminated. The train/validation/test split protects against this by separating learning, tuning, and final assessment.

5.7 Interpolation vs Extrapolation

It is useful to distinguish two kinds of prediction beyond observed points.

5.7.1 Interpolation

Interpolation refers to prediction within regions well covered by training data. Smoothness or local similarity assumptions often make this easier.

5.7.2 Extrapolation

Extrapolation refers to prediction outside the range or regime represented in the data. This is typically harder and depends much more strongly on inductive bias.

[Interpolation vs Extrapolation] A polynomial fitted on inputs in the interval $[-1, 1]$ may interpolate well within that interval but behave erratically outside it. Good interpolation does not guarantee good extrapolation.

This distinction matters because many models that appear accurate on familiar inputs can fail badly under distribution shift or out-of-range prediction.

5.8 A Brief Remark on Double Descent

The classical picture suggests that test error eventually worsens as model complexity increases. In modern overparameterized models, however, the behavior can be more subtle. In some settings, test error first increases near interpolation and then decreases again as model size grows further. This is known as *double descent*.

For now, the main point is only this: classical bias–variance intuition remains useful, but modern high-dimensional models can display more complicated generalization behavior than the simplest textbook picture suggests.

5.9 A Unifying Perspective

Generalization is the central success criterion of learning. A model should not merely explain the past; it should predict the future under the same underlying data-generating process. Whether this happens depends on a balance among hypothesis-class richness, sample size, noise level, regularization, and optimization.

The central tension is:

$$\text{fit the data well} \qquad \text{without fitting accidental noise.}$$

Underfitting comes from insufficient flexibility; overfitting comes from excessive flexibility relative to the data and the inductive biases in use. Generalization theory studies when empirical performance is a trustworthy guide to population performance, and why complexity control matters.

5.10 Summary

In this chapter we defined generalization through the distinction between empirical and population risk and introduced the generalization gap. We explained overfitting and underfitting, discussed approximation, estimation, and optimization errors, and reviewed the bias–variance intuition. We also introduced uniform convergence and concentration at an intuitive level, emphasizing that sample averages can approximate expectations when model complexity is controlled.

Finally, we described the train/validation/test protocol and distinguished interpolation from extrapolation. These ideas will be used repeatedly throughout the book when we study regularization, model selection, and the generalization behavior of deep networks.

5.11 Exercises

1. Define the population risk, empirical risk, and generalization gap of a predictor.
2. Explain the difference between training error and test error.
3. What does it mean for a model to overfit? What does it mean to underfit?
4. Suppose model A has high training error and high test error. Suppose model B has near-zero training error but much larger test error than training error. Which model is underfitting? Which is overfitting?
5. In polynomial regression, what may happen if the degree is too small? What may happen if the degree is too large relative to the sample size?
6. Give a noisy classification example in which a flexible classifier may memorize training labels.
7. Explain the difference between approximation error and estimation error.
8. What is optimization error? Give one practical reason it may be large.
9. State the bias–variance tradeoff in words.
10. Why is model complexity relevant to generalization?
11. What is the intuition behind uniform convergence?
12. What is the intuition behind concentration of empirical averages around expectations?
13. Why do we use separate validation and test sets?
14. A researcher repeatedly adjusts hyperparameters based on test performance. Why is this problematic?
15. Explain the difference between interpolation and extrapolation.
16. A model performs very well on inputs similar to the training sample but poorly on inputs far outside the training range. Is this primarily a failure of interpolation or extrapolation?
17. How can regularization reduce overfitting?
18. Why might effective complexity differ from raw parameter count?

19. Sketch the classical training-error and test-error curves as model complexity increases, and explain their shape.

20. What is the basic idea of double descent, and why does it complicate the classical picture?

Chapter 6

Regularization and Structure

6.1 Introduction

A learner should fit data well, but not in an arbitrary way. If a model is too flexible relative to the available sample, it may fit accidental variation rather than stable structure. Regularization is the collection of methods that prevent or reduce this failure. It introduces preference for certain solutions over others: smaller parameters, sparser representations, smoother functions, invariance to transformations, or solutions reached by specific optimization dynamics.

Regularization is therefore one of the clearest places where statistical learning and deep learning meet. In classical models, regularization often appears as an explicit penalty or constraint. In deep learning, it also appears through early stopping, data augmentation, architecture, and the implicit bias of optimization. This chapter develops these ideas in a compact mathematical form.

6.2 Why Regularization Works

Suppose we choose a predictor by minimizing empirical risk alone:

$$\min_{f \in \mathcal{H}} \widehat{R}_n(f).$$

If the hypothesis class is rich, there may be many near-minimizers, some of which generalize well and others poorly. Regularization selects among them by preferring structured solutions.

At a high level, regularization works because it reduces effective complexity. It discourages hypotheses that are unnecessarily flexible, unstable, or sensitive to the sample. In this way, it trades a small increase in training error for a potentially large decrease in test error.

A useful schematic is:

$$\text{good generalization} \approx \text{data fit} + \text{controlled complexity}.$$

6.3 Penalized ERM and Constraints

The standard regularized form of empirical risk minimization is

[Penalized ERM] Given a penalty $\Omega(f)$ and parameter $\lambda > 0$, penalized ERM chooses

$$\min_{f \in \mathcal{H}} \widehat{R}_n(f) + \lambda \Omega(f).$$

Here: - $\widehat{R}_n(f)$ measures fit to the data, - $\Omega(f)$ measures complexity, - λ controls the tradeoff.

An alternative is a constrained formulation:

$$\min_{f \in \mathcal{H}} \widehat{R}_n(f) \qquad \text{subject to } \Omega(f) \leq c.$$

These two viewpoints are closely related. One says "minimize error under a complexity budget"; the other says "minimize error while paying for complexity."

6.3.1 Constrained–Penalized Equivalence

Under suitable conditions, constrained and penalized problems are equivalent in the sense that for an appropriate choice of λ, a solution of

$$\min_f \widehat{R}_n(f) + \lambda \Omega(f)$$

also solves

$$\min_f \widehat{R}_n(f) \quad \text{subject to } \Omega(f) \leq c$$

for some corresponding c.

This equivalence is conceptually important: regularization may be interpreted either as a soft preference or as a hard structural restriction.

6.4 Tikhonov and Ridge Regularization

The most classical regularizer is the squared Euclidean norm. For linear regression with parameter vector $\theta \in \mathbb{R}^p$, ridge regression solves

$$\min_{\theta \in \mathbb{R}^p} \frac{1}{n} \|X\theta - y\|^2 + \lambda \|\theta\|^2.$$

This is also called *Tikhonov regularization.*

6.4.1 Closed-Form Solution

Differentiate the objective:

$$\nabla_\theta \left(\frac{1}{n} \|X\theta - y\|^2 + \lambda \|\theta\|^2 \right) = \frac{2}{n} X^\top (X\theta - y) + 2\lambda\theta.$$

Setting the gradient to zero gives

$$\frac{1}{n} X^\top X\theta - \frac{1}{n} X^\top y + \lambda\theta = 0,$$

so

$$\left(X^\top X + n\lambda I \right) \theta = X^\top y.$$

Hence

$$\hat{\theta}_{\text{ridge}} = \left(X^\top X + n\lambda I \right)^{-1} X^\top y.$$

6.4.2 Shrinkage Interpretation

Ridge regularization shrinks coefficients toward zero. It does not usually make them exactly zero, but it reduces their magnitude. This shrinkage stabilizes estimation, especially when features are correlated or the design matrix is ill-conditioned.

Geometrically, ridge regression balances least-squares error with preference for small Euclidean norm. Statistically, it reduces variance at the cost of some bias.

6.5 Lasso and Sparsity

If we want not just small coefficients but sparse ones, a natural penalty is the ℓ_1 norm:

$$\min_{\theta \in \mathbb{R}^p} \frac{1}{n} \|X\theta - y\|^2 + \lambda \|\theta\|_1.$$

This is the *lasso*.

The key difference from ridge is qualitative: - ridge prefers small coefficients, - lasso can drive some coefficients exactly to zero.

This makes lasso attractive when one believes only a small number of features are truly relevant.

6.5.1 Geometric Picture of L_1L1 vs L_2L2

The difference is often explained geometrically. In two dimensions: - an ℓ_2 constraint $\|\theta\|_2 \leq c$ is a disk, - an ℓ_1 constraint $\|\theta\|_1 \leq c$ is a diamond.

The least-squares objective has elliptical level sets. The first point where such a level set touches the feasible set is often: - smooth on the ridge disk, leading to shrinkage, - at a corner of the lasso diamond, leading to zeros in coordinates.

So the geometry of the constraint explains sparsity.

6.6 Structure-Aware Regularization

Not all regularization is simple norm control. Many forms of regularization encode task structure more directly.

6.6.1 Smoothness and Stability

A regularizer may prefer functions that change slowly with the input, discouraging unstable fits. This is common in kernel methods and inverse problems.

6.6.2 Group and Structured Sparsity

Sometimes features come in groups or known structures. Then one may regularize entire blocks together rather than individual coordinates. This encourages sparsity at the level of groups, channels, or modules.

6.6.3 Architecture as Regularization

A convolutional network is regularized not only by penalties but by its structure: - local connectivity, - weight sharing, - translation-related bias.

Likewise, transformer architectures regularize the space of functions by making token interaction occur through attention and layered composition. In this broader sense, architecture is a structural regularizer.

6.7 Data Augmentation and Invariance as Regularization

Regularization can also come from modifying the data rather than the objective. If a task should be invariant to certain transformations, then training on transformed examples encourages the model to learn this invariance.

Examples include: - image flips and crops, - noise injection, - token masking, - time-series perturbations.

This acts as regularization because it reduces the set of acceptable hypotheses to those that behave consistently under the chosen transformations. Thus data augmentation is not merely more data; it is a way of imposing structure.

6.8 Early Stopping

Another important regularization method is *early stopping*. Instead of optimizing until the training objective is minimized as much as possible, one stops training once validation performance stops improving.

Why can this help? Because very long optimization may eventually fit noise or idiosyncrasies of the sample. Stopping early limits how far the optimizer moves into highly adapted solutions.

In many settings, early stopping behaves similarly to explicit norm regularization: it restricts the effective complexity of the fitted model even when no formal penalty is written down.

6.9 Implicit Regularization from Optimization

Modern learning systems often generalize better than one would predict from parameter count alone. One reason is that optimization itself can induce preference for certain kinds of solutions.

Examples include: - gradient descent favoring low-norm or large-margin solutions in some settings, - initialization affecting which minima are reachable, - stochastic optimization adding noise that biases training dynamics.

This is called *implicit regularization*. The objective may allow many minimizing solutions, but the optimizer does not select uniformly among them.

This idea is especially important in deep learning, where explicit penalties alone do not fully explain generalization.

6.10 A Unifying Perspective

Regularization is the deliberate control of complexity through penalties, constraints, data transformations, architecture, or optimization dynamics. Its purpose is not to prevent learning, but to steer learning toward solutions that capture persistent structure rather than accidental sample noise.

The unifying principle is:

$$\text{fit the data} \quad \text{subject to structural preference.}$$

Ridge prefers small norms, lasso prefers sparsity, augmentation prefers invariance, early stopping prefers simpler optimization trajectories, and architecture prefers task-aligned function structure. These are different expressions of the same idea.

6.11 Summary

In this chapter we introduced regularization as the bridge between fitting and structure. We formulated penalized ERM, discussed its constrained counterpart, and studied two central examples: ridge regularization and lasso. We derived the ridge closed form, explained shrinkage, and gave the geometric intuition behind the different behavior of ℓ_1 and ℓ_2 penalties.

We also emphasized that regularization in modern learning goes beyond explicit penalties. Data augmentation, early stopping, architectural design, and optimization dynamics all shape effective complexity and therefore affect generalization. The next chapters will build on this view when we study specific models and their statistical properties.

Chapter 7

Linear Regression and Least Squares

7.1 Introduction

Linear regression is one of the foundational models in machine learning, statistics, and applied mathematics. Its importance is not only historical. It is also the simplest setting in which many central ideas appear clearly: empirical risk minimization, geometry of approximation, orthogonal projection, conditioning, regularization, and statistical interpretation.

This chapter studies least squares in a more mathematical way. We derive the normal equations, interpret least squares as projection onto a linear subspace, discuss uniqueness and pseudoinverses, and explain why ill-conditioning matters in practice. We also briefly connect the optimization problem to the classical statistical noise model.

7.2 Linear Models and Design Matrices

Let the input be $x \in \mathbb{R}^p$ and the output be $y \in \mathbb{R}$. A linear model has the form

$$f_\theta(x) = x^\top \theta,$$

where $\theta \in \mathbb{R}^p$ is the parameter vector.

Given data

$$(x_1, y_1), \ldots, (x_n, y_n),$$

we assemble the *design matrix*

$$X = \begin{bmatrix} - & x_1^\top & - \\ - & x_2^\top & - \\ & \vdots & \\ - & x_n^\top & - \end{bmatrix} \in \mathbb{R}^{n \times p}, \qquad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n.$$

Then the vector of predictions is

$$X\theta \in \mathbb{R}^n.$$

The least-squares problem is

$$\min_{\theta \in \mathbb{R}^p} \|X\theta - y\|^2.$$

Equivalently, one minimizes the empirical squared loss

$$\frac{1}{n}\sum_{i=1}^{n}(x_i^\top \theta - y_i)^2.$$

7.2.1 Polynomial Basis Regression

Linear regression is "linear" in the parameters, not necessarily in the raw input variable. For example, if the input is scalar $t \in \mathbb{R}$ and we choose features

$$\phi(t) = (1, t, t^2, \ldots, t^m)^\top,$$

then

$$f_\theta(t) = \theta_0 + \theta_1 t + \cdots + \theta_m t^m$$

is polynomial regression, but it is still linear regression in the feature vector $\phi(t)$.

7.3 Least Squares as Orthogonal Projection

The least-squares problem has a clean geometric interpretation. The set of all prediction vectors

$$\{X\theta : \theta \in \mathbb{R}^p\}$$

is the column space of X, denoted $\mathrm{col}(X) \subseteq \mathbb{R}^n$.

Thus least squares asks for the vector in $\mathrm{col}(X)$ closest to y:

$$\min_{z \in \mathrm{col}(X)} \|z - y\|^2.$$

[Projection Interpretation of Least Squares] A vector $\hat{\theta}$ is a least-squares minimizer if and only if the fitted vector $X\hat{\theta}$ is the orthogonal projection of y onto $\mathrm{col}(X)$. Equivalently, the residual

$$r = y - X\hat{\theta}$$

is orthogonal to every vector in $\mathrm{col}(X)$.

Proof. If $X\hat{\theta}$ minimizes the distance from y to $\mathrm{col}(X)$, then by the geometry of Euclidean projection, the error vector

$$r = y - X\hat{\theta}$$

must be orthogonal to the subspace $\mathrm{col}(X)$. Conversely, if r is orthogonal to $\mathrm{col}(X)$, then for any $z \in \mathrm{col}(X)$,

$$\|y - z\|^2 = \|r + (X\hat{\theta} - z)\|^2 = \|r\|^2 + \|X\hat{\theta} - z\|^2,$$

since $r \perp (X\hat{\theta} - z)$. Hence $\|y - z\|^2 \geq \|r\|^2$, so $X\hat{\theta}$ is the closest point. □

This theorem is the geometric core of least squares.

7.3.1 Orthogonality of the Residual

Since r is orthogonal to the column space of X, it is orthogonal to each column of X. In matrix form,

$$X^\top(y - X\hat{\theta}) = 0.$$

These are exactly the normal equations.

7.4 Normal Equations and Pseudoinverses

Differentiating the objective

$$\|X\theta - y\|^2 = (X\theta - y)^\top(X\theta - y)$$

gives

$$\nabla_\theta \|X\theta - y\|^2 = 2X^\top(X\theta - y).$$

Setting the gradient to zero yields the normal equations:

$$X^\top X\theta = X^\top y.$$

[Least-Squares Minimizer] A vector $\hat{\theta}$ minimizes $\|X\theta - y\|^2$ if and only if it satisfies

$$X^\top X\hat{\theta} = X^\top y.$$

If $X^\top X$ is invertible, then the minimizer is unique and given by

$$\hat{\theta} = (X^\top X)^{-1}X^\top y.$$

7.4.1 Uniqueness Conditions

The matrix $X^\top X$ is invertible exactly when the columns of X are linearly independent, that is, when $\mathrm{rank}(X) = p$. In that case, the least-squares minimizer is unique.

If $\mathrm{rank}(X) < p$, then the minimizer need not be unique. Different parameter vectors may produce the same fitted values.

7.4.2 Pseudoinverse

A standard canonical solution in the rank-deficient case is given by the Moore–Penrose pseudoinverse:

$$\hat{\theta} = X^{+}y.$$

This yields the minimum-norm least-squares solution among all minimizers.

7.5 Rank Deficiency and Ill-Conditioning

Two different issues must be distinguished.

7.5.1 Rank Deficiency

If the columns of X are linearly dependent, then $X^\top X$ is singular. This means the parameter vector is not uniquely identifiable. The fitted vector $X\hat{\theta}$ is still uniquely determined as the projection of y onto $\mathrm{col}(X)$, but the coefficients may not be unique.

This often occurs when: - one feature is a linear combination of others, - there are more parameters than informative directions in the data, - polynomial features are redundant or nearly redundant.

7.5.2 Ill-Conditioning

Even if $X^\top X$ is invertible, it may be close to singular. Then the problem is *ill-conditioned*: small perturbations in data can produce large changes in the estimated coefficients.

A standard indicator is the condition number. If the singular values of X vary widely, then inversion becomes unstable. Ill-conditioning is especially common when features are highly correlated.

This matters computationally and statistically: - numerical errors are amplified, - fitted coefficients become unstable, - interpretation becomes unreliable.

Regularization methods such as ridge regression are often introduced partly to address this instability.

7.6 Worked Examples

7.6.1 2D Geometric Least Squares

Suppose

$$X = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad y = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Then $\mathrm{col}(X)$ is the line through $(1, 2)^\top$ in $\mathbb{R}^2$. Least squares projects y onto this line.

The solution is

$$\hat{\theta} = \frac{X^\top y}{X^\top X} = \frac{1 \cdot 1 + 2 \cdot 3}{1^2 + 2^2} = \frac{7}{5}.$$

So the fitted vector is

$$X\hat{\theta} = \frac{7}{5}\begin{bmatrix}1\\2\end{bmatrix} = \begin{bmatrix}7/5\\14/5\end{bmatrix}.$$

The residual is

$$r = \begin{bmatrix}1\\3\end{bmatrix} - \begin{bmatrix}7/5\\14/5\end{bmatrix} = \begin{bmatrix}-2/5\\1/5\end{bmatrix},$$

and indeed

$$X^\top r = [1\ 2]\begin{bmatrix}-2/5\\1/5\end{bmatrix} = 0.$$

7.6.2 Polynomial Basis Regression

Given scalar inputs $t_1, \ldots, t_n$, choose basis functions

$$1,\ t,\ t^2.$$

Then the design matrix is

$$X = \begin{bmatrix}1 & t_1 & t_1^2\\1 & t_2 & t_2^2\\\vdots & \vdots & \vdots\\1 & t_n & t_n^2\end{bmatrix}.$$

Least squares in this basis fits the best quadratic approximation in empirical squared error. The geometry is the same as before: one projects y onto the span of the three feature columns.

7.7 Statistical Noise Model and Gauss–Markov Intuition

Least squares also has a natural statistical interpretation. Suppose the data satisfy

$$y = X\theta^\star + \varepsilon,$$

where $\theta^\star \in \mathbb{R}^p$ is an unknown parameter and the noise vector ε has mean zero:

$$\mathbb{E}[\varepsilon] = 0.$$

Then least squares estimates $\theta^\star$ by finding the coefficient vector whose fitted values best match the observations in squared error.

If one further assumes that the noise has constant variance and is uncorrelated, then least squares enjoys an important optimality property: among linear unbiased estimators, it has minimal variance. This is the content of the Gauss–Markov theorem. We will not prove it here, but its intuition is important: under a simple noise model, least squares is not only geometrically natural but statistically efficient within a broad class.

7.7.1 Interpretation

The fitted value $X\hat{\theta}$ can be interpreted as the signal component extracted from y, while the residual

$$y - X\hat{\theta}$$

is the unexplained part. The projection viewpoint and the statistical viewpoint therefore complement each other: - geometry explains orthogonality and normal equations, - statistics explains estimation under noise.

7.8 A Unifying Perspective

Linear regression is much more than a simple predictive model. It is the first setting in which approximation, optimization, geometry, and statistics come together in a fully analyzable form. Least squares is simultaneously: - minimization of empirical squared loss, - projection onto a linear subspace, - solution of the normal equations, - estimation under a linear noise model.

This is why linear regression remains central even in the age of deep learning: it is the cleanest laboratory for understanding ideas that reappear in more complex models.

7.9 Summary

In this chapter we studied linear regression and least squares from a mathematical viewpoint. We introduced the design matrix, derived the normal equations, and showed that least squares is the orthogonal projection of the target vector onto the column space of the design matrix. We discussed uniqueness through full column rank, introduced the pseudoinverse for rank-deficient cases, and explained why ill-conditioning makes estimation unstable.

We also connected least squares to the classical statistical noise model and stated the Gauss–Markov intuition. These ideas form the basis for much of supervised learning and prepare the way for logistic regression, kernel methods, and regularized estimators.

Chapter 8

Logistic Regression and Classification

8.1 Introduction

Linear regression is suited to real-valued prediction, but classification requires a different output model. In binary classification, the target is typically

$$Y \in \{0, 1\},$$

so a predictor should produce either a class label or a probability of class membership. A linear function

$$x^\top \theta$$

alone is not appropriate as a probability, since it may lie outside $[0, 1]$. Logistic regression resolves this by combining a linear score with a nonlinear link function that maps $\mathbb{R}$ into $(0, 1)$.

This chapter develops logistic regression from both probabilistic and optimization viewpoints. We introduce the sigmoid function, interpret coefficients through log-odds, derive the likelihood and cross-entropy loss, and compute the gradient and Hessian. We then extend the construction to multiclass classification through softmax regression.

8.2 From Regression to Classification

In regression, the model predicts a numerical value. In classification, the task is to predict a discrete class. For binary classification, one often seeks either:

- a class prediction in $\{0, 1\}$, or
- a probability estimate for class 1.

A natural idea is to begin with a linear score

$$z = x^\top \theta,$$

and convert it into a probability. The conversion should be monotone: larger scores should correspond to higher class probability. It should also map all real numbers to the interval $(0, 1)$.

This leads to the logistic sigmoid.

8.3 Bernoulli Models and the Logistic Link

8.3.1 Sigmoid Function

The logistic sigmoid is

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It maps $\mathbb{R}$ to $(0, 1)$ and satisfies

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

[Logistic Regression Model] In binary logistic regression, the conditional probability of class 1 given input x is modeled as

$$\mathbb{P}(Y = 1 \mid X = x) = \sigma(x^\top \theta),$$

and hence

$$\mathbb{P}(Y = 0 \mid X = x) = 1 - \sigma(x^\top \theta).$$

Thus the output is Bernoulli with parameter $\sigma(x^\top \theta)$:

$$Y \mid X = x \sim \text{Bernoulli}(\sigma(x^\top \theta)).$$

8.3.2 Log-Odds Interpretation

A key feature of logistic regression is that it is linear in the log-odds. Let

$$p(x) = \mathbb{P}(Y = 1 \mid X = x).$$

Then

$$p(x) = \sigma(x^\top \theta) \quad \Longleftrightarrow \quad \frac{p(x)}{1 - p(x)} = e^{x^\top \theta}.$$

Taking logarithms gives

$$\log \frac{p(x)}{1 - p(x)} = x^\top \theta.$$

So logistic regression assumes that the *log-odds* are a linear function of the features.

This gives a useful interpretation: each coefficient describes how a one-unit change in a feature changes the log-odds, holding other features fixed.

8.4 Maximum Likelihood Derivation

Suppose we observe data

$$(x_1, y_1), \ldots, (x_n, y_n), \qquad y_i \in \{0, 1\}.$$

Under the logistic model,

$$p_i := \mathbb{P}(Y_i = 1 \mid X_i = x_i) = \sigma(x_i^\top \theta).$$

Since Y_i is Bernoulli,

$$\mathbb{P}(Y_i = y_i \mid X_i = x_i) = p_i^{y_i}(1 - p_i)^{1-y_i}.$$

Assuming conditional independence across observations, the likelihood is

$$L(\theta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}.$$

Taking logarithms,

$$\ell(\theta) = \sum_{i=1}^{n} \Big(y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big).$$

Maximum likelihood chooses θ to maximize $\ell(\theta)$, or equivalently to minimize the negative log-likelihood:

$$-\ell(\theta) = -\sum_{i=1}^{n} \Big(y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big).$$

8.4.1 Cross-Entropy Loss

Dividing by n, the empirical objective becomes

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \Big(y_i \log \sigma(x_i^\top \theta) + (1 - y_i) \log(1 - \sigma(x_i^\top \theta)) \Big).$$

This is the binary *cross-entropy* or *logistic loss*.

[Cross-Entropy from Bernoulli Likelihood] For binary logistic regression, minimizing empirical cross-entropy is equivalent to maximum likelihood estimation under the Bernoulli model

$$Y \mid X = x \sim \text{Bernoulli}(\sigma(x^\top \theta)).$$

Thus logistic regression is not just a heuristic classifier; it is a probabilistic model estimated by likelihood.

8.5 Optimization: Gradient and Hessian

Let

$$z_i = x_i^\top \theta, \qquad p_i = \sigma(z_i).$$

Then the objective is

$$J(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\big(y_i \log p_i + (1-y_i)\log(1-p_i)\big).$$

8.5.1 Gradient

Using $\sigma'(z) = \sigma(z)(1-\sigma(z))$, one obtains

$$\nabla J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)x_i.$$

In matrix form, if

$$p = \begin{bmatrix} p_1 \\ \vdots \\ p_n \end{bmatrix}, \qquad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix},$$

then

$$\nabla J(\theta) = \frac{1}{n}X^\top(p-y).$$

8.5.2 Hessian

Differentiating again gives

$$\nabla^2 J(\theta) = \frac{1}{n}X^\top W X,$$

where

$$W = \operatorname{diag}\big(p_1(1-p_1), \ldots, p_n(1-p_n)\big).$$

Since each diagonal entry is nonnegative, W is positive semidefinite, and so is $X^\top W X$.

[Convexity of Logistic Loss] The logistic regression objective $J(\theta)$ is convex in θ.

This is a major advantage: unlike deep-network objectives, logistic regression has a convex optimization landscape.

8.6 Linear Decision Boundaries

Although logistic regression models probabilities, its classification rule is often obtained by thresholding:

$$\hat{y} = \begin{cases} 1, & \sigma(x^\top\theta) \geq \frac{1}{2}, \\ 0, & \sigma(x^\top\theta) < \frac{1}{2}. \end{cases}$$

Since $\sigma(z) \geq 1/2$ exactly when $z \geq 0$, this becomes

$$\hat{y} = \begin{cases} 1, & x^\top \theta \geq 0, \\ 0, & x^\top \theta < 0. \end{cases}$$

Thus logistic regression induces a linear decision boundary:

$$x^\top \theta = 0.$$

So logistic regression is linear as a classifier, even though it is nonlinear as a probability map.

8.7 Cross-Entropy and Calibration

Cross-entropy is more informative than zero-one loss because it takes confidence into account. Predicting the correct class with probability 0.99 is better than predicting it with probability 0.51, even though both give the same label.

This connects logistic regression to *calibration*. A probabilistic classifier is calibrated if its predicted probabilities match observed frequencies. Logistic regression is often used not only for classification but also for probability estimation, especially when interpretability matters.

Cross-entropy encourages truthful probability prediction: assigning low probability to the true class is penalized heavily.

8.8 Regularized Logistic Regression

As with least squares, one often adds regularization:

$$\min_\theta J(\theta) + \lambda \Omega(\theta).$$

A common choice is $\Omega(\theta) = \|\theta\|^2$, giving ℓ_2-regularized logistic regression:

$$\min_\theta -\frac{1}{n} \sum_{i=1}^{n} \Big(y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big) + \lambda \|\theta\|^2.$$

This improves stability and reduces overfitting, especially in high dimensions or when features are correlated.

8.9 Multiclass Softmax Regression

Logistic regression extends naturally to K classes. Let

$$Y \in \{1, \ldots, K\}.$$

For each class k, assign a score

$$s_k(x) = w_k^\top x,$$

and convert scores to probabilities using softmax:

$$\mathbb{P}(Y = k \mid X = x) = \frac{e^{w_k^\top x}}{\sum_{j=1}^K e^{w_j^\top x}}.$$

[Softmax Regression] In multiclass softmax regression, the conditional class probabilities are

$$p_k(x; \Theta) = \frac{e^{w_k^\top x}}{\sum_{j=1}^K e^{w_j^\top x}}, \qquad k = 1, \ldots, K,$$

where $\Theta = (w_1, \ldots, w_K)$.

8.9.1 Softmax Likelihood

Given training data, the likelihood is

$$L(\Theta) = \prod_{i=1}^n p_{y_i}(x_i; \Theta),$$

so the negative log-likelihood is

$$J(\Theta) = -\frac{1}{n} \sum_{i=1}^n \log p_{y_i}(x_i; \Theta).$$

Equivalently,

$$J(\Theta) = -\frac{1}{n} \sum_{i=1}^n \left(w_{y_i}^\top x_i - \log \sum_{j=1}^K e^{w_j^\top x_i} \right).$$

This is multiclass cross-entropy.

8.9.2 Gradient Formula

If we use one-hot targets $y_{ik} \in \{0, 1\}$, the gradient with respect to class weight vector w_k is

$$\nabla_{w_k} J(\Theta) = \frac{1}{n} \sum_{i=1}^n \big(p_k(x_i; \Theta) - y_{ik}\big) x_i.$$

This is the multiclass analogue of the binary logistic gradient.

8.9.3 Softmax Classifier

The predicted class is

$$\hat{y} = \arg\max_k \; p_k(x; \Theta).$$

Since softmax is monotone in the scores, this is equivalent to

$$\hat{y} = \arg\max_k \; w_k^\top x.$$

8.10 Worked Examples

8.10.1 Binary Gradient Interpretation

For binary logistic regression,

$$\nabla J(\theta) = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i) x_i.$$

So each data point contributes: - little when the model predicts correctly with confidence, - strongly when the prediction is poor.

This matches the intuition that optimization adjusts parameters in proportion to prediction error.

8.10.2 Softmax Classifier

Suppose there are three classes with score vectors w_1, w_2, w_3. Then

$$p_k(x) = \frac{e^{w_k^\top x}}{e^{w_1^\top x} + e^{w_2^\top x} + e^{w_3^\top x}}.$$

The model predicts the class with largest score. Thus softmax regression is a probabilistic model whose decision rule is still based on linear comparisons of class scores.

8.11 A Unifying Perspective

Logistic regression is the natural classification analogue of linear regression, but with important differences. It combines: - a linear score, - a probabilistic Bernoulli or categorical output model, - maximum likelihood estimation, - cross-entropy loss, - convex optimization.

Its coefficients are interpretable through log-odds, its loss has a clear statistical origin, and its classifier has a linear decision boundary. These features make logistic regression one of the most important baseline models in supervised learning.

8.12 Summary

In this chapter we introduced logistic regression as a probabilistic model for classification. For binary classification, the model uses the sigmoid function to map a linear score to a Bernoulli probability, and the log-odds are linear in the features. We derived the likelihood, showed that maximum likelihood leads to cross-entropy minimization, and computed the gradient and Hessian.

We then explained why logistic regression induces linear decision boundaries and briefly discussed calibration and regularization. Finally, we extended the model to multiclass classification using softmax regression and gave the corresponding likelihood and gradient formulas. These ideas prepare the way for more flexible nonlinear classifiers while retaining a clear probabilistic interpretation.

Chapter 9

Feature Maps and Nonlinear Models

9.1 Introduction

Linear models are attractive because they are simple, interpretable, and mathematically tractable. But many real prediction tasks are not linear in the raw input variables. A straight line cannot separate XOR-type patterns, a single affine function cannot represent curved decision boundaries, and many regression problems involve nonlinear dependence on the input.

A standard solution is not to abandon linear models entirely, but to change the representation. Instead of applying a linear model directly to $x \in \mathbb{R}^d$, we first map x into a new feature space and then apply a linear model there. This idea turns linear prediction into a flexible framework for nonlinear learning.

This chapter develops that viewpoint. We introduce feature maps, basis expansions, and dictionary representations; study polynomial and radial features; and explain how similarity-based methods lead naturally to kernels. The goal is to make kernels feel like a continuation of feature engineering rather than an abrupt new topic.

9.2 Why Linear Models Become Powerful After Feature Engineering

Suppose we replace the raw input x by a feature map

$$\phi : \mathcal{X} \to \mathbb{R}^m.$$

Then a linear predictor in feature space has the form

$$f(x) = w^\top \phi(x).$$

This is linear in the features $\phi(x)$, but it may be highly nonlinear as a function of the original input x.

[Feature Map] A feature map is a function

$$\phi : \mathcal{X} \to \mathbb{R}^m$$

that transforms an input into a feature vector.

This simple idea is extremely powerful. If the representation $\phi(x)$ captures the right nonlinear structure, then linear models become much more expressive. In this sense, much of machine learning can be viewed as the search for a representation in which the task becomes approximately linear.

9.2.1 Representation Lifting

The map ϕ is often called a *lifting* of the input into a higher-dimensional space. The purpose is not merely to increase dimension, but to make patterns easier to separate or approximate.

For example, data that are not linearly separable in $\mathbb{R}^2$ may become linearly separable after lifting into $\mathbb{R}^3$ or higher.

9.3 Basis Expansions and Dictionary Views

A common way to build feature maps is through basis functions. Choose functions

$$\phi_1, \phi_2, \ldots, \phi_m$$

on the input space and define

$$\phi(x) = \big(\phi_1(x), \ldots, \phi_m(x)\big)^\top.$$

Then the model becomes

$$f(x) = \sum_{j=1}^{m} w_j \phi_j(x).$$

This can be interpreted in two equivalent ways:

- as a linear model in transformed coordinates;
- as a linear combination of basis functions.

The functions ϕ_j form a *dictionary* of building blocks. Learning then amounts to choosing coefficients w_j.

9.3.1 Examples of Basis Functions

Typical choices include:

- monomials $1, x, x^2, \ldots,$
- trigonometric features,
- radial basis functions centered at selected points,
- indicator or histogram features.

Thus feature engineering means selecting functions that express useful structure of the task.

9.4 Polynomial and Radial Features

9.4.1 Polynomial Features

For input $x = (x_1, \ldots, x_d)$, a polynomial feature map of degree up to r includes monomials such as

$$x_i, \qquad x_i x_j, \qquad x_i^2, \qquad x_i x_j x_k,$$

up to total degree r.

For example, in two dimensions a degree-2 map may be

$$\phi(x_1, x_2) = \big(1, x_1, x_2, x_1^2, x_1 x_2, x_2^2\big).$$

Then a linear classifier in feature space corresponds to a quadratic decision boundary in the original coordinates.

[Polynomial Decision Boundary] If

$$f(x_1, x_2) = w^\top \phi(x_1, x_2)$$

with the degree-2 map above, then the decision boundary

$$f(x_1, x_2) = 0$$

is a quadratic curve. Thus linear classification in feature space becomes nonlinear classification in the original space.

9.4.2 Radial Basis Features

Another important family is radial basis features:

$$\phi_j(x) = \exp\left(-\frac{\|x - c_j\|^2}{2\sigma^2}\right),$$

where c_j is a center and $\sigma > 0$ is a scale parameter.

These features respond strongly when x is near the center c_j and decay with distance. A linear model built from such features can approximate complex smooth functions by combining local bumps.

[RBF Features] If we choose centers $c_1, \ldots, c_m$, then

$$f(x) = \sum_{j=1}^{m} w_j \exp\left(-\frac{\|x - c_j\|^2}{2\sigma^2}\right)$$

is a weighted sum of localized responses. This gives a flexible nonlinear model while remaining linear in the parameters w_j.

9.5 Curse and Blessing of Higher Dimensions

Lifting data into higher-dimensional feature spaces can help, but it also creates new challenges.

9.5.1 Blessing of Higher Dimensions

Higher-dimensional representations can make nonlinear patterns easier to separate. A problem that looks tangled in raw coordinates may become simple after a suitable transformation. This is the main advantage of feature lifting.

9.5.2 Curse of Higher Dimensions

At the same time, higher-dimensional spaces can be statistically and computationally difficult:

- the number of features may grow very quickly,
- data may become sparse,
- overfitting becomes easier,
- optimization and storage can become expensive.

For polynomial features, the number of monomials grows rapidly with degree and ambient dimension. Thus higher dimension is both a blessing and a curse: it increases expressive power but also increases complexity.

9.6 Similarity-Based Learning and the Kernel Idea

Explicit feature construction can become expensive or unwieldy. This motivates a different viewpoint based on similarity.

Suppose we have a feature map $\phi : \mathcal{X} \to \mathbb{R}^m$. Then the inner product between two lifted points is

$$\phi(x)^\top \phi(x').$$

This quantity measures similarity in feature space.

[Kernel] A kernel is a function

$$K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$$

such that

$$K(x, x') = \langle \phi(x), \phi(x') \rangle$$

for some feature map ϕ.

So a kernel is simply an inner product in a feature space, possibly without writing the feature map explicitly.

9.6.1 Why This Matters

If a learning algorithm depends only on inner products between feature vectors, then one may replace

$$\phi(x)^\top \phi(x')$$

by

$$K(x, x')$$

directly. This allows one to work implicitly in a high-dimensional, or even infinite-dimensional, feature space without ever constructing coordinates explicitly.

This is the core idea behind kernel methods.

9.6.2 Examples

Polynomial kernel.

$$K(x, x') = (x^\top x' + c)^r.$$

This corresponds to inner products among polynomial features.

Gaussian RBF kernel.

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2} \right).$$

This corresponds to a very rich feature space and captures similarity through distance.

Thus kernels arise naturally from the feature-map viewpoint: they are similarity functions induced by nonlinear representations.

9.7 Explicit and Implicit Representations

There are two main ways to use nonlinear features.

9.7.1 Explicit Representation

Choose a feature map $\phi(x)$ and build a linear model directly in that space:

$$f(x) = w^\top \phi(x).$$

This is transparent and often computationally convenient when the feature dimension is moderate.

9.7.2 Implicit Representation

Instead of constructing $\phi(x)$ explicitly, define a kernel

$$K(x, x') = \langle \phi(x), \phi(x') \rangle$$

and let the algorithm operate through pairwise similarities. This is useful when the feature space is very large or infinite-dimensional.

The two viewpoints are mathematically connected. The difference is computational and conceptual: explicit features describe coordinates, while kernels describe pairwise similarity.

9.8 Feature Learning vs Hand-Crafted Features

Classical machine learning relied heavily on hand-crafted feature maps. One would design basis functions, similarity measures, or domain-specific representations and then apply a relatively simple model on top.

Modern deep learning shifts much of this burden to the model itself. Instead of manually specifying ϕ, the network learns intermediate representations from data. In this sense, deep learning can be seen as automated feature learning.

The contrast is:

- **hand-crafted features:** human-designed basis expansions or kernels;
- **learned features:** representations optimized jointly with the task.

This does not make the older viewpoint obsolete. On the contrary, feature maps and kernels provide the conceptual bridge to understanding why learned representations matter so much.

9.9 A Unifying Perspective

Nonlinear learning often begins by making linear models act in richer spaces. A feature map lifts the data, a basis expansion provides building blocks, radial and polynomial features produce nonlinear decision rules, and kernels replace explicit coordinates by similarity in feature space.

The central idea is simple:

$$\text{nonlinearity in input space} \quad = \quad \text{linearity in a transformed space.}$$

This idea motivates kernel methods and also foreshadows deep learning, where the transformation itself becomes learned rather than fixed in advance.

9.10 Summary

In this chapter we introduced feature maps as a way to turn linear models into nonlinear predictors. We studied basis expansions, polynomial features, and radial basis features, and explained how lifting into higher-dimensional spaces can both help and complicate learning. We then introduced kernels as inner products in feature space, showing how similarity-based learning arises naturally from the feature-map viewpoint.

Finally, we contrasted explicit hand-crafted features with learned representations. This sets the stage for kernel methods and, more broadly, for the idea that the right representation can be as important as the predictive model itself.

Chapter 10

Support Vector Machines

10.1 Introduction

Support vector machines are among the most important geometric methods in supervised learning. They begin with a simple idea: among all separating hyperplanes, prefer the one that separates the classes with the largest margin. This turns classification into an optimization problem with a strong geometric interpretation.

SVMs are also important conceptually. They connect linear classification, convex optimization, duality, sparsity in the decision rule, and kernels. In this chapter we develop the hard-margin and soft-margin formulations, explain the role of support vectors, and show how kernels extend SVMs to nonlinear classification.

10.2 Linear Separability and Geometric Margin

Consider binary classification with labels

$$y_i \in \{-1, +1\}, \qquad i = 1, \ldots, n,$$

and inputs $x_i \in \mathbb{R}^d$. A linear classifier has the form

$$f(x) = \operatorname{sign}(w^\top x + b).$$

The decision boundary is the hyperplane

$$w^\top x + b = 0.$$

The prediction depends on which side of this hyperplane the point lies.

10.2.1 Functional and Geometric Margin

For a labeled example (x_i, y_i), the *functional margin* is

$$y_i(w^\top x_i + b).$$

This is positive if the point is correctly classified and larger when it lies farther from the boundary in score units.

Because scaling (w, b) changes this quantity, one introduces the geometric notion.

[Geometric Margin] The geometric margin of (x_i, y_i) with respect to (w, b) is

$$\gamma_i = \frac{y_i(w^\top x_i + b)}{\|w\|}.$$

The margin of the classifier on the dataset is

$$\gamma = \min_{1 \le i \le n} \frac{y_i(w^\top x_i + b)}{\|w\|}.$$

This quantity is scale-invariant and measures the signed Euclidean distance to the decision boundary.

10.2.2 Why Margin Matters

A larger margin means the classifier separates the classes more robustly. Geometrically, it places a wider empty region between the two classes. Statistically, large margins are often associated with better generalization.

10.3 Hard-Margin SVM

Assume first that the data are linearly separable, so there exists (w, b) such that

$$y_i(w^\top x_i + b) > 0 \qquad \text{for all } i.$$

Since scaling does not change the classifier, we may normalize so that

$$y_i(w^\top x_i + b) \ge 1 \qquad \text{for all } i.$$

Under this normalization, the geometric margin is

$$\gamma = \frac{1}{\|w\|}.$$

Thus maximizing the margin is equivalent to minimizing $\|w\|$.

[Hard-Margin SVM] The hard-margin support vector machine solves

$$\min_{w,b} \frac{1}{2}\|w\|^2 \qquad \text{subject to} \qquad y_i(w^\top x_i + b) \ge 1, \quad i = 1, \ldots, n.$$

This is a convex quadratic optimization problem with linear constraints.

10.3.1 Geometry

The constraints

$$y_i(w^\top x_i + b) \geq 1$$

define two supporting hyperplanes:

$$w^\top x + b = 1 \qquad \text{and} \qquad w^\top x + b = -1.$$

The decision boundary lies halfway between them. The distance between these two margin hyperplanes is

$$\frac{2}{\|w\|}.$$

So minimizing $\|w\|^2$ maximizes the separation width.

10.4 Soft-Margin SVM and Slack Variables

Perfect linear separability is often unrealistic. Some points may overlap, be noisy, or be mislabeled. To allow controlled violations, one introduces slack variables $\xi_i \geq 0$.

[Soft-Margin SVM] The soft-margin SVM solves

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

subject to

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0, \quad i = 1, \ldots, n.$$

Here: - $\frac{1}{2}\|w\|^2$ encourages a large margin, - $\sum_i \xi_i$ penalizes violations, - $C > 0$ controls the tradeoff.

If $\xi_i = 0$, the point satisfies the margin constraint. If $0 < \xi_i \leq 1$, it is correctly classified but inside the margin. If $\xi_i > 1$, it is misclassified.

10.4.1 Hinge Loss Form

The soft-margin problem is equivalent to the unconstrained optimization problem

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \max\big(0, 1 - y_i(w^\top x_i + b)\big).$$

The function

$$\ell_{\text{hinge}}(z) = \max(0, 1 - z)$$

is called the hinge loss.

This makes SVMs comparable to logistic regression: - logistic regression uses logistic loss, - SVMs use hinge loss.

10.5 Duality and Support Vectors

The SVM has an especially important dual formulation. For the hard-margin case, the Lagrangian is

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \big(y_i(w^\top x_i + b) - 1\big), \qquad \alpha_i \geq 0.$$

Setting derivatives with respect to w and b to zero gives

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i, \qquad \sum_{i=1}^{n} \alpha_i y_i = 0.$$

Substituting back yields the dual problem:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j \, x_i^\top x_j$$

subject to

$$\alpha_i \geq 0, \qquad \sum_{i=1}^{n} \alpha_i y_i = 0.$$

10.5.1 Primal-to-Dual Sketch

The dual is useful because: - it depends only on inner products $x_i^\top x_j$, - it expresses the classifier as a combination of training points, - it leads directly to kernels.

Once α is found, the classifier is

$$f(x) = \operatorname{sign}\left(\sum_{i=1}^{n} \alpha_i y_i \, x_i^\top x + b \right).$$

10.5.2 Support Vectors

Only points with $\alpha_i > 0$ appear in the decision rule. These are the *support vectors*. They are the training points that lie on or inside the margin and therefore determine the classifier.

Points far from the boundary typically have $\alpha_i = 0$ and do not affect the final decision function.

This sparsity is one of the distinctive features of SVMs.

10.5.3 KKT Interpretation

The Karush–Kuhn–Tucker conditions provide an accessible interpretation. In the hard-margin case, they include

$$\alpha_i \geq 0, \qquad y_i(w^\top x_i + b) - 1 \geq 0, \qquad \alpha_i \big(y_i(w^\top x_i + b) - 1\big) = 0.$$

The last condition is called complementary slackness.

Its meaning is simple: - if a point lies strictly outside the margin, then $\alpha_i = 0$; - if $\alpha_i > 0$, then the point lies exactly on the margin.

So support vectors are precisely the active geometric constraints.

10.6 Kernel SVMs

The dual formulation depends on the training data only through inner products $x_i^\top x_j$. This makes the kernel extension immediate.

Let

$$K(x, x') = \langle \phi(x), \phi(x') \rangle$$

be a kernel corresponding to a feature map ϕ. Replacing inner products by kernels gives the decision rule

$$f(x) = \operatorname{sign}\left(\sum_{i=1}^{n} \alpha_i y_i \, K(x_i, x) + b \right).$$

Thus the classifier is linear in feature space but nonlinear in the original input space.

10.6.1 Kernel SVM Optimization

The dual becomes

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} \alpha_i \alpha_j y_i y_j \, K(x_i, x_j)$$

subject to the same constraints

$$\alpha_i \geq 0, \qquad \sum_{i=1}^{n} \alpha_i y_i = 0$$

for the hard-margin case, with additional upper bounds in the soft-margin setting.

10.6.2 Why Kernels Fit Naturally Here

Kernels are not an add-on to SVMs; they fit naturally because the dual depends only on similarities. This makes SVMs one of the clearest examples of implicit nonlinear learning through feature space geometry.

Common choices include:

$$K(x, x') = (x^\top x' + c)^r$$

for polynomial kernels, and

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2} \right)$$

for Gaussian RBF kernels.

10.7 Worked Geometric and Optimization Examples

10.7.1 Margin in a Toy Example

Suppose two classes in one dimension are separated by the threshold $x = 0$, with the closest positive point at $x = 1$ and the closest negative point at $x = -1$. Then one possible classifier is

$$f(x) = \text{sign}(x).$$

Here $w = 1$, $b = 0$, and the margin is

$$\gamma = \frac{1}{\|w\|} = 1.$$

The width between the margin hyperplanes is

$$\frac{2}{\|w\|} = 2.$$

10.7.2 Role of Support Vectors

If many training points lie far from the decision boundary and only a few lie near it, then only the near-boundary points matter for the SVM solution. Removing a non-support-vector often leaves the classifier unchanged, while moving a support vector can change the boundary substantially.

This is why the method is called a support vector machine.

10.7.3 Hinge vs Logistic Loss

For a score $z = y(w^\top x + b)$: - hinge loss is

$$\max(0, 1 - z),$$

- logistic loss is

$$\log(1 + e^{-z}).$$

Both penalize small margins and misclassification, but they differ: - hinge loss becomes exactly zero once $z \geq 1$, - logistic loss remains smooth and positive for all finite z.

Thus SVMs emphasize margin constraints sharply, while logistic regression gives a probabilistic smooth penalty.

10.8 A Unifying Perspective

SVMs unify geometry, optimization, and similarity-based learning. Their central principle is:

$$\text{choose the separating hyperplane with maximum margin.}$$

This leads to a convex optimization problem, a sparse decision rule determined by support vectors, and a natural kernel extension for nonlinear classification.

SVMs are therefore more than just another classifier. They are a clean illustration of how inductive bias, convexity, duality, and feature-space geometry come together in supervised learning.

10.9 Summary

In this chapter we introduced support vector machines through the geometry of margin maximization. We defined the geometric margin, formulated the hard-margin and soft-margin SVM optimization problems, and interpreted slack variables as controlled violations of separability. We then sketched the dual formulation, explained why support vectors alone determine the decision rule, and used the KKT conditions to interpret active constraints.

Finally, we showed how kernels extend SVMs to nonlinear classification by replacing inner products with similarities in feature space. This makes SVMs one of the most elegant bridges between linear classification and nonlinear learning.

Chapter 11

Model Evaluation and Selection

11.1 Introduction

A learning algorithm produces a model, but a crucial question remains: how do we know whether the model is good? Evaluation is not merely a technical detail; it determines how we compare models, choose hyperparameters, and report results. Poor evaluation practices can lead to misleading conclusions, over-optimistic performance estimates, and models that fail in real use.

This chapter develops a principled approach to model evaluation and selection. We discuss appropriate performance metrics, validation strategies such as cross-validation, hyperparameter tuning, and common pitfalls including data leakage and improper reuse of test data.

11.2 What Counts as Good Performance?

Let $(X, Y) \sim P$ and let ℓ be a loss function. The true performance of a predictor f is measured by its population risk

$$R(f) = \mathbb{E}[\ell(f(X), Y)].$$

In practice, P is unknown, so we estimate $R(f)$ using data. The goal of evaluation is therefore:

- to estimate $R(f)$ reliably,
- to compare models based on these estimates,
- to select hyperparameters that minimize true risk.

[Risk Estimate] Given a test sample $(x_i, y_i)_{i=1}^m$ independent of training data, the empirical test risk is

$$\widehat{R}_{\text{test}}(f) = \frac{1}{m} \sum_{i=1}^{m} \ell(f(x_i), y_i).$$

A good evaluation procedure produces an estimate that is:

- unbiased or nearly unbiased,
- stable across samples,
- aligned with the task objective.

11.3 Regression and Classification Metrics

Different tasks require different metrics.

11.3.1 Regression Metrics

Common regression losses include:

- Mean squared error (MSE):

$$\frac{1}{n} \sum_{i=1}^{n} (f(x_i) - y_i)^2,$$

- Mean absolute error (MAE):

$$\frac{1}{n} \sum_{i=1}^{n} |f(x_i) - y_i|.$$

MSE penalizes large errors more strongly, while MAE is more robust to outliers.

11.3.2 Classification Metrics

For classification, the simplest metric is accuracy:

$$\frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{f(x_i) = y_i\}.$$

However, accuracy may be misleading in imbalanced settings. Alternative metrics include:

- precision and recall,
- F1-score,
- area under the ROC curve (AUC),
- log-loss (cross-entropy).

11.3.3 Worked Example: Imbalanced Classification

Suppose 95% of examples belong to class 0. A classifier that always predicts 0 achieves 95% accuracy but is useless for detecting class 1. In such cases:

- recall measures detection of the minority class,
- precision measures reliability of positive predictions,
- F1 balances the two.

11.3.4 ROC vs Precision–Recall

ROC curves plot true positive rate vs false positive rate. Precision–recall curves focus on performance for the positive class.

In imbalanced settings:

- ROC curves may appear overly optimistic,
- precision–recall curves give a more informative view.

11.4 Validation Strategies and Cross-Validation

To estimate generalization performance and select models, data are typically split into:

- training set,
- validation set,
- test set.

11.4.1 Hold-Out Validation

A simple strategy is to reserve part of the data as a validation set. The model is trained on the training set and evaluated on the validation set.

11.4.2 Cross-Validation

When data are limited, one uses K-fold cross-validation:

- split data into K folds,
- train on $K - 1$ folds,
- evaluate on the remaining fold,
- repeat for each fold and average results.

This gives a more stable estimate of performance.

11.4.3 Test Set Usage

The test set should be used only once, after model selection, to estimate final performance. Repeated use of the test set introduces bias.

[Bias from Test Reuse] If the test set is used repeatedly to select among models, the reported test performance becomes optimistically biased.

11.5 Hyperparameter Search

Most models depend on hyperparameters, such as:

- regularization strength,
- model complexity,
- learning rate,
- architecture choices.

Hyperparameters are chosen by minimizing validation error.

11.5.1 Search Strategies

Common approaches include:

- grid search,
- random search,
- more advanced adaptive methods.

11.5.2 Worked Example: Tuning Workflow

A typical workflow:

1. split data into training and validation sets,
2. train models with different hyperparameters,
3. evaluate on validation set,
4. select the best hyperparameters,
5. retrain on combined training+validation data,
6. evaluate once on the test set.

This separation prevents optimistic bias.

11.6 Data Leakage, Confounding, and Reproducibility

11.6.1 Data Leakage

Data leakage occurs when information from outside the training set is used improperly during training. Examples include:

- using test data during feature normalization,
- selecting features based on the full dataset,
- leaking future information in time-series tasks.

Leakage leads to overly optimistic performance estimates.

11.6.2 Confounding

Confounding occurs when models exploit spurious correlations rather than causal structure. For example, a model might learn to detect background artifacts instead of the intended signal.

11.6.3 Reproducibility

Reliable evaluation requires:

- clear data splits,
- fixed random seeds where appropriate,
- reporting of variance or confidence intervals,
- transparent experimental procedures.

11.6.4 Confidence Interval Intuition

A test error estimate is itself random. If repeated on different samples, it would vary. Confidence intervals quantify this uncertainty:

- larger test sets give narrower intervals,
- high-variance models produce more variable estimates.

Even without formal derivation, it is important to interpret reported performance as an estimate with uncertainty, not an exact value.

11.7 A Unifying Perspective

Model evaluation connects statistical theory with practical methodology. It answers:

- how to estimate performance,
- how to compare models,
- how to choose hyperparameters,
- how to avoid misleading conclusions.

The guiding principle is separation of roles:

$$\text{training data} \neq \text{validation data} \neq \text{test data}.$$

Violating this separation undermines the reliability of evaluation.

11.8 Summary

In this chapter we studied model evaluation and selection as a methodological discipline. We defined risk estimation, reviewed regression and classification metrics, and explained why different tasks require different evaluation criteria. We introduced validation strategies including cross-validation and emphasized proper use of test data.

We also discussed hyperparameter search, data leakage, confounding effects, and reproducibility. Finally, we noted that performance estimates are subject to uncertainty and should be interpreted accordingly. These principles are essential for trustworthy machine learning practice.

Chapter 12

Optimization in Practice

12.1 Introduction

Most modern learning problems are solved with gradient-based optimization, often at large scale. While Chapter 4 introduced gradient descent in a deterministic setting, practical optimization is typically stochastic, noisy, and implemented with heuristic enhancements such as momentum and adaptive learning rates.

This chapter develops a practical but mathematically interpretable view of optimization in machine learning. We introduce stochastic gradient descent (SGD), explain the role of noise and batch size, and study common variants such as momentum and Adam. We also discuss learning-rate schedules and their effect on convergence and generalization.

12.2 Full-Batch vs Mini-Batch Optimization

Let the objective be

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta),$$

where $\ell_i(\theta) = \ell(f_\theta(x_i), y_i)$.

12.2.1 Full-Batch Gradient Descent

Full-batch gradient descent computes

$$\nabla J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\theta),$$

and updates

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t).$$

This uses all data at every step, which is computationally expensive for large datasets.

12.2.2 Mini-Batch Gradient Descent

Instead, one samples a subset (mini-batch) $B_t \subset \{1, \ldots, n\}$ and uses

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla \ell_i(\theta_t),$$

leading to

$$\theta_{t+1} = \theta_t - \eta g_t.$$

This reduces computation per step and introduces stochasticity.

12.3 Stochastic Gradients and Noise

[Unbiased Gradient Estimate] If the mini-batch is sampled uniformly, then

$$\mathbb{E}[g_t \mid \theta_t] = \nabla J(\theta_t).$$

Thus the stochastic gradient is an unbiased estimator of the true gradient.

12.3.1 Variance of the Gradient Estimate

Although unbiased, g_t has variance:

$$\mathrm{Var}(g_t) \propto \frac{1}{|B_t|}.$$

Smaller batches produce noisier gradients, while larger batches reduce variance.

12.3.2 Stochastic Optimization Intuition

The update

$$\theta_{t+1} = \theta_t - \eta g_t$$

can be viewed as following the true gradient plus noise. This noise has two effects:

- it introduces randomness in the trajectory,
- it may help escape shallow local minima or saddle points.

Thus stochasticity is not only a computational necessity but can also be beneficial.

12.4 Batch Size Effects

Batch size controls the tradeoff between computational efficiency and gradient accuracy.

12.4.1 Small Batches

- high variance gradients,
- more frequent updates,
- potentially better generalization due to noise.

12.4.2 Large Batches

- low variance gradients,
- more stable updates,
- may converge faster in terms of iterations but not always in wall-clock time,
- sometimes worse generalization.

12.4.3 Worked Example: Effect of Batch Size

On a simple quadratic objective, small batches produce a noisy trajectory that oscillates around the optimum, while large batches follow a smoother path. In high-dimensional nonconvex problems, small-batch noise can help explore the landscape more effectively.

12.5 Momentum and Acceleration

Plain SGD can be slow, especially when gradients vary across directions. Momentum accelerates optimization by accumulating past gradients.

12.5.1 Momentum Update

Define a velocity variable v_t:

$$v_{t+1} = \beta v_t + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_{t+1},$$

where $\beta \in [0, 1)$ is the momentum parameter.

12.5.2 Intuition

Momentum averages gradients over time:

- in directions where gradients are consistent, it accelerates,
- in directions with oscillation, it dampens noise.

This is analogous to a physical system with inertia, where updates build up velocity in consistent directions.

12.5.3 Trajectory Example

On an elongated quadratic surface, gradient descent zigzags across steep directions. Momentum smooths this motion and follows the valley more directly, leading to faster convergence.

12.6 Adaptive Methods

Adaptive methods scale updates differently across coordinates based on past gradients.

12.6.1 Adam Optimizer

Adam maintains exponential moving averages of gradients and squared gradients:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) g_t,$$
$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) g_t^2,$$

and updates

$$\theta_{t+1} = \theta_t - \eta \frac{m_{t+1}}{\sqrt{v_{t+1}} + \varepsilon}.$$

12.6.2 Interpretation

Adam adapts the learning rate per coordinate:

- large-gradient coordinates get smaller steps,
- small-gradient coordinates get larger steps.

This can improve stability and reduce the need for manual tuning.

12.6.3 Tradeoffs

Adaptive methods:

- often converge quickly,
- are robust to scaling of features,
- may generalize worse than SGD with momentum in some settings.

12.7 Learning-Rate Schedules and Warmup

The learning rate η plays a crucial role in optimization.

12.7.1 Constant Learning Rate

A fixed η may work initially but can lead to oscillation or slow convergence near the optimum.

12.7.2 Decay Schedules

Common schedules reduce η over time:

- step decay,
- exponential decay,
- cosine annealing.

Reducing the learning rate allows finer convergence.

12.7.3 Warmup

In large-scale training, it is common to start with a small learning rate and increase it gradually:

$$\eta_t \uparrow \text{ over initial iterations.}$$

This stabilizes early training when gradients may be poorly scaled.

12.7.4 Intuition

Learning-rate schedules balance exploration and refinement:

- large η: explore the landscape quickly,
- small η: refine near good solutions.

12.8 A Unifying Perspective

Practical optimization is not just about following gradients, but about managing noise, curvature, and scale. The key ingredients are:

- stochastic gradients for efficiency,
- momentum for acceleration,
- adaptive scaling for stability,
- learning-rate schedules for convergence.

These components interact. For example, batch size affects gradient noise, which interacts with learning rate and momentum. Understanding these interactions is crucial for effective training.

12.9 Summary

In this chapter we studied practical optimization methods used in modern machine learning. We introduced stochastic gradient descent as an unbiased but noisy approximation of full gradients, analyzed the effect of batch size, and discussed how noise influences optimization.

We then introduced momentum as a method for acceleration and adaptive optimizers such as Adam for coordinate-wise scaling. Finally, we examined learning-rate schedules and warmup strategies. Together, these ideas form the core toolkit for training large-scale models.

Chapter 13

Numerical and Statistical Stability

13.1 Introduction

A model may be mathematically well-defined and achieve low training error, yet still behave poorly in practice. Two key reasons are numerical instability and statistical instability. Numerical instability arises from finite-precision computation and ill-conditioned problems. Statistical instability arises when small changes in the dataset lead to large changes in the learned model.

This chapter makes these notions concrete. We study conditioning, floating-point effects, sensitivity to perturbations, and variability under resampling. Stability is not only a technical concern; it is a prerequisite for reliable and trustworthy machine learning.

13.2 What Stability Means Numerically

Computations in practice are carried out in finite precision. Real numbers are approximated, and arithmetic operations introduce rounding errors.

A numerical procedure is *stable* if small errors in inputs or intermediate computations lead to small errors in outputs. Conversely, an unstable procedure can amplify tiny perturbations into large deviations.

In learning problems, numerical instability may manifest as:

- large variation in fitted parameters,
- sensitivity to scaling of features,
- failure of optimization to converge reliably.

13.3 Conditioning of Learning Problems

The conditioning of a problem measures how sensitive its solution is to perturbations in the input.

[Condition Number] For a matrix A, the condition number is

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|,$$

when A is invertible.

In linear regression, the matrix of interest is $X^\top X$. If $\kappa(X^\top X)$ is large, the problem is ill-conditioned.

13.3.1 Perturbation Intuition

Suppose we solve

$$A\theta = b.$$

If A is well-conditioned, small changes in b produce small changes in θ. If A is ill-conditioned, small perturbations in b may produce large changes in θ.

In least squares, noise in y or small changes in X can be greatly amplified when $X^\top X$ is nearly singular.

13.3.2 Consequences

Ill-conditioning leads to:

- unstable parameter estimates,
- sensitivity to noise,
- numerical difficulties in inversion,
- poor generalization if instability reflects overfitting.

Regularization (e.g., ridge regression) improves conditioning by adding a stabilizing term.

13.4 Floating-Point Issues

Computers represent numbers using finite precision. This leads to:

- rounding errors,
- loss of significance,
- accumulation of numerical error.

13.4.1 Example: Loss of Significance

Subtracting nearly equal numbers can lead to large relative error. In iterative algorithms, such effects may accumulate.

13.4.2 Practical Implications

In machine learning:

- gradients may become inaccurate,
- optimization may stall or diverge,
- normalization and scaling become important.

Well-designed algorithms aim to be robust to such effects.

13.5 Sensitivity to Preprocessing and Scaling

Many learning algorithms are sensitive to the scale of input features.

13.5.1 Feature Scaling

If one feature has much larger magnitude than others, it can dominate the objective and distort optimization. For example, in gradient descent:

- directions corresponding to large-scale features may lead to large gradients,
- small-scale features may be neglected.

Standardization (zero mean, unit variance) often improves conditioning and convergence.

13.5.2 Worked Example

Consider least squares with features of vastly different scales. The level sets of the objective become elongated ellipsoids, leading to slow convergence of gradient descent. Rescaling features makes the geometry more isotropic and improves optimization.

13.6 Statistical Stability Under Resampling

Numerical stability concerns computation. Statistical stability concerns the dependence of the learned model on the dataset.

13.6.1 Resampling Variability

If we draw two different training samples from the same distribution and train the same algorithm, we may obtain different models. The extent of this variation reflects statistical stability.

High-variance models (e.g., very flexible models) may change significantly with small data perturbations.

13.6.2 Train/Test Instability

A model that fits training data extremely well may perform inconsistently on new data if it is unstable. This is closely related to overfitting:

- unstable models adapt to noise,
- stable models capture persistent structure.

13.6.3 Robustness of Estimates

A stable estimator changes little when:

- a data point is removed,
- noise is added,
- the dataset is slightly perturbed.

Robustness is therefore a desirable property both statistically and practically.

13.7 Algorithmic Stability (Preview)

Algorithmic stability formalizes the idea that small changes in the training set should not drastically change the learned predictor.

Informally, an algorithm is stable if replacing one training example leads to only a small change in the output model or predictions.

This concept connects directly to generalization: stable algorithms tend to generalize better because they are less sensitive to sampling variability.

We will revisit this idea in more detail in later theoretical chapters.

13.8 Stability as a Precursor to Trustworthy ML

Stability is not merely a technical condition; it is essential for reliability:

- numerically unstable methods may produce arbitrary results,
- statistically unstable models may not reproduce across datasets,
- unstable systems are difficult to debug and trust.

A trustworthy machine learning system should be:

- numerically stable,
- statistically stable,
- robust to perturbations in data and computation.

Regularization, proper preprocessing, careful optimization, and sound evaluation protocols all contribute to stability.

13.9 A Unifying Perspective

Stability links numerical analysis, statistics, and machine learning. Conditioning governs how input perturbations affect solutions, floating-point arithmetic introduces unavoidable noise, and statistical variability determines how models change across datasets.

The key principle is:

$$\text{small perturbations} \Rightarrow \text{small changes in output.}$$

When this principle fails, both computation and generalization suffer.

13.10 Summary

In this chapter we examined numerical and statistical stability in learning systems. We defined conditioning and explained how ill-conditioned problems amplify perturbations. We discussed floating-point issues and their practical consequences, and showed how preprocessing and scaling affect stability.

We then turned to statistical stability, highlighting variability under resampling and its connection to overfitting and robustness. Finally, we introduced algorithmic stability as a bridge between stability and generalization. These ideas emphasize that reliable learning requires not only good models, but also stable computations and estimators.

Part I Exercises: Foundations of Learning

Conceptual Foundations

1. Explain the slogan

$$\text{learning} = \text{approximation} + \text{data} + \text{structure}.$$

   Give one concrete example illustrating each component.

2. For each of the following tasks, identify:

   (a) the input space,
   (b) the output space,
   (c) a plausible loss function,
   (d) a suitable hypothesis class.

   Tasks:

   (a) predicting house prices,
   (b) spam classification,
   (c) next-word prediction,
   (d) image classification.

3. Give an example of two distinct functions that agree on a finite training set but differ on unseen inputs. What does this show about learning from finite data?

4. Explain why unrestricted learning is impossible from finite data. State the no-free-lunch intuition in your own words.

5. What is inductive bias? Give one example from:

   (a) linear models,
   (b) kernel methods,

(c) convolutional neural networks,
(d) transformers.

6. Explain the difference between explicit and implicit inductive bias.
7. Compare invariance and equivariance. Give one machine learning example of each.
8. Why is architecture a form of inductive bias?
9. Explain the difference between supervised, unsupervised, and self-supervised learning using the function-approximation viewpoint.
10. Why is simplicity representation-dependent rather than absolute?

Probability, Risk, and Optimal Prediction

11. Let (X, Y) be a random pair and ℓ a loss function. Define:
    (a) population risk,
    (b) empirical risk,
    (c) Bayes-optimal predictor.
12. Show that for a real-valued random variable Y, the quantity
$$\mathbb{E}[(Y-a)^2]$$
is minimized at $a = \mathbb{E}[Y]$.
13. Show that
$$\mathbb{E}[(Y-a)^2 \mid X = x]$$
is minimized by
$$a = \mathbb{E}[Y \mid X = x].$$
14. Explain why a conditional median minimizes conditional expected absolute loss.
15. For binary classification under zero-one loss, show that the Bayes classifier predicts the most probable class.
16. Suppose that for a fixed input x,
$$\mathbb{P}(Y = 1 \mid X = x) = 0.7, \qquad \mathbb{P}(Y = 0 \mid X = x) = 0.3.$$
What is the Bayes classifier at x? What is the conditional Bayes error?
17. Let
$$Y = g(X) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid X] = 0.$$
Show that
$$\mathbb{E}[Y \mid X] = g(X).$$

18. Distinguish aleatoric uncertainty from epistemic uncertainty. Give one example of each.
19. Why might a probabilistic predictor be more useful than a deterministic predictor in high-stakes applications?
20. Compare the Bayesian and frequentist interpretations of uncertainty in model parameters.

Optimization and Regularization

21. State the empirical risk minimization principle.
22. Let
$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(x_i^\top \theta - y_i)^2.$$
Compute $\nabla J(\theta)$.
23. Derive the gradient descent update for least squares.
24. Explain why gradient descent can be interpreted as minimizing a local linear approximation of the objective.
25. Define convexity and strong convexity. Why are these useful in optimization?
26. Compare the constrained problem
$$\min_\theta J(\theta) \quad \text{subject to } \Omega(\theta) \le c$$
with the penalized problem
$$\min_\theta J(\theta) + \lambda\Omega(\theta).$$
27. Give one example where optimization error is large even though approximation error may be small.
28. State the penalized ERM formulation.
29. Derive the ridge regression solution
$$\hat{\theta}_{\text{ridge}} = (X^\top X + n\lambda I)^{-1}X^\top y$$
starting from the regularized least-squares objective.
30. Explain the shrinkage interpretation of ridge regression.
31. Compare ℓ_1 and ℓ_2 penalties geometrically. Why can ℓ_1 produce sparsity?
32. Explain how data augmentation acts as regularization.
33. Why can early stopping be understood as a form of regularization?
34. Give one example of implicit regularization from optimization.

Generalization, Evaluation, and Stability

35. Define the generalization gap of a predictor.
36. Explain the difference between underfitting and overfitting.
37. Describe the classical relation between model complexity, training error, and test error.
38. In polynomial regression, what happens when the degree is too small? What can happen when it is too large?
39. Explain the difference among approximation error, estimation error, and optimization error.
40. State the bias–variance tradeoff in words.
41. What is the intuition behind uniform convergence?
42. Why does concentration of empirical averages matter in learning theory?
43. Explain the difference between interpolation and extrapolation.
44. What is double descent, at a high level?
45. Why is accuracy a misleading metric in imbalanced classification?
46. Compare ROC curves with precision–recall curves. When is precision–recall typically more informative?
47. Explain the purpose of training, validation, and test splits.
48. Why does repeated reuse of the test set introduce optimistic bias?
49. Outline a correct hyperparameter tuning workflow.
50. Give two examples of data leakage.
51. What is confounding in model evaluation?
52. Why should performance estimates be reported with uncertainty rather than as exact values?
53. Define the condition number of an invertible matrix.
54. Explain why ill-conditioning can make least-squares estimation unstable.
55. What kinds of floating-point issues can affect optimization in practice?
56. Why does feature scaling improve optimization behavior?
57. Explain the difference between numerical stability and statistical stability.
58. What is algorithmic stability, informally?
59. Why is stability a prerequisite for trustworthy machine learning?

Integrated Problems

59. Consider a regression problem with a small dataset and many polynomial features. Discuss:
    (a) approximation error,
    (b) estimation error,
    (c) conditioning,
    (d) regularization,
    (e) expected generalization behavior.
60. A classifier achieves 99% training accuracy and 72% test accuracy. Give at least four plausible explanations using concepts from Part I.
61. You are given a tabular medical dataset with highly correlated features and moderate label noise. Propose:
    (a) a reasonable baseline model,
    (b) an evaluation protocol,
    (c) at least two regularization or stability measures,
    (d) one appropriate metric beyond raw accuracy.
62. Compare the following two workflows:
    (a) normalize features using the full dataset before splitting,
    (b) split first, then fit normalization on training data only.

    Explain why one is valid and the other leaks information.
63. Suppose two models have the same number of parameters, but one is much more stable under resampling. What might this suggest about effective complexity and generalization?
64. Explain how the themes of Part I connect:

    inductive bias, optimization, regularization, generalization, stability.

Chapter 14

From Classical Models to Neural Networks

14.1 Introduction

Classical supervised learning models—linear regression, logistic regression, and kernel methods—provide a powerful and mathematically elegant framework. However, their performance often depends critically on the choice of representation. When the representation is inadequate, these models may plateau in performance even as more data are provided.

This chapter motivates neural networks as a response to these representational limitations. The key idea is to move from *fixed* feature maps to *learned* feature maps. Neural networks can be viewed as systems that learn hierarchical representations of data, enabling them to capture complex nonlinear structure.

14.2 Why Classical Models Plateau

Linear models operate in a fixed feature space. Even when extended through feature maps or kernels, their effectiveness depends on the choice of representation.

Two key limitations arise:

- **Representation bottleneck:** if the feature map $\phi(x)$ does not capture relevant structure, no linear model on top can recover it.
- **Manual design burden:** constructing effective features often requires domain expertise and significant effort.

Even with sophisticated kernels, performance may plateau because:

- the representation is not adapted to the data,
- computational cost grows with data size,

- flexibility is limited by the chosen feature family.

This motivates models that *learn representations from data*.

14.3 Hand-Designed Features vs Learned Representations

In classical approaches, one first chooses a feature map $\phi(x)$ and then learns a linear model:

$$f(x) = w^\top \phi(x).$$

In neural networks, the feature map itself is parameterized and learned:

$$f(x) = w^\top \phi_\theta(x),$$

where ϕ_θ is a composition of transformations depending on parameters θ.

Thus:

- classical models: fixed ϕ, learn w,
- neural networks: learn both ϕ_θ and w.

This shift transforms learning from selecting coefficients in a fixed space to discovering useful representations automatically.

14.4 Hidden Layers as Adaptive Feature Maps

A neural network with one hidden layer has the form

$$f(x) = \sum_{j=1}^{m} w_j \sigma(a_j^\top x + b_j),$$

where σ is a nonlinear activation function.

This can be interpreted as:

- computing features $\phi_j(x) = \sigma(a_j^\top x + b_j)$,
- combining them linearly.

Thus hidden units act as learned basis functions. Unlike classical basis expansions, both the shape and location of these features are learned from data.

14.4.1 Adaptive Representation

Each hidden unit detects a pattern in the input space. The network adjusts these detectors during training so that they become useful for the task. This makes neural networks flexible and data-driven.

14.5 Depth, Hierarchy, and Composition

Deeper networks stack multiple layers:

$$x \mapsto h^{(1)} \mapsto h^{(2)} \mapsto \cdots \mapsto f(x).$$

Each layer applies a transformation of the form

$$h^{(\ell)} = \sigma(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}).$$

14.5.1 Compositional Functions

A deep network represents a composition of functions:

$$f = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}.$$

This compositional structure allows the model to build complex features from simpler ones.

14.5.2 Hierarchical Representation

Lower layers capture simple patterns, while higher layers combine them into more abstract features. This hierarchy is particularly effective in domains such as vision and language.

[Hierarchical Visual Features] In image processing:

- early layers detect edges,
- intermediate layers detect shapes,
- higher layers detect objects.

Thus depth enables structured representation learning.

14.6 Worked Examples

14.6.1 XOR Problem

Consider the XOR dataset in $\mathbb{R}^2$:

$$(0,0) \mapsto 0, \quad (1,1) \mapsto 0, \quad (1,0) \mapsto 1, \quad (0,1) \mapsto 1.$$

No linear classifier can separate these points. However, a network with a hidden layer can represent XOR by transforming the input into a space where it becomes linearly separable.

This illustrates that shallow linear models fail on simple nonlinear tasks, while even small neural networks can succeed.

14.6.2 Piecewise Nonlinear Decision Boundaries

A network with ReLU activations

$$\sigma(z) = \max(0, z)$$

produces piecewise linear functions. Even a single hidden layer can create decision boundaries composed of multiple linear segments, allowing flexible classification.

14.6.3 Feature Composition

Consider a function that depends on interactions between variables, such as

$$f(x_1, x_2, x_3) = g(x_1 + x_2, x_2 + x_3).$$

A deep network can represent this efficiently by computing intermediate sums and reusing them, illustrating the benefit of compositional structure.

14.7 The Historical Transition to Modern Deep Learning

Early neural network research recognized the importance of nonlinear representations, but training deep models was difficult due to optimization challenges and limited data.

The modern resurgence of deep learning was driven by:

- improved optimization methods (e.g., SGD variants),
- larger datasets,
- increased computational power,
- architectural innovations.

These developments made it practical to learn deep hierarchical representations at scale.

14.8 A Unifying Perspective

Neural networks extend classical models by replacing fixed feature maps with learned ones. The central shift is:

$$\text{learning a predictor} \quad \longrightarrow \quad \text{learning a representation and a predictor.}$$

Hidden layers act as adaptive feature maps, and depth enables hierarchical composition. This allows neural networks to overcome the representational limits of linear models and fixed feature expansions.

14.9 Summary

In this chapter we motivated neural networks from the limitations of classical models. We showed that linear models depend critically on feature design and may fail on simple nonlinear tasks such as XOR. Neural networks address this by learning feature maps through hidden layers.

We interpreted hidden units as adaptive basis functions and explained how depth enables compositional and hierarchical representations. These ideas form the conceptual foundation for the deep learning models studied in the following chapters.

Chapter 15

Neural Networks as Compositional Models

15.1 Introduction

A neural network is not merely a nonlinear predictor. Its defining structural idea is *composition*. Instead of mapping an input directly to an output in one step, a network applies a sequence of affine maps and nonlinearities, building intermediate representations along the way. This compositional structure is one of the main reasons neural networks can represent complicated functions efficiently.

This chapter develops feedforward neural networks as compositional models. We define the basic architecture, discuss expressivity, compare depth and width, and state the universal approximation theorem in its proper limited role. We also emphasize an important distinction: a class may be highly expressive without being easy to learn from data.

15.2 Feedforward Networks as Compositions of Maps

A feedforward neural network is built by alternating affine maps and nonlinear activation functions.

[Feedforward Neural Network] A feedforward neural network of depth L is a function of the form

$$f(x) = W^{(L)} h^{(L-1)} + b^{(L)},$$

where

$$h^{(0)} = x, \qquad h^{(\ell)} = \sigma\big(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\big), \quad \ell = 1, \dots, L-1.$$

Here each $W^{(\ell)}$ is a weight matrix, each $b^{(\ell)}$ is a bias vector, and σ is an activation function applied componentwise.

Thus the network is a composition

$$f = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)},$$

with each hidden layer performing an affine transformation followed by a nonlinearity.

15.2.1 Why Nonlinearity Matters

If σ were the identity, then the composition of affine maps would still be affine. So depth alone would add no expressive power. The activation function is what allows the network to represent nonlinear structure.

Common choices include:

$$\sigma(z) = \max(0, z) \quad \text{(ReLU)}, \qquad \sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{(sigmoid)}.$$

15.3 Width, Depth, and Expressive Power

Two basic architectural quantities are:

- **width:** the number of units in a layer,
- **depth:** the number of successive layers.

Both affect expressivity, but in different ways.

15.3.1 Width as Parallel Representation

A wider layer can represent more features at the same stage of computation. For example, a one-hidden-layer network

$$f(x) = \sum_{j=1}^{m} a_j \, \sigma(w_j^\top x + b_j)$$

builds a linear combination of m nonlinear units. Increasing m gives more basis-like components.

15.3.2 Depth as Sequential Composition

Greater depth allows the network to reuse intermediate computations. Instead of representing a complicated function in one step, the model can build it hierarchically. This can be much more efficient when the target function itself has compositional structure.

15.3.3 Piecewise Linear Region Intuition

For ReLU networks, the represented function is piecewise linear. Each hidden unit introduces a threshold

$$w^\top x + b = 0$$

that divides the input space into regions. As layers are stacked, these regions are repeatedly transformed and recombined. Deeper networks can therefore create highly intricate piecewise linear structures with relatively few parameters.

The key intuition is that depth does not just add more units; it allows repeated partitioning and recombination of the input space.

15.4 Universal Approximation: What It Says and What It Does Not Say

One of the most famous theoretical results about neural networks is the universal approximation theorem.

[Universal Approximation, Informal Statement] Let $K \subset \mathbb{R}^d$ be compact, and let σ be a suitable non-polynomial activation function. Then for every continuous function $f : K \to \mathbb{R}$ and every $\varepsilon > 0$, there exists a neural network with a single hidden layer such that

$$\sup_{x \in K} |f(x) - f_\theta(x)| < \varepsilon.$$

This theorem is important because it shows that shallow neural networks are, in principle, extremely expressive.

15.4.1 What the Theorem Says

It says that one hidden layer can approximate any continuous function on a compact domain arbitrarily well, provided the hidden layer is allowed to be sufficiently wide.

15.4.2 What the Theorem Does Not Say

It does *not* say:

- that approximation is parameter-efficient,
- that the required width is reasonable,
- that the network can be learned efficiently from data,
- that generalization will be good,
- that depth is unimportant.

Thus universal approximation is a foundational theorem, but not a full explanation of why deep learning works well in practice.

15.5 Hierarchical Structure and Reuse

Many real-world functions are naturally compositional. A target function may be built from simpler subfunctions that are reused.

For example, a function of four variables may have the form

$$f(x_1, x_2, x_3, x_4) = g\big(h_1(x_1, x_2),\, h_2(x_3, x_4)\big).$$

A deep architecture can represent this naturally:

- first compute h_1 and h_2,
- then combine them through g.

A shallow model can also represent such a function in principle, but often less efficiently.

15.5.1 Hierarchical Feature Construction

This reuse principle appears across domains:

- in vision: edges $\to$ motifs $\to$ parts $\to$ objects,
- in language: tokens $\to$ phrases $\to$ larger semantic structure,
- in control and reasoning: small subroutines composed into larger computations.

This is one of the main motivations for deep models: they align with hierarchical structure in the world.

15.6 Depth Efficiency

A major modern insight is that depth may give not only expressive power, but *expressive efficiency*. Some functions can be represented compactly by deep networks but require much larger shallow networks.

[Depth Efficiency, Informal] There exist function families that can be represented by deep networks with polynomial size, but for which shallow networks require exponentially larger width to achieve comparable approximation.

This statement should be understood as a qualitative theorem pattern rather than a single universal theorem. Its meaning is that depth can matter not only quantitatively, but qualitatively.

The idea is that compositional structure is efficiently matched by compositional architectures. A shallow network may flatten the whole computation into one stage, losing the advantage of reuse.

15.7 Expressivity vs Learnability

A model class may be highly expressive without being easy to train or statistically efficient.

15.7.1 Expressivity

Expressivity concerns what functions can be represented at all, or approximated well, by the architecture.

15.7.2 Learnability

Learnability concerns whether a good representation can be found from finite data and practical optimization. These are different questions.

For example:

- a network class may contain an excellent predictor,
- but optimization may not find it,
- or finite data may not be enough to identify it reliably.

This distinction is crucial. Universal approximation and depth-efficiency results concern expressivity, not necessarily trainability or generalization.

15.7.3 Why This Matters

The success of deep learning cannot be explained by expressivity alone. It also depends on:

- optimization dynamics,
- regularization,
- architecture-task alignment,
- data scale.

So one should not confuse "can represent" with "can learn well."

15.8 A Unifying Perspective

Neural networks are compositional models built from affine maps and nonlinearities. Their power comes not merely from nonlinearity, but from repeated nonlinear composition. Width allows many features in parallel; depth allows hierarchical reuse and structured computation.

The universal approximation theorem shows that even shallow networks are expressive in principle. But the deeper lesson is that depth can make representation more efficient when the target function itself is compositional. This helps explain why deep architectures often outperform shallow alternatives on complex tasks.

15.9 Summary

In this chapter we defined feedforward neural networks as compositions of affine maps and nonlinear activation functions. We discussed how width and depth contribute differently to expressivity, and gave piecewise linear intuition for ReLU networks. We stated the universal approximation theorem and emphasized its limitations: representability does not imply efficiency, learnability, or good generalization.

We then explained why hierarchical structure and reuse favor deep models, and introduced the idea of depth efficiency. This chapter provides the theoretical motivation for studying training dynamics, backpropagation, and architectural design in the chapters that follow.

Chapter 16

Backpropagation and Automatic Differentiation

16.1 Introduction

Training neural networks requires computing gradients of a loss with respect to many parameters. Direct symbolic differentiation is impractical, and naive application of the chain rule can be computationally expensive. Backpropagation provides an efficient and systematic way to compute these gradients.

This chapter develops backpropagation as an instance of *reverse-mode automatic differentiation* on computation graphs. We interpret the chain rule as message passing, derive the reverse recursion, and apply it to multilayer perceptrons. We also discuss parameter sharing and memory–computation tradeoffs.

16.2 Differentiation on Computation Graphs

[Computation Graph] A computation graph is a directed acyclic graph in which:

- nodes represent intermediate variables,
- edges represent functional dependencies,
- each node computes a function of its parents.

Let the graph define a scalar output L (the loss) as a function of inputs and parameters. Each node v has a value z_v computed from its parents.

Differentiation means computing $\frac{\partial L}{\partial z_v}$ for all nodes v.

16.3 The Chain Rule as Message Passing

If a node v influences the loss through multiple paths, then by the chain rule,

$$\frac{\partial L}{\partial z_v} = \sum_{u:\, v \to u} \frac{\partial L}{\partial z_u} \frac{\partial z_u}{\partial z_v}.$$

This expresses a recursive dependency: gradients at a node are obtained by aggregating contributions from its children.

Thus differentiation can be viewed as *message passing*:

- each node receives gradient messages from its outputs,
- it combines them using local derivatives,
- and passes messages backward to its inputs.

16.4 Reverse-Mode Automatic Differentiation

Reverse-mode automatic differentiation computes all gradients $\frac{\partial L}{\partial z_v}$ in a single backward pass.

16.4.1 Reverse Recursion

Let $\bar{z}_v = \frac{\partial L}{\partial z_v}$ denote the adjoint variable. Then:

$$\bar{z}_v = \sum_{u:\, v \to u} \bar{z}_u \frac{\partial z_u}{\partial z_v}.$$

The algorithm proceeds as follows:

1. **Forward pass:** compute all node values z_v.
2. **Backward pass:** initialize $\bar{z}_L = 1$ and propagate gradients backward using the recursion.

16.4.2 Complexity

- Forward mode computes derivatives with cost proportional to input dimension.
- Reverse mode computes derivatives with cost proportional to output dimension.

Since neural networks typically have a scalar output (loss), reverse mode is far more efficient.

[Efficiency of Reverse Mode] For a scalar output L, reverse-mode automatic differentiation computes all partial derivatives $\frac{\partial L}{\partial \theta_j}$ at a cost comparable to a small constant multiple of the forward computation.

16.5 Backpropagation in Multilayer Perceptrons

Consider a network

$$h^{(1)} = \sigma(W^{(1)}x + b^{(1)}), \qquad h^{(2)} = \sigma(W^{(2)}h^{(1)} + b^{(2)}),$$

$$\hat{y} = W^{(3)}h^{(2)} + b^{(3)}, \qquad L = \ell(\hat{y}, y).$$

We derive gradients using reverse-mode differentiation.

16.5.1 Backward Pass

Define error signals:

$$\delta^{(3)} = \frac{\partial L}{\partial \hat{y}},$$

$$\delta^{(2)} = (W^{(3)})^\top \delta^{(3)} \odot \sigma'(z^{(2)}),$$

$$\delta^{(1)} = (W^{(2)})^\top \delta^{(2)} \odot \sigma'(z^{(1)}),$$

where $z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}$ and $\odot$ denotes elementwise multiplication.

16.5.2 Gradients

The gradients are:

$$\frac{\partial L}{\partial W^{(3)}} = \delta^{(3)}(h^{(2)})^\top, \qquad \frac{\partial L}{\partial b^{(3)}} = \delta^{(3)},$$

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)}(h^{(1)})^\top, \qquad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)},$$

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)}x^\top, \qquad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

This shows a general pattern:

$$\text{gradient} = \text{error signal} \times \text{input to layer}.$$

16.5.3 Matrix Form Insight

Backpropagation propagates gradients through matrix multiplications using transposes:

$$\delta^{(\ell)} = (W^{(\ell+1)})^\top \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)}).$$

Thus each layer receives a transformed version of the error from the layer above.

16.6 Parameter Sharing

In many models, parameters are reused across multiple parts of the computation graph (e.g., convolutional filters, recurrent networks).

If a parameter θ appears in multiple nodes, then:

$$\frac{\partial L}{\partial \theta} = \sum_k \frac{\partial L}{\partial z_k} \frac{\partial z_k}{\partial \theta}.$$

Thus gradients accumulate over all occurrences. This is a direct consequence of the chain rule and is naturally handled by reverse-mode differentiation.

16.7 Memory and Computation Tradeoffs

Reverse-mode differentiation requires storing intermediate values from the forward pass in order to compute derivatives in the backward pass.

16.7.1 Tradeoff

- storing all intermediate activations uses more memory but speeds up backpropagation,
- recomputing activations reduces memory but increases computation.

This tradeoff becomes important in large models.

16.7.2 Checkpointing

A common strategy is to store only some intermediate values and recompute others when needed. This balances memory and computation.

16.8 Worked Derivation: Two-Layer Network

Consider

$$h = \sigma(Wx + b), \qquad \hat{y} = v^\top h, \qquad L = \ell(\hat{y}, y).$$

Then:

$$\frac{\partial L}{\partial v} = \frac{\partial L}{\partial \hat{y}}\, h,$$

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial \hat{y}}\, v,$$

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial h} \odot \sigma'(Wx + b) \right) x^\top.$$

This illustrates the layered structure of gradient computation.

16.9 Common Gradient Pitfalls

- forgetting to apply the chain rule through all paths,
- missing elementwise products with activation derivatives,
- incorrect matrix dimensions,
- failing to sum gradients for shared parameters.

Backpropagation avoids these issues by systematically applying the reverse recursion.

16.10 A Unifying Perspective

Backpropagation is not a special trick, but a structured application of the chain rule on computation graphs. Reverse-mode automatic differentiation makes gradient computation efficient by reusing intermediate results and propagating information backward.

The key ideas are:

- represent computation as a graph,
- apply the chain rule locally,
- propagate gradients backward,
- reuse structure for efficiency.

These principles extend beyond neural networks to any differentiable program.

16.11 Summary

In this chapter we developed backpropagation as reverse-mode automatic differentiation on computation graphs. We interpreted the chain rule as message passing, derived the reverse recursion, and applied it to multilayer perceptrons. We showed how gradients take a structured matrix form and how parameter sharing leads to gradient accumulation.

We also discussed computational efficiency and memory tradeoffs. Backpropagation is the central algorithm enabling the training of modern neural networks, and its principles apply broadly to differentiable computation.

Chapter 17

Training Deep Networks

17.1 Introduction

Deep neural networks are expressive, but they are also difficult to train. Unlike convex models such as logistic regression, deep networks involve highly nonconvex objectives, long chains of transformations, and delicate interactions between initialization, optimization, and architecture.

This chapter examines why training deep networks is challenging and how these challenges are addressed in practice. We focus on signal propagation, gradient behavior, and initialization, and develop intuition through simple mathematical arguments.

17.2 Why Deep Networks Are Hard to Train

A deep network is a composition of many layers:

$$x \mapsto h^{(1)} \mapsto h^{(2)} \mapsto \cdots \mapsto h^{(L)}.$$

This creates several difficulties:

- gradients must propagate through many transformations,
- small numerical effects can accumulate,
- optimization landscapes are complex and high-dimensional,
- different layers may learn at very different rates.

Even if a good solution exists, it may be difficult to reach with standard optimization methods.

17.3 Initialization and Signal Propagation

Initialization determines the starting point of optimization and strongly affects whether signals propagate stably through the network.

17.3.1 Forward Signal Propagation

Consider a simplified model:

$$h^{(\ell)} = \sigma(W^{(\ell)} h^{(\ell-1)}),$$

with random initialization of weights.

We ask: how does the variance of activations evolve with depth?

17.3.2 Variance Propagation Intuition

Assume:

- entries of $W^{(\ell)}$ are i.i.d. with variance σ_w^2,
- inputs have zero mean and variance $q^{(\ell-1)}$.

Then approximately:

$$q^{(\ell)} \approx \sigma_w^2 \, \mathbb{E}[\sigma(z)^2],$$

where z is a random variable with variance $q^{(\ell-1)}$.

If σ_w^2 is too large, activations explode. If σ_w^2 is too small, activations shrink toward zero.

Thus stable propagation requires careful scaling of weights.

17.3.3 Initialization Schemes

This leads to practical initialization rules:

- **Xavier initialization:** variance scaled by input dimension,
- **He initialization:** adjusted for ReLU activations.

These aim to keep activations at a consistent scale across layers.

17.4 Vanishing and Exploding Gradients

The backward pass involves repeated multiplication by Jacobians:

$$\delta^{(\ell)} = (W^{(\ell+1)})^\top \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)}).$$

17.4.1 Jacobian Product Intuition

Gradients involve products of many terms:

$$\prod_{\ell} W^{(\ell)} \quad \text{and} \quad \prod_{\ell} \sigma'(z^{(\ell)}).$$

If these factors have norms less than 1, gradients shrink exponentially with depth (vanishing gradients). If greater than 1, gradients grow exponentially (exploding gradients).

17.4.2 Consequences

- **Vanishing gradients:** early layers learn very slowly.
- **Exploding gradients:** updates become unstable.

Both phenomena make deep networks difficult to train.

17.5 Gradient Flow Through Depth

Training requires gradients to flow from the loss back to early layers.

17.5.1 Deep Linear Case

Even in a linear network

$$f(x) = W^{(L)} \cdots W^{(1)} x,$$

gradients involve products of matrices. If these matrices are poorly conditioned, gradient flow deteriorates.

17.5.2 Nonlinear Case

Nonlinearities such as sigmoid can saturate:

$$\sigma'(z) \approx 0 \quad \text{for large } |z|.$$

This further amplifies vanishing gradients.

ReLU mitigates this partially, since

$$\sigma'(z) = 1 \text{ for } z > 0,$$

but introduces other issues such as inactive units.

17.6 Loss Landscapes

The objective function for deep networks is highly nonconvex.

17.6.1 Key Features

- many local minima and saddle points,
- flat regions and sharp regions,
- symmetries due to parameterization.

Despite this complexity, gradient-based methods often find good solutions in practice.

17.6.2 Practical Observation

Many local minima are comparable in value, and saddle points appear to be more problematic than local minima. Noise in stochastic optimization can help escape saddle regions.

17.7 Practical Training Recipe

Successful training of deep networks relies on a combination of techniques:

- **Initialization:** use variance-preserving schemes,
- **Activation choice:** prefer non-saturating functions (e.g., ReLU),
- **Normalization:** stabilize intermediate activations,
- **Optimization:** use SGD with momentum or adaptive methods,
- **Learning rate tuning:** adjust schedules carefully.

These techniques work together to maintain stable signal and gradient propagation.

17.8 Worked Example: Deep Linear Network

Consider a deep linear network:

$$f(x) = W^{(L)} \cdots W^{(1)} x.$$

If each matrix has singular values slightly less than 1, then

$$\|W^{(L)} \cdots W^{(1)}\| \approx \prod_{\ell=1}^{L} \|W^{(\ell)}\|,$$

which decays exponentially with L. Thus gradients vanish.

If singular values exceed 1, the product grows exponentially, leading to exploding gradients.

This simple example illustrates why depth alone creates instability.

17.9 A Unifying Perspective

Training deep networks requires maintaining stable signal propagation both forward and backward. Initialization controls the starting scale, nonlinearities affect gradient flow, and optimization interacts with the landscape.

The central challenge is:

> propagate information across many layers without distortion.

Modern techniques can be understood as ways of controlling variance, conditioning, and gradient flow across depth.

17.10 Summary

In this chapter we examined why deep networks are difficult to train. We analyzed signal propagation, showing how poor initialization leads to exploding or vanishing activations. We explained the role of Jacobian products in gradient instability and discussed how nonlinearities influence gradient flow.

We also described the structure of loss landscapes and outlined practical strategies for stable training. These insights form the foundation for understanding architectural improvements such as normalization and residual connections.

Chapter 18

Activation Functions and Architectures

18.1 Introduction

Nonlinearity is the key ingredient that gives neural networks their expressive power. Without nonlinear activation functions, a deep network would collapse into a single affine map. However, not all nonlinearities behave the same. Their shape, smoothness, and saturation properties strongly influence optimization, stability, and generalization.

This chapter goes beyond listing activation functions. We analyze their mathematical properties and connect them to architectural design and training behavior. We also discuss common architectural motifs and how activation choices interact with them.

18.2 Nonlinearity as the Source of Expressive Power

A feedforward network alternates affine maps and nonlinearities:

$$h^{(\ell)} = \sigma(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}).$$

If σ were linear, then the composition would remain linear. Thus all expressive power comes from the activation function.

Nonlinearities enable:

- representation of nonlinear decision boundaries,
- hierarchical feature construction,
- partitioning of the input space into regions.

The choice of activation determines how these effects occur.

18.3 Saturating vs Non-Saturating Activations

18.3.1 Sigmoid and Tanh

The sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

and the hyperbolic tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

are smooth and bounded.

However, they are *saturating*: for large $|z|$, their derivatives approach zero:

$$\sigma'(z) \to 0, \qquad \tanh'(z) \to 0.$$

18.3.2 Implications of Saturation

Saturation causes:

- vanishing gradients,
- slow learning in deep networks,
- sensitivity to initialization.

Thus while sigmoid and tanh are theoretically appealing, they can be difficult to train in deep architectures.

18.3.3 ReLU and Non-Saturating Behavior

The rectified linear unit (ReLU) is

$$\sigma(z) = \max(0, z).$$

It is non-saturating for $z > 0$, since

$$\sigma'(z) = 1 \quad \text{for } z > 0.$$

This allows gradients to propagate more effectively, which is one reason for its widespread use.

However, ReLU introduces:

- non-differentiability at $z = 0$,
- inactive units when $z < 0$ ("dead neurons").

18.4 Piecewise Linear vs Smooth Activations

Activation functions can be grouped by smoothness.

18.4.1 Piecewise Linear Activations

ReLU and its variants (e.g., leaky ReLU) produce piecewise linear functions. Networks built from them are piecewise linear in the input.

Advantages:

- simple gradients,
- efficient computation,
- stable optimization.

18.4.2 Smooth Activations

Smooth activations include sigmoid, tanh, and GELU:

$$\mathrm{GELU}(z) = z \cdot \Phi(z),$$

where Φ is the Gaussian cumulative distribution function.

Smooth activations:

- provide continuous derivatives,
- may better model gradual transitions,
- can improve optimization in some architectures (e.g., transformers).

18.4.3 Tradeoffs

- piecewise linear: simpler and often faster,
- smooth: potentially better gradient flow and expressivity in certain settings.

The choice depends on the architecture and task.

18.5 Inductive Tradeoffs of Activation Functions

Activation functions impose implicit biases on the function class.

- ReLU favors piecewise linear functions,
- sigmoid/tanh favor smooth bounded outputs,
- GELU introduces smooth gating behavior.

These choices affect:

- optimization dynamics,
- gradient stability,
- representational properties.

Thus activation functions are not neutral; they shape the hypothesis space.

18.6 Architectural Motifs

Activation functions interact with network architecture. Several common motifs illustrate this.

18.6.1 Multilayer Perceptron (MLP)

An MLP is a stack of fully connected layers with nonlinearities. Its behavior depends strongly on activation choice:

- ReLU enables deep MLPs to train effectively,
- sigmoid/tanh may lead to vanishing gradients.

18.6.2 Bottleneck Architectures

A bottleneck layer has lower dimensionality than surrounding layers:

$$\mathbb{R}^d \to \mathbb{R}^k \to \mathbb{R}^d, \quad k < d.$$

This forces the network to compress information, encouraging structured representations. Nonlinearities determine how information is compressed and reconstructed.

18.6.3 Encoder–Decoder Architectures

These architectures map input to a latent representation and back:

$$x \to z \to \hat{x}.$$

They are widely used in autoencoders, sequence models, and generative systems. Activation functions influence both encoding (feature extraction) and decoding (reconstruction quality).

18.7 Choosing Activations in Practice

In practice, the choice of activation depends on both optimization and modeling considerations.

18.7.1 Common Guidelines

- use ReLU or variants for general-purpose deep networks,
- use GELU or similar smooth activations in transformer architectures,
- avoid sigmoid in deep hidden layers unless specifically needed,
- match activation to output type (e.g., sigmoid for probabilities).

18.7.2 Interaction with Initialization and Normalization

Activation choice interacts with:

- initialization schemes,
- normalization layers,
- learning rates.

A good configuration balances signal propagation, gradient stability, and expressivity.

18.8 Worked Comparisons

18.8.1 Derivative Comparison

- sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, small for large $|z|$,
- tanh: $\tanh'(z) = 1 - \tanh^2(z)$, also saturating,
- ReLU: $\sigma'(z) = 1$ for $z > 0$, 0 otherwise,
- GELU: smooth, nonzero derivative over a wide range.

This explains why ReLU often avoids vanishing gradients compared to sigmoid.

18.8.2 Optimization Consequences

- saturating activations slow learning in deep networks,
- non-saturating activations improve gradient flow,
- smooth activations may stabilize training in certain architectures.

Thus activation choice directly affects optimization behavior.

18.9 A Unifying Perspective

Activation functions determine how linear transformations are combined into nonlinear models. Their properties—saturation, smoothness, and shape—affect both expressivity and trainability.

Architectures organize these nonlinearities into structured computations. Together, activation functions and architecture define the effective hypothesis class and its optimization properties.

18.10 Summary

In this chapter we examined activation functions as the source of nonlinearity in neural networks. We compared saturating and non-saturating activations, discussed piecewise linear versus smooth functions, and analyzed their impact on gradient flow and optimization.

We also introduced common architectural motifs and explained how activation choices interact with them. These insights are essential for designing and training effective deep learning systems.

Chapter 19

Normalization Methods

19.1 Introduction

Normalization layers are a central component of modern deep networks. They improve training stability, accelerate optimization, and enable deeper architectures. However, their role is often described informally (e.g., "reducing internal covariate shift"), which can obscure their true effect.

This chapter develops normalization methods from both mathematical and practical perspectives. We study batch normalization, layer normalization, and related variants, and interpret normalization as a form of reparameterization that improves conditioning and gradient flow.

19.2 Why Normalization Helps

Deep networks suffer from unstable signal propagation and poor conditioning. Activations can grow or shrink across layers, and gradients may become ill-scaled.

Normalization addresses these issues by:

- controlling the scale of activations,
- improving gradient conditioning,
- making optimization less sensitive to parameter initialization,
- introducing a mild regularization effect.

Rather than viewing normalization as fixing data distributions, it is more accurate to view it as improving the geometry of the optimization problem.

19.3 Batch Normalization

Batch normalization (BN) normalizes activations across a mini-batch.

19.3.1 Transformation

Given activations $x_1, \ldots, x_m$ in a batch, define:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized activations are:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}.$$

A learned affine transformation is then applied:

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are trainable parameters.

19.3.2 Interpretation

BN standardizes activations within each batch, then allows the network to recover any necessary scale and shift through γ and β.

This can be interpreted as:

- enforcing a normalized intermediate representation,
- while preserving expressive flexibility.

19.3.3 Train/Test Discrepancy

During training, BN uses batch statistics (μ_B, σ_B^2). At test time, it uses running averages:

$$\mu_{\text{test}}, \quad \sigma_{\text{test}}^2.$$

- this introduces a discrepancy between training and inference,
- small batch sizes can lead to noisy estimates.

Thus BN depends on batch size and training dynamics.

19.3.4 Internal Covariate Shift (Nuanced View)

The original motivation for BN was to reduce "internal covariate shift," meaning changes in layer input distributions during training.

However, modern understanding suggests:

- the main benefit is improved optimization geometry,
- BN smooths the loss landscape and stabilizes gradients,
- it enables larger learning rates.

Thus its effectiveness is better explained by conditioning rather than distributional stability alone.

19.4 Layer Normalization and Sequence Models

Batch normalization depends on batch statistics, which can be problematic for:

- small batch sizes,
- sequential or autoregressive models.

19.4.1 Layer Normalization (LN)

Layer normalization normalizes across features for a single example:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \qquad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2.$$

Then:

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \qquad y_j = \gamma \hat{x}_j + \beta.$$

19.4.2 Properties

- independent of batch size,
- consistent between training and testing,
- widely used in transformers and sequence models.

19.4.3 Instance and Group Normalization

Other variants include:

- **Instance normalization:** normalize each channel independently per example,
- **Group normalization:** normalize over groups of channels.

These provide flexibility between BN and LN, especially in vision tasks.

19.5 Normalization as Geometry and Conditioning

Normalization can be understood as a reparameterization of the model.

19.5.1 Reparameterization View

Consider a layer output:

$$z = Wx.$$

With normalization:

$$\hat{z} = \frac{z - \mu}{\sigma}.$$

This effectively rescales directions in parameter space, making optimization less sensitive to parameter magnitudes.

19.5.2 Conditioning Intuition

Poorly scaled features lead to ill-conditioned optimization problems. Normalization:

- makes level sets more isotropic,
- improves gradient alignment,
- allows larger and more stable updates.

Thus normalization acts as a preconditioner embedded within the network.

19.6 When Normalization Fails or Is Unnecessary

Normalization is not universally beneficial.

19.6.1 Failure Modes

- very small batch sizes degrade BN performance,
- mismatch between training and inference statistics,
- added computational overhead.

19.6.2 Alternatives

Some architectures reduce reliance on normalization by:

- careful initialization,
- residual connections,
- architectural constraints.

In such cases, normalization may be less critical.

19.7 Worked Comparison: BN vs LN

- BN normalizes across the batch:
  - depends on batch statistics,
  - effective in large-batch vision models.
- LN normalizes across features:
  - independent of batch size,
  - preferred in sequence and transformer models.

Thus the choice depends on data structure and training setup.

19.8 A Unifying Perspective

Normalization layers control scale and improve conditioning within deep networks. Rather than merely stabilizing distributions, they reshape the optimization landscape through reparameterization.

Different normalization methods correspond to different choices of:

- normalization axis (batch, feature, channel),
- statistical estimation (batch vs per-sample),
- invariance properties.

Understanding these choices clarifies when each method is appropriate.

19.9 Summary

In this chapter we studied normalization methods in deep learning. We defined batch normalization and analyzed its transformation, training/inference behavior, and practical effects. We introduced layer normalization and related variants, highlighting their role in sequence models.

We interpreted normalization as a geometric reparameterization that improves conditioning and gradient flow. Finally, we discussed when normalization may fail or be unnecessary. These ideas are essential for designing stable and efficient deep architectures.

Chapter 20

Regularization in Deep Learning

20.1 Introduction

Deep neural networks are highly expressive and can easily overfit finite datasets. Regularization is therefore essential for achieving good generalization. Unlike classical models, where regularization is often explicit and well-defined, deep learning combines both *explicit* and *implicit* forms of regularization.

This chapter surveys modern regularization techniques, including dropout, weight decay, data augmentation, label smoothing, and early stopping. We also discuss implicit bias from optimization and heuristic ideas such as flatness and sharpness of minima.

20.2 Explicit vs Implicit Regularization

20.2.1 Explicit Regularization

Explicit regularization modifies the objective or training procedure directly:

- adding penalty terms (e.g., weight decay),
- injecting noise (e.g., dropout),
- augmenting data,
- smoothing targets.

These are designed interventions to control model complexity.

20.2.2 Implicit Regularization

Implicit regularization arises from the interaction of:

- optimization algorithms (e.g., SGD),
- initialization,
- architecture.

Even without explicit penalties, these factors bias the learned solution toward certain structures.

20.3 Dropout and Stochastic Subnetworks

Dropout randomly sets a subset of activations to zero during training.

20.3.1 Mechanism

For each unit, independently:

$$h_j \leftarrow m_j h_j, \quad m_j \sim \text{Bernoulli}(p).$$

At test time, activations are scaled to account for this randomness.

20.3.2 Interpretation

Dropout can be viewed as:

- training an ensemble of subnetworks,
- averaging their predictions at test time,
- preventing co-adaptation of features.

20.3.3 Effect

- reduces overfitting,
- encourages redundancy and robustness,
- introduces noise during training.

20.4 Weight Decay and Norm Control

Weight decay adds an ℓ_2 penalty to the objective:

$$J(\theta) + \lambda \|\theta\|^2.$$

20.4.1 Interpretation

- encourages small parameter norms,
- reduces model complexity,
- improves conditioning of optimization.

20.4.2 Connection to Optimization

In gradient descent, weight decay corresponds to shrinking parameters at each step:

$$\theta \leftarrow (1 - \eta\lambda)\theta - \eta\nabla J(\theta).$$

Thus it acts both as a regularizer and as a stabilizing force.

20.5 Data Augmentation and Invariance

Data augmentation expands the training set by applying transformations to inputs:

$$x \mapsto T(x),$$

where T preserves the label.

20.5.1 Examples

- image flips, rotations, crops,
- noise injection,
- time shifts in sequences.

20.5.2 Interpretation

Data augmentation encodes prior knowledge:

- invariance to transformations,
- robustness to perturbations.

Thus it regularizes by constraining the hypothesis space.

20.6 Label Smoothing

Label smoothing replaces hard targets with softened distributions.

20.6.1 Definition

Instead of a one-hot target y, use:

$$\tilde{y} = (1 - \alpha)y + \alpha u,$$

where u is a uniform distribution.

20.6.2 Effect

- discourages overconfident predictions,
- improves calibration,
- acts as a form of regularization.

20.7 Early Stopping

Early stopping halts training before full convergence.

20.7.1 Mechanism

Monitor validation error and stop when it begins to increase.

20.7.2 Interpretation

- prevents overfitting to training data,
- limits effective model complexity,
- acts as implicit regularization.

20.8 Implicit Bias of Optimization

Optimization algorithms introduce biases even without explicit penalties.

20.8.1 Example: Gradient Descent

In linear models, gradient descent often converges to minimum-norm solutions among all interpolating solutions.

20.8.2 In Deep Networks

The implicit bias is more complex but may favor:

- smoother functions,
- lower-complexity representations,

- structured solutions aligned with the architecture.

Understanding implicit bias remains an active area of research.

20.9 Sharpness, Flatness, and Generalization Heuristics

A common heuristic relates generalization to the geometry of minima.

20.9.1 Sharp vs Flat Minima

- **sharp minimum:** loss increases rapidly near the solution,
- **flat minimum:** loss remains stable under perturbations.

Flat minima are often associated with better generalization.

20.9.2 Interpretation

Flatness can be viewed as robustness to parameter perturbations. This connects to stability and regularization.

However, this is a heuristic rather than a complete theory.

20.10 Combining Regularization Techniques

In practice, multiple regularization methods are used together:

- weight decay + data augmentation,
- dropout + early stopping,
- label smoothing + normalization.

These methods interact and should be tuned jointly.

20.11 A Unifying Perspective

Regularization in deep learning controls complexity through multiple mechanisms:

- explicit penalties constrain parameters,
- stochastic methods introduce noise,
- data augmentation encodes invariances,
- optimization induces implicit biases.

Together, these shape the learned model and its generalization behavior.

20.12 Summary

In this chapter we surveyed modern regularization techniques in deep learning. We distinguished explicit and implicit regularization, and studied dropout, weight decay, data augmentation, label smoothing, and early stopping.

We also discussed implicit bias from optimization and heuristic connections between flatness and generalization. These ideas highlight that generalization in deep learning is controlled by a combination of architectural, algorithmic, and data-driven factors.

Chapter 21

Residual Networks and Skip Connections

21.1 Introduction

Increasing depth improves expressivity, but early attempts at very deep networks encountered a surprising phenomenon: adding more layers sometimes *degraded* performance. This was not merely overfitting, but an optimization difficulty.

Residual networks (ResNets) address this issue by introducing skip connections that allow information and gradients to flow more directly through the network. This chapter develops both the intuition and mathematical framing of residual connections, including their interpretation as perturbations of the identity and as discrete dynamical systems.

21.2 Why Deeper Networks Once Failed

Empirically, deeper plain networks (without skip connections) often exhibit:

- higher training error than shallower networks,
- difficulty in optimization,
- unstable gradient propagation.

This is known as the *degradation problem*. Even though a deeper network can represent the same function as a shallower one (by setting additional layers to identity), standard optimization struggles to find such solutions.

21.3 Skip Connections as Identity Highways

The key idea of residual networks is to reformulate layers as deviations from the identity.

21.3.1 Residual Update Form

Instead of learning

$$h^{(\ell+1)} = F^{(\ell)}(h^{(\ell)}),$$

we learn

$$h^{(\ell+1)} = h^{(\ell)} + F^{(\ell)}(h^{(\ell)}).$$

- the term $h^{(\ell)}$ is a *skip connection*,
- $F^{(\ell)}$ is a residual function.

21.3.2 Interpretation

Each layer modifies its input slightly rather than replacing it entirely. If $F^{(\ell)} \approx 0$, the layer behaves like the identity.

Thus skip connections create *identity highways* that allow signals to propagate across many layers without degradation.

21.4 Residual Blocks and Feature Refinement

A residual block typically consists of a few layers (e.g., linear + activation) forming $F^{(\ell)}$, combined with a skip connection.

21.4.1 Feature Refinement View

Residual networks can be interpreted as performing iterative refinement:

$$h^{(\ell+1)} = h^{(\ell)} + \text{correction}.$$

Each block refines the representation slightly rather than reconstructing it from scratch.

This leads to:

- smoother optimization,
- more stable representations,
- easier training of very deep networks.

21.5 Gradient Transport Through Skip Connections

Consider the backward pass:

$$\frac{\partial L}{\partial h^{(\ell)}} = \frac{\partial L}{\partial h^{(\ell+1)}} \left(I + \frac{\partial F^{(\ell)}}{\partial h^{(\ell)}} \right).$$

The identity term ensures that gradients can flow directly:

$$\frac{\partial L}{\partial h^{(\ell)}} \approx \frac{\partial L}{\partial h^{(\ell+1)}}$$

when $F^{(\ell)}$ is small.

21.5.1 Implication

- gradients are less likely to vanish,
- optimization becomes more stable,
- deep networks become trainable.

Thus skip connections improve gradient transport across depth.

21.6 Stability as Perturbation of Identity

Residual networks can be viewed as small perturbations of the identity map.

21.6.1 Stability Intuition

If each block satisfies

$$\|F^{(\ell)}(h)\| \ll \|h\|,$$

then the overall transformation remains close to identity.

This prevents:

- explosive growth of activations,
- collapse of representations,
- instability across layers.

Thus residual connections stabilize both forward and backward propagation.

21.7 Optimization and Dynamical Systems Perspective

Residual networks admit a natural interpretation as discretized dynamical systems.

21.7.1 Discrete Dynamics

The update

$$h^{(\ell+1)} = h^{(\ell)} + F^{(\ell)}(h^{(\ell)})$$

resembles a forward Euler step for an ordinary differential equation:

$$\frac{dh}{dt} = F(h).$$

21.7.2 ODE Perspective

In the limit of many small layers, the network approximates a continuous-time system:

$$h(t + \Delta t) \approx h(t) + \Delta t\, F(h(t)).$$

This perspective suggests:

- depth corresponds to time,
- layers correspond to discrete evolution steps,
- stability depends on step size and dynamics.

This viewpoint has inspired continuous-depth models and neural ODEs.

21.8 Beyond ResNets: Variants of Skip Connections

Skip connections appear in many architectures.

21.8.1 Dense Connections

In DenseNet-style architectures, each layer receives inputs from all previous layers:

$$h^{(\ell)} = F^{(\ell)}(h^{(0)}, \ldots, h^{(\ell-1)}).$$

This encourages feature reuse and improves gradient flow.

21.8.2 Encoder–Decoder Skips (U-Net)

In encoder–decoder models, skip connections link early layers to later layers:

- preserve spatial information,
- combine low-level and high-level features.

This is widely used in segmentation and generative tasks.

21.9 Worked Comparison: Plain vs Residual Networks

21.9.1 Plain Network

$$h^{(\ell+1)} = F^{(\ell)}(h^{(\ell)}).$$

Each layer must learn a full transformation.

21.9.2 Residual Network

$$h^{(\ell+1)} = h^{(\ell)} + F^{(\ell)}(h^{(\ell)}).$$

Each layer learns only a correction.

21.9.3 Implication

Residual networks:

- are easier to optimize,
- support greater depth,
- improve gradient flow.

21.10 A Unifying Perspective

Skip connections fundamentally change the geometry of deep networks. By anchoring each layer to the identity, they:

- stabilize signal propagation,
- improve gradient transport,
- align with iterative refinement and dynamical systems.

Thus residual networks succeed not by increasing expressivity alone, but by making deep models *trainable*.

21.11 Summary

In this chapter we studied residual networks and skip connections. We explained the degradation problem and showed how residual updates reformulate learning as perturbations of the identity. We analyzed gradient transport and demonstrated how skip connections stabilize training.

We also introduced the dynamical-systems perspective and discussed extensions such as DenseNet and U-Net. Residual connections are one of the key architectural innovations enabling modern deep learning.

Chapter 22

Convolutional Neural Networks

22.1 Introduction

Convolutional neural networks (CNNs) are designed to exploit spatial structure in data, particularly images. Unlike fully connected networks, CNNs incorporate locality, weight sharing, and translation equivariance directly into their architecture.

This chapter develops CNNs from a mathematical perspective. We interpret convolution as a structured linear operator, formalize translation equivariance, and discuss pooling and receptive fields. We also briefly connect convolution to the Fourier domain.

22.2 Convolution as a Local Linear Operator

A convolution layer applies a linear operator with a special structure.

22.2.1 Discrete Convolution

For a 1D signal $x \in \mathbb{R}^n$ and kernel $k \in \mathbb{R}^r$, the discrete convolution is:

$$(y * k)_i = \sum_{j=0}^{r-1} k_j \, x_{i-j}.$$

In 2D (for images), with input x and kernel k:

$$(y * k)_{i,j} = \sum_{u,v} k_{u,v} \, x_{i-u,j-v}.$$

22.2.2 Structured Linear Map

Convolution can be written as a matrix-vector product:

$$y = Kx,$$

where K is a Toeplitz (or block Toeplitz) matrix. Unlike a fully connected layer, K has:

- repeated entries (weight sharing),
- sparse local structure.

Thus convolution is a linear map with strong structural constraints.

22.3 Weight Sharing and Equivariance

22.3.1 Weight Sharing

In convolution, the same kernel is applied across all spatial locations. This drastically reduces the number of parameters compared to fully connected layers.

22.3.2 Translation Equivariance

[Translation Operator] Let T_τ shift an input by τ. For example:

$$(T_\tau x)_i = x_{i-\tau}.$$

[Equivariance of Convolution] For a convolution operator C, we have:

$$C(T_\tau x) = T_\tau(Cx).$$

22.3.3 Interpretation

A shift in the input produces a corresponding shift in the output. Thus convolution preserves spatial structure.

This is a key inductive bias:

- features detected in one location are detected everywhere,
- the model is robust to translations.

22.4 Pooling, Receptive Fields, and Hierarchy

22.4.1 Pooling

Pooling reduces spatial resolution:

- max pooling selects the maximum in a local region,

- average pooling computes local averages.

Pooling introduces:

- invariance to small translations,
- reduced computational cost.

22.4.2 Receptive Fields

The receptive field of a unit is the region of the input that influences it.

- early layers: small receptive fields (local features),
- deeper layers: larger receptive fields (global structure).

Stacking convolution and pooling layers increases the effective receptive field.

22.4.3 Hierarchical Representation

CNNs build hierarchical features:

- edges and textures in early layers,
- shapes and parts in intermediate layers,
- objects in deeper layers.

This mirrors the compositional structure of visual data.

22.5 CNN Architectures in Vision

Typical CNN architectures combine:

- convolution layers,
- nonlinear activations,
- pooling or strided convolutions,
- normalization,
- fully connected or global pooling layers.

Modern architectures often incorporate:

- residual connections,
- bottleneck blocks,
- deeper hierarchies.

These design choices reflect tradeoffs between expressivity, efficiency, and trainability.

22.6 Spectral/Fourier Perspective

Convolution has a natural interpretation in the frequency domain.

22.6.1 Convolution Theorem

For suitable signals:

$$\mathcal{F}(x * k) = \mathcal{F}(x) \cdot \mathcal{F}(k),$$

where $\mathcal{F}$ denotes the Fourier transform.

22.6.2 Interpretation

- convolution in space corresponds to multiplication in frequency,
- kernels act as frequency filters,
- CNNs can be viewed as learning filters in the spectral domain.

For example:

- edge detectors emphasize high frequencies,
- smoothing filters suppress high frequencies.

Thus CNNs can be interpreted as adaptive filter banks.

22.7 Worked Examples

22.7.1 Feature Map Size

For input size n, kernel size r, and stride s, the output size is approximately:

$$\left\lfloor \frac{n-r}{s} \right\rfloor + 1.$$

Padding can adjust this to preserve spatial dimensions.

22.7.2 Equivariance Example

Let x be a signal and T_τ a shift. Then:

$$(Cx)_i = \sum_j k_j x_{i-j},$$

$$(C(T_\tau x))_i = \sum_j k_j x_{i-j-\tau} = (Cx)_{i-\tau}.$$

Thus:

$$C(T_\tau x) = T_\tau(Cx).$$

22.8 A Unifying Perspective

CNNs impose strong structural assumptions:

- locality: interactions are local,
- weight sharing: same features across space,
- equivariance: structure preserved under translation.

These assumptions reduce the hypothesis space while aligning it with natural image structure.

From a linear-algebra viewpoint, convolution is a structured matrix. From a signal-processing viewpoint, it is filtering. From a learning viewpoint, it encodes inductive bias.

22.9 Summary

In this chapter we developed convolutional neural networks from a mathematical perspective. We defined discrete convolution as a structured linear operator and showed how weight sharing leads to translation equivariance. We discussed pooling, receptive fields, and hierarchical feature construction.

We also introduced the Fourier viewpoint, interpreting convolution as frequency-domain filtering. CNNs illustrate how architectural structure can encode powerful inductive biases for specific data domains.

Chapter 23

Sequence Models

23.1 Introduction

Many learning problems involve sequential data: text, speech, time series, and control signals. In such settings, predictions depend not only on the current input but also on past inputs. Sequence models address this by maintaining a *hidden state* that evolves over time.

This chapter develops recurrent neural networks (RNNs) as dynamical systems with memory. We derive backpropagation through time (BPTT), analyze gradient behavior, and introduce gated architectures such as LSTM and GRU. We also discuss the limitations of recurrence.

23.2 Sequential Data and Hidden State

Given a sequence $(x_1, x_2, \ldots, x_T)$, a sequence model produces outputs $(y_1, \ldots, y_T)$, where each output may depend on the entire history.

A common approach is to maintain a hidden state h_t:

$$h_t = f(h_{t-1}, x_t), \qquad y_t = g(h_t).$$

The hidden state serves as a summary of past information.

23.2.1 State-Space View

This can be interpreted as a state-space model:

$$\text{state update: } h_t = f(h_{t-1}, x_t), \qquad \text{output: } y_t = g(h_t).$$

Thus sequence models evolve over time, storing information in the state.

23.3 Recurrent Neural Networks as Dynamical Systems

A standard RNN has the form:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b),$$

$$y_t = W_y h_t.$$

23.3.1 Dynamical Systems Interpretation

The recurrence defines a discrete-time dynamical system:

$$h_t = F(h_{t-1}, x_t).$$

- h_t is the system state,
- x_t acts as an external input,
- the network evolves over time.

This perspective emphasizes:

- memory as persistence of state,
- dynamics as repeated application of the same transformation.

23.4 Backpropagation Through Time

Training RNNs requires computing gradients through the entire sequence.

23.4.1 Unrolling the Network

An RNN over T steps can be viewed as a depth-T network:

$$h_1 \to h_2 \to \cdots \to h_T.$$

This is called *unrolling* in time.

23.4.2 Gradient Recursion

Let L be the total loss. Then:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t} + \frac{\partial \ell_t}{\partial h_t}.$$

This recursion propagates gradients backward through time.

23.4.3 Jacobian Product

Gradients involve products of Jacobians:

$$\prod_{k=t}^{T} \frac{\partial h_{k+1}}{\partial h_k}.$$

As in deep feedforward networks, repeated multiplication leads to vanishing or exploding gradients.

23.5 Vanishing and Exploding Gradients in RNNs

If the Jacobian norms are less than 1, gradients decay exponentially:

$$\left\| \frac{\partial h_T}{\partial h_t} \right\| \to 0.$$

If greater than 1, they grow exponentially.

23.5.1 Implications

- long-range dependencies are hard to learn,
- early inputs have little influence on gradients,
- training becomes unstable.

This is a central limitation of basic RNNs.

23.6 Gated Architectures: LSTM and GRU

To address gradient issues, gated architectures introduce mechanisms for controlling information flow.

23.6.1 LSTM (Long Short-Term Memory)

An LSTM maintains a cell state c_t:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

where:

- f_t is the forget gate,
- i_t is the input gate,
- $\tilde{c}_t$ is the candidate update.

The hidden state is:

$$h_t = o_t \odot \tanh(c_t),$$

with output gate o_t.

23.6.2 Key Idea

The cell state evolves additively rather than multiplicatively:

$$c_t \approx c_{t-1} + \text{correction}.$$

This helps preserve gradients over long time spans.

23.6.3 GRU (Gated Recurrent Unit)

GRU simplifies LSTM:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t,$$

where z_t is an update gate.

This combines memory and update in a simpler form.

23.6.4 Interpretation

Gates control:

- what to remember,
- what to forget,
- what to update.

They enable adaptive memory and improved gradient flow.

23.7 Limits of Recurrence

Despite improvements, recurrent models have limitations:

- difficulty capturing very long-range dependencies,
- sequential computation limits parallelization,
- training can still be unstable,
- memory capacity is constrained by state dimension.

These limitations motivate alternative architectures, such as attention-based models.

23.8 Worked Example: Gradient Through Time

For a simple RNN:

$$h_t = \sigma(W h_{t-1}),$$

we have:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t}^{T-1} W^\top \operatorname{diag}(\sigma'(z_k)).$$

This illustrates how repeated multiplication governs gradient behavior.

23.9 A Unifying Perspective

Sequence models extend neural networks into time by introducing state. RNNs are dynamical systems, and training them requires propagating gradients through time.

The main challenge is maintaining information across many steps. Gated architectures address this by introducing controlled memory updates, while the dynamical-systems view highlights the role of stability and long-term behavior.

23.10 Summary

In this chapter we studied sequence models from a dynamical perspective. We introduced recurrent neural networks, interpreted them as state-space systems, and derived backpropagation through time. We showed how gradient instability limits basic RNNs and motivates gated architectures such as LSTM and GRU.

We also discussed the limitations of recurrence, setting the stage for attention-based models in the next chapter.

Chapter 24

Attention and Transformers

24.1 Introduction

Recurrent models process sequences sequentially, which limits parallelization and makes long-range dependencies difficult to learn. Early sequence-to-sequence (seq2seq) models compress an entire input sequence into a single fixed-size vector, creating a severe information bottleneck.

Attention mechanisms and transformers overcome these limitations by allowing direct interactions between all elements of a sequence. Instead of compressing information into a single state, attention enables *content-based retrieval* across the entire sequence.

24.2 Why Recurrence Was Not Enough

RNN-based seq2seq models map an input sequence to a single hidden vector:

$$(x_1, \ldots, x_T) \mapsto h_T.$$

This creates:

- a bottleneck limiting information capacity,
- difficulty in modeling long-range dependencies,
- sequential computation preventing parallelism.

Attention addresses these issues by allowing each output to access all input positions directly.

24.3 Attention as Adaptive Weighting

Attention computes a weighted combination of values based on similarity.

24.3.1 Attention Formula

Given:

- query q,
- keys k_i,
- values v_i,

attention is:

$$\mathrm{Attn}(q, K, V) = \sum_i \alpha_i v_i, \quad \alpha_i = \frac{\exp(q^\top k_i)}{\sum_j \exp(q^\top k_j)}.$$

24.3.2 Interpretation

- $q^\top k_i$ measures similarity,
- α_i selects relevant information,
- output is a weighted aggregation.

Thus attention is a form of content-based retrieval.

24.4 Self-Attention and Token Interaction

In self-attention, queries, keys, and values all come from the same sequence.

24.4.1 Matrix Form

Let $X \in \mathbb{R}^{n \times d}$. Define:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Then:

$$\mathrm{Attn}(X) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

24.4.2 Interpretation

- $QK^\top$ computes all pairwise interactions,
- softmax normalizes attention weights,
- each token aggregates information from all others.

Thus self-attention enables direct token-to-token communication.

24.5 Positional Encoding and Order Information

Self-attention is permutation-invariant: it does not encode order by itself.

To incorporate sequence order, positional encodings are added:

$$X \leftarrow X + P,$$

where P encodes position information.

24.5.1 Types

- fixed sinusoidal encodings,
- learned embeddings.

This allows the model to distinguish positions in the sequence.

24.6 Multi-Head Attention

Instead of a single attention operation, transformers use multiple heads:

$$\text{MultiHead}(X) = \text{concat}(\text{head}_1, \ldots, \text{head}_h)W_O.$$

Each head computes:

$$\text{head}_i = \text{Attn}(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)}).$$

24.6.1 Interpretation

- different heads capture different relationships,
- the model attends to multiple patterns simultaneously.

24.7 Transformer Blocks

A transformer layer consists of:

- multi-head self-attention,
- feedforward (MLP) layer,
- normalization,
- residual connections.

24.7.1 Structure

$$X \to X + \mathrm{Attn}(X) \to \mathrm{Norm} \to X + \mathrm{MLP}(X) \to \mathrm{Norm}.$$

This combines attention with nonlinear transformation and stabilization.

24.8 Encoder, Decoder, and Causal Masking

24.8.1 Encoder

Processes the entire sequence with full self-attention.

24.8.2 Decoder

Generates outputs sequentially, using *causal masking* to prevent access to future tokens.

[Causal Masking] In attention weights, enforce:

$$\alpha_{ij} = 0 \quad \text{for } j > i.$$

This ensures autoregressive behavior.

24.8.3 Encoder–Decoder Attention

The decoder attends both to:

- its own past outputs,
- the encoder outputs.

24.9 From Similarity to Aggregation (Worked Derivation)

Starting from similarity scores:

$$s_i = q^\top k_i,$$

we normalize:

$$\alpha_i = \frac{e^{s_i}}{\sum_j e^{s_j}},$$

and compute:

$$\text{output} = \sum_i \alpha_i v_i.$$

Thus attention is:

$$\text{similarity} \to \text{weights} \to \text{aggregation}.$$

24.10 Scaling and Limitations

24.10.1 Advantages

- parallel computation,
- direct modeling of long-range dependencies,
- flexible interaction patterns.

24.10.2 Limitations

- quadratic cost in sequence length,
- large memory usage,
- sensitivity to data scale.

These motivate efficient attention variants.

24.11 A Unifying Perspective

Attention replaces recurrence with direct interaction. Instead of passing information through a chain of states, it allows every element to access every other element in one step.

Transformers combine:

- attention for interaction,
- MLPs for nonlinear transformation,
- normalization and residuals for stability.

This architecture has become a dominant paradigm in modern machine learning.

24.12 Summary

In this chapter we developed attention mechanisms and transformer architectures. We motivated attention from the limitations of recurrence, defined attention as content-based retrieval, and derived self-attention in matrix form.

We introduced positional encoding, multi-head attention, and transformer blocks, and explained encoder–decoder structures and causal masking. Transformers provide a powerful and flexible framework for sequence modeling.

Part II Exercises: Core Supervised Learning Models and Deep Learning

Exercises

1. Compare linear models, kernel methods, and neural networks in terms of:
   (a) representation,
   (b) inductive bias,
   (c) scalability.
2. Show why XOR cannot be solved by a linear classifier but can be solved by a two-layer network.
3. Write a two-layer neural network explicitly as a composition of functions.
4. Count the number of parameters in:
   (a) a fully connected layer,
   (b) a convolutional layer.
5. Derive backpropagation updates for a simple two-layer network.
6. Explain why gradients vanish in deep networks.
7. Compare ReLU and sigmoid activations in terms of gradient behavior.
8. Explain why batch normalization improves optimization.
9. Compare dropout and weight decay conceptually.
10. Show how a residual block approximates an identity mapping.
11. Prove translation equivariance of convolution in a simple 1D case.
12. Compute the output size of a convolution layer given kernel size and stride.

13. Unroll an RNN for T steps and write it as a feedforward network.
14. Derive the gradient of an RNN through time.
15. Compare LSTM and GRU gating mechanisms.
16. Compute a simple attention output for a toy example.
17. Compare path length between two tokens in:
    (a) an RNN,
    (b) a transformer.
18. Explain why transformers handle long-range dependencies better than RNNs.
19. Describe the role of positional encoding.
20. Explain how multi-head attention increases model capacity.
21. Design a simple architecture for:
    (a) image classification,
    (b) sequence modeling,
    (c) tabular data.

Chapter 25

Foundations of Generative Modeling

25.1 Introduction

Generative modeling aims to learn the underlying distribution of data rather than just predicting labels. Instead of learning a mapping $x \mapsto y$, a generative model seeks to capture the structure of $p(x)$ or related distributions, enabling tasks such as sampling, density estimation, and representation learning.

This chapter establishes the conceptual and mathematical foundations for generative modeling. We introduce different types of generative models, clarify their objectives, and distinguish between likelihood-based, sampling-based, and representation-based perspectives.

25.2 What It Means to Model a Data Distribution

Let $X \sim p_{\text{data}}$ denote the unknown data-generating distribution. A generative model aims to learn a distribution $p_\theta(x)$ such that:

$$p_\theta(x) \approx p_{\text{data}}(x).$$

This can be approached in several ways:

- estimating densities,
- generating samples,
- learning latent representations.

Each perspective emphasizes a different aspect of modeling.

25.3 Joint, Marginal, and Conditional Generative Models

Generative modeling can involve different distributions:

25.3.1 Marginal Models

Model $p(x)$ directly:

$$p_\theta(x) \approx p_{\text{data}}(x).$$

25.3.2 Joint Models

Model the joint distribution:

$$p_\theta(x, y),$$

which allows derivation of conditionals via:

$$p(y|x) = \frac{p(x, y)}{p(x)}.$$

25.3.3 Conditional Models

Model $p_\theta(x|y)$ or $p_\theta(y|x)$ directly. Examples include conditional generative models and sequence-to-sequence systems.

Each choice reflects a different modeling objective and application.

25.4 Sampling, Likelihood, and Inference

Generative models are often evaluated along three axes:

25.4.1 Sampling

Can we generate realistic samples $x \sim p_\theta(x)$?

25.4.2 Likelihood

Can we evaluate or approximate $p_\theta(x)$?

25.4.3 Inference

Can we infer latent structure, such as hidden variables z?

Different models prioritize different aspects:

- likelihood-based models focus on density estimation,
- implicit models focus on sampling,
- latent-variable models emphasize representation.

25.5 Latent-Variable Models

Many generative models introduce latent variables z:

$$p_\theta(x) = \int p_\theta(x|z)p(z)\,dz.$$

25.5.1 Interpretation

- z captures underlying structure,
- $p(z)$ is a prior,
- $p_\theta(x|z)$ generates data from latent codes.

Latent-variable models enable:

- structured generation,
- disentangled representations,
- dimensionality reduction.

25.6 Explicit Density, Implicit Density, and Energy Views

Generative models differ in how they represent distributions.

25.6.1 Explicit Density Models

These define $p_\theta(x)$ directly and allow likelihood evaluation.

Examples:

- autoregressive models,
- normalizing flows,
- variational autoencoders (approximate likelihood).

25.6.2 Implicit Models

These define a sampling procedure:

$$x = G_\theta(z), \quad z \sim p(z),$$

without explicit density evaluation.

Example:

- generative adversarial networks (GANs).

25.6.3 Energy-Based Models

These define:

$$p_\theta(x) \propto e^{-E_\theta(x)},$$

where $E_\theta(x)$ is an energy function.

They avoid explicit normalization but require specialized inference methods.

25.7 Evaluation of Generative Models

Evaluating generative models is challenging because there is no single metric.

25.7.1 Likelihood-Based Evaluation

- log-likelihood,
- bits-per-dimension.

These measure how well the model fits the data distribution.

25.7.2 Sample Quality

- visual inspection,
- metrics such as FID (in practice).

These measure realism of generated samples.

25.7.3 Tradeoffs

High likelihood does not always imply high-quality samples, and vice versa. This reflects differences between modeling density and capturing perceptual structure.

25.8 Discriminative vs Generative Objectives

Discriminative models learn:

$$p(y|x),$$

while generative models learn:

$$p(x) \quad \text{or} \quad p(x, y).$$

25.8.1 Comparison

- discriminative models focus on prediction accuracy,
- generative models aim to capture data structure,
- generative models can simulate data and support unsupervised learning.

25.9 A Unifying Perspective

Generative modeling can be understood as learning a probability distribution over data. Different approaches emphasize:

- likelihood (explicit density models),
- sampling (implicit models),
- structure (latent-variable models),
- energy (unnormalized models).

These perspectives are complementary and lead to different algorithmic approaches.

25.10 Summary

In this chapter we introduced the foundations of generative modeling. We defined the goal of modeling a data distribution and distinguished between marginal, joint, and conditional models. We discussed sampling, likelihood, and inference as key evaluation axes, and introduced latent-variable models.

We also classified generative models into explicit density, implicit, and energy-based approaches, and discussed challenges in evaluation. This framework sets the stage for specific generative models in the following chapters.

Chapter 26

Likelihood-Based Learning with Latent Variables

26.1 Introduction

Many generative models introduce hidden or latent variables to capture underlying structure in data. While this increases modeling flexibility, it also complicates learning: likelihood functions now involve integrals or sums over unobserved variables.

This chapter develops maximum likelihood learning in latent-variable models. We introduce marginal likelihood, derive the Expectation–Maximization (EM) algorithm, and show how intractability leads naturally to variational approximations.

26.2 Why Latent Variables Are Useful

Latent variables z represent hidden causes or factors explaining observed data x.

- in clustering, z indicates cluster membership,
- in representation learning, z encodes underlying features,
- in generative models, z controls variation in outputs.

They allow complex distributions to be expressed as mixtures or compositions of simpler ones.

26.3 Latent-Variable Models and Marginalization

[Latent-Variable Model] A latent-variable model specifies a joint distribution:

$$p_\theta(x, z) = p_\theta(x|z)p(z),$$

where z is unobserved.

The marginal distribution over observed data is:

$$p_\theta(x) = \int p_\theta(x, z)\, dz.$$

26.3.1 Marginal Log-Likelihood

Given data $\{x_i\}_{i=1}^n$, the objective is:

$$\ell(\theta) = \sum_{i=1}^n \log p_\theta(x_i) = \sum_{i=1}^n \log \int p_\theta(x_i, z)\, dz.$$

This integral often makes optimization difficult.

26.4 Maximum Likelihood with Hidden Variables

Unlike fully observed models, the likelihood involves sums or integrals over latent variables:

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz.$$

This creates two challenges:

- **inference:** computing the posterior $p_\theta(z|x)$,
- **learning:** maximizing the marginal likelihood.

These are coupled problems: learning requires inference, and inference depends on the model.

26.5 The EM Algorithm: Intuition

EM alternates between estimating latent variables and updating parameters.

- **E-step:** estimate distribution over latent variables,
- **M-step:** maximize expected complete-data log-likelihood.

26.5.1 Key Idea

Instead of optimizing $\log p_\theta(x)$ directly, EM optimizes a surrogate objective based on the current estimate of latent variables.

26.6 Lower-Bound Viewpoint via Jensen

Let $q(z)$ be any distribution. Then:

$$\log p_\theta(x) = \log \int q(z) \frac{p_\theta(x,z)}{q(z)}\, dz \geq \int q(z) \log \frac{p_\theta(x,z)}{q(z)}\, dz.$$

[Evidence Lower Bound (ELBO)]

$$\mathcal{L}(q,\theta) = \mathbb{E}_{q(z)}[\log p_\theta(x,z)] - \mathbb{E}_{q(z)}[\log q(z)].$$

This provides a lower bound on $\log p_\theta(x)$.

26.6.1 Interpretation

- the bound becomes tight when $q(z) = p_\theta(z|x)$,
- EM can be viewed as alternating optimization of this bound.

26.7 The EM Algorithm: Derivation

EM alternates:

26.7.1 E-step

Set:

$$q(z) = p_{\theta^{\text{old}}}(z|x).$$

26.7.2 M-step

Maximize:

$$\theta^{\text{new}} = \arg\max_\theta \mathbb{E}_{z\sim p_{\theta^{\text{old}}}(z|x)}[\log p_\theta(x,z)].$$

26.7.3 Monotonicity

Each iteration increases the log-likelihood:

$$\ell(\theta^{\text{new}}) \geq \ell(\theta^{\text{old}}).$$

Thus EM is guaranteed to improve (or maintain) likelihood at each step.

26.8 Worked Example: Gaussian Mixture Model

Consider a mixture of K Gaussians:

$$p_\theta(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k).$$

Introduce latent variables $z \in \{1, \ldots, K\}$.

26.8.1 E-step

Compute responsibilities:

$$\gamma_{ik} = p(z = k \mid x_i) = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}.$$

26.8.2 M-step

Update parameters:

$$\mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}}, \qquad \pi_k = \frac{1}{n} \sum_i \gamma_{ik}.$$

This alternates soft clustering and parameter updates.

26.9 Limits of Exact Inference

In many models, computing $p_\theta(z|x)$ is intractable:

- high-dimensional latent spaces,
- complex dependencies,
- non-conjugate models.

This makes exact EM impractical.

26.9.1 Consequences

- need for approximate inference,
- motivates variational inference,
- leads to modern deep generative models.

26.10 Why Latent Variables Complicate Optimization

Latent variables introduce:

- non-convex objectives,
- multiple modes,
- dependence on inference quality.

Optimization becomes a joint problem of:

- fitting parameters,
- inferring hidden structure.

Thus learning is more complex than in fully observed models.

26.11 A Unifying Perspective

Latent-variable models express complex distributions through hidden structure. Maximum likelihood requires marginalizing over latent variables, which leads to intractability.

EM provides a principled approach by alternating between inference and optimization, while the ELBO offers a variational perspective. These ideas form the foundation for modern methods such as variational autoencoders.

26.12 Summary

In this chapter we developed likelihood-based learning with latent variables. We defined latent-variable models and marginal likelihood, and explained why hidden variables complicate optimization. We derived the EM algorithm and interpreted it through a lower-bound (ELBO) perspective.

We also worked through Gaussian mixtures and discussed the limits of exact inference. These ideas provide the conceptual bridge to variational inference and modern generative models.

Chapter 27

Variational Autoencoders

27.1 Introduction

Variational Autoencoders (VAEs) combine latent-variable generative modeling with neural networks and variational inference. They provide a principled framework for learning probabilistic generative models while remaining computationally tractable.

This chapter develops VAEs from first principles: we derive the Evidence Lower Bound (ELBO), introduce the encoder–decoder architecture, and explain the reparameterization trick. We also interpret the tradeoff between reconstruction and regularization and discuss common failure modes.

27.2 Autoencoding and Latent-Variable Modeling

A latent-variable generative model assumes:

$$p_\theta(x) = \int p_\theta(x|z)p(z)\,dz.$$

In VAEs:

- $p(z)$ is a simple prior (e.g., $\mathcal{N}(0, I)$),
- $p_\theta(x|z)$ is a neural decoder.

27.2.1 Autoencoding View

We introduce an encoder $q_\phi(z|x)$ that maps data to latent variables. Thus:

- encoder: $x \to z$,
- decoder: $z \to x$.

Unlike classical autoencoders, this mapping is probabilistic.

27.3 Variational Inference and the ELBO

The log-likelihood is:

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz.$$

Introduce an approximate posterior $q_\phi(z|x)$:

$$\log p_\theta(x) = \log \int q_\phi(z|x) \frac{p_\theta(x, z)}{q_\phi(z|x)}\, dz.$$

Applying Jensen's inequality:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right].$$

[ELBO]

$$\mathcal{L}(x; \theta, \phi) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(z|x) \| p(z)).$$

27.3.1 KL Decomposition

We have:

$$\log p_\theta(x) = \mathcal{L}(x; \theta, \phi) + \mathrm{KL}(q_\phi(z|x) \| p_\theta(z|x)).$$

Thus maximizing the ELBO both:

- increases likelihood,
- improves approximation of the true posterior.

27.4 Encoder–Decoder Architecture

- Encoder: $q_\phi(z|x)$ outputs parameters of a distribution over z,
- Decoder: $p_\theta(x|z)$ generates data from latent variables.

27.4.1 Typical Gaussian Encoder

$$q_\phi(z|x) = \mathcal{N}(z \mid \mu_\phi(x), \Sigma_\phi(x)).$$

27.4.2 Decoder

$$p_\theta(x|z) = \text{neural network output distribution.}$$

Thus learning involves jointly optimizing θ and ϕ.

27.5 Reparameterization Trick

Direct gradient estimation of the ELBO is difficult due to sampling from $q_\phi(z|x)$.

27.5.1 Reparameterization

For Gaussian latent variables:

$$z \sim \mathcal{N}(\mu, \sigma^2) \quad \Longleftrightarrow \quad z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

27.5.2 Reparameterization Estimator

The expectation becomes:

$$\mathbb{E}_{q_\phi(z|x)}[f(z)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0,I)}[f(\mu_\phi(x) + \sigma_\phi(x)\epsilon)].$$

This allows gradients to flow through μ_ϕ and σ_ϕ.

27.6 Full VAE Objective

The ELBO becomes:

$$\mathcal{L}(x) = \mathbb{E}_\epsilon \left[\log p_\theta(x|z)\right] - \mathrm{KL}(q_\phi(z|x) \| p(z)).$$

27.6.1 Interpretation

- first term: reconstruction accuracy,
- second term: regularization toward prior.

Thus VAE balances:

$$\text{reconstruction} \quad \text{vs} \quad \text{latent regularization.}$$

27.7 Gaussian Latent VAE

For:

$$q_\phi(z|x) = \mathcal{N}(\mu, \sigma^2), \quad p(z) = \mathcal{N}(0, I),$$

the KL term has closed form:

$$\mathrm{KL} = \frac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\right).$$

This makes training efficient and stable.

27.8 Interpretation of the KL Term

The KL term:

- encourages $q_\phi(z|x)$ to stay close to $p(z)$,
- prevents overfitting,
- enforces a structured latent space.

27.8.1 Tradeoff

- strong KL: better regularization, worse reconstruction,
- weak KL: better reconstruction, risk of overfitting.

27.9 Failure Modes and Variants

27.9.1 Posterior Collapse

The encoder ignores x and outputs:

$$q_\phi(z|x) \approx p(z).$$

Then:

- latent variables carry little information,
- model behaves like an unconditional generator.

27.9.2 Causes

- overly powerful decoder,
- strong KL regularization.

27.9.3 Variants

- β-VAE: scale KL term,
- KL annealing,
- richer posterior families.

27.10 A Unifying Perspective

VAEs combine:

- latent-variable generative modeling,
- variational inference,
- neural parameterization.

The ELBO provides a tractable objective balancing data fit and latent structure. The reparameterization trick enables efficient gradient-based learning.

Thus VAEs bridge probabilistic modeling and deep learning.

27.11 Summary

In this chapter we developed variational autoencoders. We derived the ELBO, introduced encoder–decoder architectures, and explained the reparameterization trick. We interpreted the KL term and the reconstruction tradeoff, and discussed failure modes such as posterior collapse.

VAEs provide a principled and flexible framework for generative modeling with latent variables.

Chapter 28

Generative Adversarial Networks

28.1 Introduction

Generative Adversarial Networks (GANs) take a fundamentally different approach to generative modeling. Instead of explicitly modeling a likelihood, GANs learn to generate data by playing a game between two networks: a generator and a discriminator.

This chapter develops GANs from a game-theoretic perspective. We derive the minimax objective, analyze the role of the discriminator, and interpret GANs as implicitly minimizing a divergence between distributions. We also discuss training instabilities and common remedies.

28.2 Adversarial Learning Setup

GANs consist of two components:

- **Generator** $G_\theta(z)$: maps noise $z \sim p(z)$ to data space,
- **Discriminator** $D_\phi(x)$: outputs a score indicating whether x is real or generated.

The goal is:

- G_θ: produce realistic samples,
- D_ϕ: distinguish real data from generated samples.

Thus training becomes a two-player game.

28.3 Minimax Formulation

The standard GAN objective is:

$$\min_{\theta} \max_{\phi} \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D_\phi(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D_\phi(G_\theta(z)))].$$

28.3.1 Interpretation

- the discriminator maximizes classification accuracy,
- the generator minimizes the discriminator's ability to distinguish samples.

This is a minimax optimization problem.

28.4 What the Discriminator Learns

For a fixed generator, the discriminator solves:

$$\max_{D} \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{x \sim p_G}[\log(1 - D(x))].$$

[Optimal Discriminator] The optimal discriminator is:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

28.4.1 Interpretation

- $D^*(x)$ estimates the probability that x is real,
- it compares densities of real and generated data.

Thus the discriminator implicitly learns a density ratio.

28.5 Divergence Minimization View

Substituting D^* into the objective yields:

$$\min_{G} \; 2 \cdot \mathrm{JS}(p_{\text{data}} \,\|\, p_G) - \log 4,$$

where JS is the Jensen–Shannon divergence.

28.5.1 Interpretation

GAN training can be viewed as minimizing a divergence between:

- the true data distribution,
- the generator distribution.

However, this divergence is implicit and not computed directly.

28.6 Training Pathologies

GAN training is notoriously unstable.

28.6.1 Non-Convergence

The minimax game may not converge:

- oscillations between generator and discriminator,
- failure to reach equilibrium.

28.6.2 Mode Collapse

The generator produces limited diversity:

- maps many z values to similar outputs,
- ignores parts of the data distribution.

28.6.3 Vanishing Gradients

If the discriminator becomes too strong:

$$D(x) \approx 1, \quad D(G(z)) \approx 0,$$

then gradients for the generator become very small.

28.7 Stabilization Tricks

Several techniques improve GAN training:

- **Non-saturating loss:** replace $\log(1 - D(G(z)))$ with $-\log D(G(z))$,
- **Gradient penalties:** enforce smoothness of discriminator,
- **Label smoothing:** reduce overconfidence,
- **Architecture design:** use convolutional and residual structures,
- **Normalization:** stabilize training dynamics.

These do not fully solve instability but improve practical performance.

28.8 Conditional GANs

GANs can be extended to conditional settings:

$$G(z, y), \quad D(x, y).$$

28.8.1 Objective

$$\min_G \max_D \ \mathbb{E}_{x,y}[\log D(x,y)] + \mathbb{E}_{z,y}[\log(1 - D(G(z,y),y))].$$

28.8.2 Applications

- image-to-image translation,
- class-conditional generation,
- super-resolution.

Conditioning allows controlled generation.

28.9 Worked Derivation: Optimal Discriminator

We maximize:

$$\mathcal{L}(D) = \int p_{\text{data}}(x) \log D(x)\, dx + \int p_G(x) \log(1 - D(x))\, dx.$$

For each x, maximize:

$$p_{\text{data}}(x) \log D + p_G(x) \log(1 - D).$$

Setting derivative to zero:

$$\frac{p_{\text{data}}(x)}{D(x)} - \frac{p_G(x)}{1 - D(x)} = 0.$$

Solving:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}.$$

28.10 Wasserstein GAN (WGAN)

Standard GANs minimize a Jensen–Shannon divergence, which can lead to vanishing gradients when the real and generated distributions have little overlap. Wasserstein GANs address this by replacing the divergence with the *Wasserstein distance* (Earth Mover's distance), which provides more meaningful gradients.

28.10.1 Wasserstein Distance (Intuition)

The Wasserstein-1 distance between distributions p and q measures the minimum cost of transporting mass to transform p into q.

- robust even when supports do not overlap,
- provides informative gradients,
- better reflects distributional differences.

28.10.2 WGAN Objective

Using the Kantorovich–Rubinstein duality, the objective becomes:

$$\min_G \max_{D \in \mathcal{D}} \mathbb{E}_{x \sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z \sim p(z)}[D(G(z))],$$

where D (called a critic) is constrained to be 1-Lipschitz.

28.10.3 Interpretation

- $D(x)$ assigns higher values to real samples,
- the difference estimates the Wasserstein distance,
- the generator minimizes this distance.

28.11 Enforcing the Lipschitz Constraint

The key requirement in WGAN is that the critic must be 1-Lipschitz.

28.11.1 Weight Clipping (Original WGAN)

The original method enforces this by clipping weights:

$$w \leftarrow \text{clip}(w, -c, c).$$

28.11.2 Limitations

- restricts model capacity,
- can lead to poor optimization,
- sensitive to choice of clipping threshold.

28.12 WGAN-GP (Gradient Penalty)

WGAN-GP improves stability by enforcing the Lipschitz constraint through a gradient penalty.

28.12.1 Objective

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[D(x)] - \mathbb{E}_{z \sim p(z)}[D(G(z))] + \lambda \, \mathbb{E}_{\hat{x}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right],$$

where $\hat{x}$ are points interpolated between real and generated samples.

28.12.2 Interpretation

- penalizes deviation from unit gradient norm,
- enforces approximate Lipschitz continuity,
- leads to more stable training.

28.12.3 Advantages

- improved convergence,
- reduced mode collapse,
- more reliable gradients.

28.13 Comparison with Standard GANs

- Standard GAN:
  - minimizes JS divergence,
  - prone to vanishing gradients,
  - unstable training.
- WGAN:
  - minimizes Wasserstein distance,
  - stable gradients,
  - requires Lipschitz constraint.
- WGAN-GP:
  - improved enforcement via gradient penalty,
  - more robust in practice.

28.14 Updated Perspective on GANs

Wasserstein GANs highlight that the choice of divergence is crucial for training dynamics. By using a distance with better geometric properties, WGANs improve stability and interpretability.

This reinforces a broader principle:

$$\text{good optimization} \iff \text{well-behaved objective geometry.}$$

Thus GAN variants can be understood as modifying the training objective to improve gradient behavior and convergence.

28.15 A Unifying Perspective

GANs replace likelihood-based learning with adversarial learning. The generator and discriminator form a game in which the generator improves by fooling the discriminator, while the discriminator improves by detecting generated samples.

This leads to:

- powerful sample generation,
- implicit density modeling,
- challenging optimization dynamics.

GANs highlight a key idea: good generative models can be learned without explicitly computing likelihoods.

28.16 Summary

In this chapter we developed generative adversarial networks. We formulated GANs as a minimax game, derived the optimal discriminator, and interpreted training as divergence minimization. We discussed common training pathologies such as instability and mode collapse, along with practical remedies.

We also introduced conditional GANs for controlled generation. GANs provide a powerful alternative to likelihood-based generative modeling.

Chapter 29

Autoregressive Generative Models

29.1 Introduction

Autoregressive models provide a simple and powerful approach to generative modeling by factorizing a joint distribution into a product of conditional distributions. They form the foundation of modern language models and connect naturally to sequence modeling, attention, and transformers.

This chapter develops autoregressive models from first principles, covering likelihood-based training, causal masking, and decoding strategies. We also discuss their role in modern large language models.

29.2 Probability Factorization by the Chain Rule

Any joint distribution over variables $x = (x_1, \ldots, x_n)$ can be factorized as:

$$p(x) = \prod_{i=1}^{n} p(x_i \mid x_{<i}), \quad \text{where } x_{<i} = (x_1, \ldots, x_{i-1}).$$

- this is an exact identity,
- no approximation is introduced,
- modeling reduces to learning conditional distributions.

29.3 Autoregressive Models for Different Data Types

29.3.1 Sequences (Text)

For text:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

Each token is predicted from previous tokens.

29.3.2 Images

Images can be flattened into sequences (e.g., row-wise):

$$p(x) = \prod_{i=1}^{N} p(x_i \mid x_{<i}).$$

Here x_i corresponds to pixel intensities.

29.3.3 General Data

Any structured data can be ordered and modeled autoregressively, though the choice of ordering affects modeling efficiency.

29.4 Log-Likelihood Objective

Given data $\{x^{(j)}\}$, the log-likelihood is:

$$\sum_j \log p(x^{(j)}) = \sum_j \sum_{i=1}^{n} \log p(x_i^{(j)} \mid x_{<i}^{(j)}).$$

- training decomposes into local prediction problems,
- likelihood is tractable and exact,
- gradients are straightforward to compute.

29.5 Teacher Forcing and Next-Token Prediction

During training, models are typically trained with *teacher forcing*:

- the true previous tokens $x_{<t}$ are provided,
- the model predicts x_t.

This leads to the objective:

$$\sum_t \log p(x_t \mid x_{<t}),$$

which corresponds to next-token prediction.

29.6 Causal Masking and Training

In neural implementations (e.g., transformers), causal structure is enforced by masking.

[Causal Masking] During training, ensure that the prediction at position i depends only on $x_{<i}$ by masking future positions:

$$\text{attention weight } \alpha_{ij} = 0 \quad \text{for } j > i.$$

This ensures that the model respects the autoregressive factorization.

29.7 Sampling and Decoding

To generate samples, we proceed sequentially:

$$x_1 \sim p(x_1), \quad x_2 \sim p(x_2 \mid x_1), \quad \ldots$$

29.7.1 Decoding Strategies

- **Greedy decoding:** choose most probable token,
- **Sampling:** sample from the distribution,
- **Temperature scaling:** adjust randomness,
- **Top-k / nucleus sampling:** restrict candidate tokens.

29.7.2 Tradeoffs

- deterministic decoding yields coherent but less diverse outputs,
- stochastic decoding increases diversity but may reduce quality.

29.8 Exposure Bias

During training:

- model sees true history,

During generation:

- model conditions on its own predictions.

This mismatch is called *exposure bias*.

29.8.1 Implication

Errors can accumulate during generation, as mistakes affect future predictions.

29.9 Worked Examples

29.9.1 Character-Level Language Modeling

For a text sequence:

$$p(x) = \prod_{t=1}^{T} p(x_t \mid x_{<t}),$$

where x_t are characters.

The model predicts each character given previous ones.

29.9.2 Pixel Ordering in Images

Flatten an image into a sequence:

$$(x_{1,1}, x_{1,2}, \ldots, x_{H,W}).$$

Then:

$$p(x) = \prod_{i} p(x_i \mid x_{<i}).$$

This models spatial dependencies sequentially.

29.9.3 Simple Decoding Example

Given probabilities:

$$p(x_t \mid x_{<t}) = (0.7, 0.2, 0.1),$$

- greedy decoding selects the first token,
- sampling may choose any token according to probabilities.

29.10 Relationship to Modern Language Models

Modern large language models are autoregressive models trained on massive text corpora.

- transformers implement $p(x_t \mid x_{<t})$,
- causal masking enforces autoregression,
- training is next-token prediction.

Thus autoregressive modeling is the foundation of modern NLP systems.

29.11 Comparison with Latent-Variable Models

- autoregressive models:
  - exact likelihood,
  - sequential generation,
  - no latent variables required.
- latent-variable models:
  - introduce hidden structure,
  - may require approximate inference,
  - allow global representations.

Each approach has strengths and limitations.

29.12 A Unifying Perspective

Autoregressive models reduce generative modeling to conditional prediction. By factorizing distributions via the chain rule, they transform a complex global problem into a sequence of local prediction problems.

They connect naturally to:

- sequence modeling,
- attention mechanisms,
- modern language models.

29.13 Summary

In this chapter we developed autoregressive generative models. We derived the chain-rule factorization, introduced likelihood-based training, and explained teacher forcing and causal masking. We discussed sampling and decoding strategies, as well as exposure bias.

Autoregressive models provide a simple yet powerful framework that underlies modern large-scale generative systems.

Exercises

1. Write the autoregressive factorization for:
   (a) a sequence of words,
   (b) a grayscale image,
   (c) a time series.

2. Show that maximizing log-likelihood in an autoregressive model reduces to a sum of conditional log-probabilities.
3. Explain the difference between teacher forcing and free-running generation.
4. Compare greedy decoding and sampling. When might each be preferred?
5. Explain exposure bias and propose one possible mitigation strategy.
6. Compare autoregressive models and latent-variable models in terms of:
   (a) likelihood computation,
   (b) sampling,
   (c) representation learning.
7. For a simple sequence model, compute $p(x)$ given conditional probabilities.
8. Explain how causal masking enforces autoregressive structure in transformers.

Chapter 30

Diffusion Models: Intuition

30.1 Introduction

Diffusion models generate data by reversing a gradual corruption process. Instead of producing a sample in a single step, they start from noise and iteratively refine it into a structured output.

This chapter develops an intuitive understanding of diffusion models. We explain progressive noising, denoising, and the idea of score-based learning, and connect diffusion to iterative refinement.

30.2 Why Generate by Denoising?

Directly generating complex data (e.g., images) in one step is difficult. Diffusion models take a different approach:

- start from simple noise,
- gradually transform it into structured data,
- learn each small transformation.

This breaks a difficult problem into many simpler steps.

30.3 Forward Diffusion as Gradual Corruption

The forward process adds noise step by step:

$$x_0 \to x_1 \to x_2 \to \cdots \to x_T.$$

Each step adds small Gaussian noise:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I).$$

30.3.1 Interpretation

- early steps: slightly corrupted data,
- later steps: heavily corrupted,
- final state: nearly pure Gaussian noise.

Thus the forward process gradually destroys structure.

30.3.2 Gaussian Corruption Sequence (Example)

Starting from an image:

- x_0: clean image,
- x_1: slightly noisy,
- x_{10}: visibly corrupted,
- x_T: indistinguishable from noise.

30.4 Reverse Denoising as Learned Reconstruction

The generative process reverses this corruption:

$$x_T \to x_{T-1} \to \cdots \to x_0.$$

A neural network learns to remove noise at each step:

$$p_\theta(x_{t-1} \mid x_t).$$

30.4.1 Interpretation

Each step performs:

- a small denoising operation,
- a refinement of structure,
- a move toward the data distribution.

Thus generation is iterative reconstruction.

30.5 Noise Prediction and Score Intuition

Instead of directly predicting x_{t-1}, diffusion models often predict the noise ϵ added at each step.

30.5.1 Noise Prediction

Given x_t, the model predicts ϵ_t such that:

$$x_0 \approx \frac{x_t - \sqrt{\beta_t}\,\epsilon_t}{\sqrt{1-\beta_t}}.$$

30.5.2 Score Interpretation

The model implicitly learns the *score function*:

$$\nabla_x \log p(x),$$

which points toward regions of higher probability.

- denoising corresponds to moving toward higher-density regions,
- each step nudges samples toward realistic configurations.

Thus diffusion can be viewed as gradient ascent in probability space with noise.

30.6 Iterative Refinement View

Diffusion models generate by repeated small corrections:

$$x_{t-1} = x_t + \text{small update}.$$

This resembles:

- residual refinement,
- gradient-based optimization,
- iterative reconstruction algorithms.

30.6.1 Thought Experiment

Imagine starting from random pixels:

- each step removes a bit of noise,
- patterns begin to emerge,
- structure gradually sharpens,
- final result resembles real data.

This incremental process is easier to learn than direct generation.

30.7 Why Diffusion Models Work Well in Practice

Several factors contribute to their success:

- **Stable training:** objective resembles supervised denoising,
- **Well-behaved gradients:** avoids adversarial instability,
- **Iterative refinement:** decomposes complex generation,
- **Flexibility:** can model diverse data distributions.

Compared to GANs:

- more stable,
- less prone to mode collapse,
- but slower at sampling.

30.8 Worked Example: Small-Step Denoising

Suppose:

$$x_t = x_0 + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I).$$

If σ is small:

- x_t is close to x_0,
- predicting ϵ is easy,
- recovering x_0 is accurate.

Thus learning small denoising steps is easier than large jumps.

30.9 Comparison with Other Generative Models

- **Autoencoders:** reconstruct inputs in one step,
- **GANs:** generate samples via adversarial training,
- **Diffusion:** generate via many small denoising steps.

Diffusion trades speed for stability and quality.

30.10 A Unifying Perspective

Diffusion models view generation as reversing noise. The forward process simplifies data into noise; the reverse process reconstructs structure step by step.

This connects to:

- score matching (learning gradients of log-density),
- iterative optimization,
- dynamical systems.

Thus diffusion provides a conceptually simple and powerful generative paradigm.

30.11 Summary

In this chapter we developed an intuitive understanding of diffusion models. We described progressive noising, reverse denoising, and noise prediction. We connected these ideas to score functions and iterative refinement.

Diffusion models succeed by decomposing generation into many small, learnable steps, providing stability and high-quality outputs.

Exercises

1. Explain why small-step denoising is easier to learn than direct generation.
2. Consider a Gaussian corruption process. Show how variance increases over steps.
3. Compare diffusion models and autoencoders in terms of reconstruction.
4. Compare diffusion models and GANs in terms of:
   (a) training stability,
   (b) sample quality,
   (c) computational cost.
5. Describe how iterative refinement improves generation quality.
6. Explain the role of noise prediction in diffusion models.
7. Give an intuitive explanation of the score function $\nabla_x \log p(x)$.
8. Why do diffusion models avoid mode collapse more easily than GANs?

Chapter 31

Diffusion Models: Mathematical Foundations

31.1 Introduction

We now formalize diffusion models as probabilistic processes. The key idea is to define a forward Markov chain that gradually corrupts data into noise, and then learn a reverse process that reconstructs data.

This chapter derives the forward and reverse processes, shows how training objectives arise, and connects diffusion to score matching and stochastic differential equations.

31.2 Discrete-Time Diffusion Formulation

We define a forward Markov chain:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}),$$

where each transition adds Gaussian noise.

31.2.1 Forward Transitions

[Forward Process]

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t \;\middle|\; \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right),$$

where $\beta_t \in (0,1)$ is a noise schedule.

This defines a Markov chain that gradually destroys structure.

31.3 Forward Process: Closed-Form Marginals

A key property is that x_t can be sampled directly from x_0.

[Closed-Form Marginal]

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t \;\middle|\; \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)I\right),$$

where:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

31.3.1 Derivation Sketch

Repeated substitution gives:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Thus:

- mean shrinks toward zero,
- variance increases over time.

31.4 Reverse Process Parameterization

We define a learned reverse chain:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t),$$

with:

$$p(x_T) = \mathcal{N}(0, I).$$

31.4.1 Gaussian Parameterization

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1} \mid \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)).$$

The model learns to predict the mean (and optionally variance) of the reverse transition.

31.5 Training Objectives: ELBO View

Training maximizes log-likelihood:

$$\log p_\theta(x_0).$$

Using variational inference, we obtain an ELBO:

$$\mathbb{E}_q\left[\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0)\right].$$

This decomposes into terms comparing forward and reverse transitions.

31.5.1 Simplified Objective

After algebraic simplification, training reduces to:

$$\mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right],$$

where:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon.$$

31.5.2 Interpretation

- model predicts noise ϵ,
- equivalent to learning reverse dynamics,
- yields stable supervised learning objective.

31.6 Training Objective Transformations

Different parameterizations are equivalent:

- predicting noise ϵ,
- predicting x_0,
- predicting score $\nabla_x \log p(x_t)$.

These correspond to different but related loss formulations.

31.7 Score Matching and SDE View

Diffusion models are closely related to score matching.

31.7.1 Score Function

$$s(x) = \nabla_x \log p(x).$$

The model learns an approximation:

$$s_\theta(x_t, t).$$

31.7.2 Connection

Noise prediction is proportional to the score:

$$\epsilon_\theta(x_t, t) \iff -\nabla_x \log q(x_t).$$

Thus diffusion training implicitly performs score matching.

31.7.3 Continuous-Time View

As $T \to \infty$, the process becomes a stochastic differential equation (SDE):

$$dx = f(x,t)\,dt + g(t)\,dW_t.$$

The reverse process is another SDE that removes noise.

31.8 Langevin Dynamics Connection

Sampling can be interpreted as approximate Langevin dynamics:

$$x_{t-1} = x_t + \eta \nabla_x \log p(x_t) + \sqrt{2\eta}\,\xi.$$

- gradient term moves toward high-density regions,
- noise maintains stochasticity,
- diffusion combines both ideas.

31.9 DDPM, DDIM, and Sampler Intuition

31.9.1 DDPM (Stochastic Sampling)

- follows learned reverse transitions,
- includes stochastic noise at each step,
- produces diverse samples.

31.9.2 DDIM (Deterministic Sampling)

- removes stochasticity,
- produces deterministic trajectories,
- faster sampling with fewer steps.

31.9.3 Tradeoffs

- stochastic samplers: more diversity,
- deterministic samplers: faster and more controllable.

31.10 A Unifying Perspective

Diffusion models define:

- a forward stochastic process that simplifies data,
- a reverse learned process that reconstructs it.

Training can be viewed as:

- variational inference (ELBO),
- score matching,
- learning dynamics of a stochastic system.

Thus diffusion sits at the intersection of probability, optimization, and dynamical systems.

31.11 Summary

In this chapter we formalized diffusion models. We defined the forward Markov process, derived closed-form marginals, and introduced the reverse generative model. We showed how training arises from a variational objective and reduces to noise prediction.

We also connected diffusion to score matching, stochastic differential equations, and Langevin dynamics, and discussed different sampling schemes such as DDPM and DDIM.

Exercises

1. Derive the mean and variance of $q(x_t \mid x_0)$.
2. Show how x_t can be written as a linear combination of x_0 and Gaussian noise.
3. Derive the simplified noise-prediction objective from the ELBO.
4. Explain why predicting ϵ is equivalent to learning the score function.
5. Compare stochastic (DDPM) and deterministic (DDIM) sampling.
6. Show how variance accumulates over diffusion steps.
7. Derive the relationship between score matching and denoising.
8. Explain the role of the noise schedule $\{\beta_t\}$.

Chapter 32

Diffusion Models: Mathematical Foundations

32.1 Introduction

We now formalize diffusion models as probabilistic processes. The key idea is to define a forward Markov chain that gradually corrupts data into noise, and then learn a reverse process that reconstructs data.

This chapter derives the forward and reverse processes, shows how training objectives arise, and connects diffusion to score matching and stochastic differential equations.

32.2 Discrete-Time Diffusion Formulation

We define a forward Markov chain:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}),$$

where each transition adds Gaussian noise.

32.2.1 Forward Transitions

[Forward Process]

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t \;\middle|\; \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right),$$

where $\beta_t \in (0,1)$ is a noise schedule.

This defines a Markov chain that gradually destroys structure.

32.3 Forward Process: Closed-Form Marginals

A key property is that x_t can be sampled directly from x_0.

[Closed-Form Marginal]

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t \;\middle|\; \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)I\right),$$

where:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

32.3.1 Derivation Sketch

Repeated substitution gives:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Thus:

- mean shrinks toward zero,
- variance increases over time.

32.4 Reverse Process Parameterization

We define a learned reverse chain:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t),$$

with:

$$p(x_T) = \mathcal{N}(0, I).$$

32.4.1 Gaussian Parameterization

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1} \mid \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)).$$

The model learns to predict the mean (and optionally variance) of the reverse transition.

32.5 Training Objectives: ELBO View

Training maximizes log-likelihood:

$$\log p_\theta(x_0).$$

Using variational inference, we obtain an ELBO:

$$\mathbb{E}_q\left[\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0)\right].$$

This decomposes into terms comparing forward and reverse transitions.

32.5.1 Simplified Objective

After algebraic simplification, training reduces to:

$$\mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right],$$

where:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon.$$

32.5.2 Interpretation

- model predicts noise ϵ,
- equivalent to learning reverse dynamics,
- yields stable supervised learning objective.

32.6 Training Objective Transformations

Different parameterizations are equivalent:

- predicting noise ϵ,
- predicting x_0,
- predicting score $\nabla_x \log p(x_t)$.

These correspond to different but related loss formulations.

32.7 Score Matching and SDE View

Diffusion models are closely related to score matching.

32.7.1 Score Function

$$s(x) = \nabla_x \log p(x).$$

The model learns an approximation:

$$s_\theta(x_t, t).$$

32.7.2 Connection

Noise prediction is proportional to the score:

$$\epsilon_\theta(x_t, t) \iff -\nabla_x \log q(x_t).$$

Thus diffusion training implicitly performs score matching.

32.7.3 Continuous-Time View

As $T \to \infty$, the process becomes a stochastic differential equation (SDE):

$$dx = f(x,t)\,dt + g(t)\,dW_t.$$

The reverse process is another SDE that removes noise.

32.8 Langevin Dynamics Connection

Sampling can be interpreted as approximate Langevin dynamics:

$$x_{t-1} = x_t + \eta \nabla_x \log p(x_t) + \sqrt{2\eta}\,\xi.$$

- gradient term moves toward high-density regions,
- noise maintains stochasticity,
- diffusion combines both ideas.

32.9 DDPM, DDIM, and Sampler Intuition

32.9.1 DDPM (Stochastic Sampling)

- follows learned reverse transitions,
- includes stochastic noise at each step,
- produces diverse samples.

32.9.2 DDIM (Deterministic Sampling)

- removes stochasticity,
- produces deterministic trajectories,
- faster sampling with fewer steps.

32.9.3 Tradeoffs

- stochastic samplers: more diversity,
- deterministic samplers: faster and more controllable.

32.10 A Unifying Perspective

Diffusion models define:

- a forward stochastic process that simplifies data,
- a reverse learned process that reconstructs it.

Training can be viewed as:

- variational inference (ELBO),
- score matching,
- learning dynamics of a stochastic system.

Thus diffusion sits at the intersection of probability, optimization, and dynamical systems.

32.11 Summary

In this chapter we formalized diffusion models. We defined the forward Markov process, derived closed-form marginals, and introduced the reverse generative model. We showed how training arises from a variational objective and reduces to noise prediction.

We also connected diffusion to score matching, stochastic differential equations, and Langevin dynamics, and discussed different sampling schemes such as DDPM and DDIM.

Exercises

1. Derive the mean and variance of $q(x_t \mid x_0)$.
2. Show how x_t can be written as a linear combination of x_0 and Gaussian noise.
3. Derive the simplified noise-prediction objective from the ELBO.
4. Explain why predicting ϵ is equivalent to learning the score function.
5. Compare stochastic (DDPM) and deterministic (DDIM) sampling.
6. Show how variance accumulates over diffusion steps.
7. Derive the relationship between score matching and denoising.
8. Explain the role of the noise schedule $\{\beta_t\}$.

Chapter 33

Generalization in Deep Learning

33.1 Why Classical Intuition Is Strained in Deep Learning

Classical learning theory often suggests a simple picture: as model complexity increases, the risk of overfitting should increase unless complexity is carefully controlled. In particular, if a model has far more parameters than training examples, one might expect it to memorize the training data and fail on unseen inputs.

Modern deep learning challenges this picture.

Empirical tension.

- Deep neural networks are often highly overparameterized
- They can frequently drive the training error to zero
- Yet they often achieve strong test performance

This creates a central puzzle: why does interpolation of the training data not necessarily destroy generalization?

Classical viewpoint. In traditional settings, one often studies a nested family of hypothesis classes

$$\mathcal{H}_1 \subset \mathcal{H}_2 \subset \cdots$$

and balances empirical fit against class complexity. The rough intuition is:

- too little capacity leads to underfitting,
- too much capacity leads to overfitting.

This intuition is valuable, but in deep learning it is incomplete.

Why it becomes strained.

- Parameter count is often much larger than sample size
- Many distinct parameter settings fit the data exactly
- Optimization methods do not search these solutions uniformly
- Real data possess structure that is not captured by worst-case analyses

Thus, the question is no longer only *"How large is the hypothesis class?"* but also *"Which interpolating solution is actually found by training, and why?"*

A shift of perspective. In overparameterized learning, one often does not select among models of different expressive power. Instead, one selects among many solutions inside an already expressive model class.

Guiding question. What properties of the learned solution—rather than mere parameter count—help explain good generalization?

33.2 Interpolation and Benign Overfitting

A model is said to be in the *interpolation regime* when it can fit the training data exactly. If the training sample is

$$S = \{(x_i, y_i)\}_{i=1}^n,$$

and the predictor is f_θ, then interpolation means

$$f_\theta(x_i) = y_i \quad \text{for all } i = 1, \dots, n,$$

or equivalently, for a suitable loss,

$$\hat{R}_n(\theta) = 0.$$

33.2.1 Interpolation

In classical statistics, interpolation was often viewed with suspicion. Exact fitting was associated with variance explosion and poor predictive performance. In modern machine learning, however, interpolation is common.

Observation.

- Large neural networks often interpolate the training set
- Kernel methods can also interpolate
- Some interpolating solutions generalize surprisingly well

This motivates the notion of *benign overfitting*.

33.2.2 Benign Overfitting

Benign overfitting refers to a situation in which a predictor fits the training data exactly, possibly even fitting noise, yet still achieves low test error.

This phenomenon is especially visible in high-dimensional linear models and kernel methods, and it has influenced how researchers think about deep networks.

Important nuance. Interpolation alone is not enough. Some interpolating solutions generalize well, while others generalize poorly. The key issue is therefore not whether interpolation occurs, but *which* interpolating solution is selected.

33.2.3 Overparameterization

A model is overparameterized when the number of free parameters is sufficiently large relative to the sample size that many exact fits are possible.

Consequences of overparameterization.

- There may be infinitely many zero-training-error solutions
- Optimization becomes a problem of solution selection
- The geometry of the parameter space matters
- Initialization and training dynamics become central

Overparameterization does not automatically imply good generalization. Rather, it creates a regime in which good and bad interpolating solutions may coexist.

33.2.4 A Linear Illustration

Consider an underdetermined linear system

$$Xw = y,$$

where $X \in \mathbb{R}^{n \times d}$ with $d > n$. Then there are typically many vectors w satisfying the interpolation condition.

Among these, gradient descent initialized at the origin converges to the minimum Euclidean norm solution:

$$w_* = \arg\min_w \|w\|_2 \quad \text{subject to } Xw = y.$$

This already illustrates an important modern lesson:

> Even when exact fitting is possible, the optimization algorithm may favor a special interpolating solution with additional structure.

In deep learning, the corresponding picture is more complicated, but the same principle remains central.

33.3 Norms, Margins, and Implicit Bias

If parameter count does not by itself explain generalization, what should replace it? Several influential perspectives emphasize *effective complexity* rather than raw dimensionality.

33.3.1 Implicit Bias of Optimization

Implicit bias is the tendency of a training algorithm to prefer certain solutions over others, even without explicit regularization.

This is crucial in overparameterized settings because there are often many global minima of the training objective.

Examples of factors affecting implicit bias.

- gradient descent versus stochastic gradient descent,
- learning rate schedule,
- initialization scale,
- architecture and parametrization,
- normalization layers and residual structure.

The optimizer does not merely *find* a solution; it helps *define which solution is found*.

33.3.2 Norm-Based Complexity Sketch

One recurring idea is that generalization is better explained by suitable norms of the learned predictor than by the number of parameters.

Norm-based intuition. A predictor with smaller norm is often less sensitive, less oscillatory, or more stable. In linear prediction,

$$f_w(x) = \langle w, x \rangle,$$

one expects that controlling $\|w\|$ can help control the capacity of the predictor.

A rough complexity sketch takes the form

$$R(f) \;\leq\; \hat{R}_n(f) + \text{complexity term},$$

where the complexity term may depend on a norm of the parameters or of the function. For example, one may informally think of bounds of the flavor

$$R(f) \;\lesssim\; \hat{R}_n(f) + \frac{\|w\|}{\sqrt{n}},$$

or, in more refined analyses, on products of layer norms, path norms, spectral norms, or other scale-sensitive quantities.

Important warning. Such expressions are schematic, not universal formulas. In deep networks, the appropriate notion of norm is subtle because different parametrizations can represent the same function with different parameter magnitudes.

Still, the main message is clear:

> Generalization may depend more on the *size of the learned solution in a suitable geometry* than on the number of trainable parameters.

33.3.3 Margin-Based Intuition

Margins are especially important in classification.

Suppose labels satisfy $y_i \in \{-1, +1\}$, and the classifier is given by the sign of a score function f. The *margin* at a sample (x_i, y_i) is

$$y_i f(x_i).$$

A positive and large value means that the example is not only classified correctly, but classified with confidence.

Margin-based intuition. Two classifiers may make the same training predictions, but the one that separates the data with larger margin is often expected to generalize better.

For linear classifiers,

$$f_w(x) = \langle w, x \rangle,$$

the geometric margin is related to

$$\frac{y_i \langle w, x_i \rangle}{\|w\|}.$$

Thus, good generalization is connected not merely to fitting the labels, but to doing so with a favorable ratio of margin to norm.

Formal intuition. Classical large-margin results suggest that a classifier can generalize well when:

- training error is small,
- margins are large,
- effective norm or complexity is controlled.

This helps explain why zero training error need not be problematic: what matters is not only interpolation, but the *quality* of the interpolating separator.

33.3.4 Compression Perspective

Another influential viewpoint is that learned predictors generalize when they admit a compressed description.

Compression idea. If a trained network can be represented, approximated, or specified using fewer effective degrees of freedom than its raw parameter count suggests, then its true complexity may be smaller than the ambient dimension indicates.

Compression may appear through:

- sparse representations,
- low-rank structure,
- quantization,
- pruning,
- reuse of shared features.

The intuition is that predictors with shorter effective descriptions are less likely to have fit arbitrary noise in a fragile way.

33.3.5 Flatness and Sharpness

A related empirical idea concerns the geometry of minima.

Flat minima. A minimum is called *flat* if the loss remains small under small perturbations of the parameters.

Sharp minima. A minimum is *sharp* if small perturbations cause the loss to increase quickly.

Researchers have often suggested that flatter minima generalize better because they are more robust. While this idea is appealing, it must be handled carefully:

- flatness can depend on parametrization,
- some naive measures of sharpness are not invariant,
- the relation to test error is suggestive but not fully definitive.

Thus flatness is best viewed as one useful perspective among several, not as a complete theory.

33.4 Double Descent

The classical bias–variance tradeoff predicts a U-shaped test error curve as model complexity increases:

- high bias for simple models,
- high variance for overly complex models.

Modern practice often reveals a richer pattern called *double descent*.

33.4.1 Phenomenon

Double descent refers to the observation that as model size increases, test error may:

- first decrease,
- then increase near the interpolation threshold,
- then decrease again in the highly overparameterized regime.

This produces a curve with two descents rather than one.

33.4.2 Interpolation Threshold

The interpolation threshold is the regime where the model becomes just expressive enough to fit the training data exactly. Near this threshold, predictions can become unstable, and test error may peak.

Interpretation. Near interpolation, the model may fit the data in a brittle way. With still greater overparameterization, optimization may gain access to interpolating solutions with more favorable norm, margin, or stability properties.

33.4.3 Why This Matters

Double descent shows that classical monotonic notions of complexity are insufficient.

Key lesson. Increasing the number of parameters does not always worsen generalization. In some regimes, larger models are easier to optimize into "good" interpolating solutions.

This has been observed in:

- neural networks,
- random feature models,
- kernel methods,

- linear regression in high dimensions.

The phenomenon strengthens the modern view that effective complexity depends on the learned solution and training dynamics, not only on nominal model size.

33.5 What Current Theory Explains and What Remains Open

The theory of generalization in deep learning has advanced significantly, but it remains incomplete.

33.5.1 What Current Theory Helps Explain

Several broad ideas now have substantial theoretical or empirical support.

1. Overparameterization need not destroy generalization. The existence of many interpolating solutions means that fitting the training data exactly is not itself fatal.

2. Optimization has implicit bias. Training algorithms do not sample all solutions uniformly; they prefer certain types of solutions.

3. Effective complexity matters more than raw parameter count. Norms, margins, compression, and stability often provide better explanatory tools than the number of parameters alone.

4. Data structure matters. Natural data are highly non-generic. Real-world inputs have symmetries, low-dimensional structure, hierarchical organization, and semantic regularities that are absent from worst-case analyses.

33.5.2 Careful Statement of Unresolved Issues

Despite progress, several central questions remain open.

Unresolved issue 1: A complete predictive theory is missing. There is no single theory that reliably predicts the generalization performance of modern deep networks across architectures, datasets, and optimizers.

Unresolved issue 2: Implicit bias in deep nonlinear models is not fully characterized. For linear models, one can often identify the exact bias of gradient descent. For deep nonlinear networks, the corresponding characterization is far more delicate.

Unresolved issue 3: The right complexity measure is unclear. Competing perspectives emphasize:

- norms,

- margins,
- compression,
- flatness,
- stability,
- information-theoretic quantities.

No single measure has emerged as universally satisfactory.

Unresolved issue 4: Existing worst-case bounds are often loose. Many rigorous bounds for deep networks are mathematically valid but numerically too weak to explain observed performance.

Unresolved issue 5: Optimization and generalization are deeply entangled. A theory that cleanly separates approximation, optimization, and statistical generalization may be impossible in modern deep learning; these aspects appear strongly coupled.

33.5.3 Connections to Broader Theory

Current research draws tools from several areas:

- **Statistical learning theory:** generalization bounds, margins, complexity measures
- **Optimization theory:** gradient flow, stochastic effects, landscape geometry
- **Information theory:** compression, representation, mutual information viewpoints
- **Statistical physics:** energy landscapes, phase transitions, high-dimensional effects
- **Approximation theory:** expressive power and inductive biases of architectures

The field remains active precisely because deep learning sits at the intersection of all these perspectives.

33.6 A Unifying Perspective

Generalization in deep learning appears to arise from the interaction of several ingredients:

- **Expressive capacity:** neural networks can represent highly complex functions

- **Overparameterization:** many interpolating solutions may exist
- **Optimization dynamics:** training selects some solutions rather than others
- **Implicit regularization:** gradient-based methods favor structured solutions
- **Data geometry:** real datasets contain patterns that can be exploited

Core message. Generalization is not governed by parameter count alone. It emerges from the interplay of representation, optimization, and data structure.

Modern viewpoint. Deep learning does not invalidate classical theory; rather, it reveals that classical theory captures only part of the story. New regimes require new complexity notions and a closer study of training dynamics.

33.7 Outlook

This chapter has emphasized several central ideas:

- why classical intuition becomes strained in deep learning,
- why interpolation can coexist with good test performance,
- how overparameterization changes the learning problem,
- how norms, margins, compression, and implicit bias help explain generalization,
- why double descent reveals a richer complexity picture,
- what current theory explains and what remains unresolved.

Guiding principle. To understand generalization in modern machine learning, one must study not only what functions a model *can* represent, but also what solutions training dynamics *prefer* and how those solutions interact with the structure of data.

Exercises

1. **Classical versus deep-learning intuition.** Explain why the statement

   "more parameters always implies worse generalization"

   is too simplistic in modern deep learning. Give at least three reasons.

2. **Interpolation regime.** Let f_θ be a predictor trained on a dataset $S = \{(x_i, y_i)\}_{i=1}^n$. What does it mean mathematically for f_θ to interpolate the data? Why does interpolation not by itself determine test performance?

3. **Underdetermined linear regression.** Consider $Xw = y$ with $X \in \mathbb{R}^{n \times d}$ and $d > n$. Why are there generally many interpolating solutions? State the optimization problem solved by gradient descent initialized at zero.

4. **Norm-based perspective.** Give a conceptual explanation of why a small-norm solution may generalize better than a large-norm solution, even if both achieve zero training error.

5. **Margin intuition.** For binary classification with labels $y_i \in \{-1, +1\}$, define the margin $y_i f(x_i)$. Why is a classifier with larger margins often expected to generalize better?

6. **Compression viewpoint.** Describe the compression perspective on generalization. Why might pruning or quantization reveal that a trained network has lower effective complexity than its raw parameter count suggests?

7. **Double descent.** Sketch the qualitative shape of the double descent curve. Identify:

   - the underparameterized regime,
   - the interpolation threshold,
   - the highly overparameterized regime.

   Explain why the test error may decrease again after the interpolation peak.

8. **Benign overfitting.** What is meant by benign overfitting? In what sense does it challenge classical intuitions about exact fitting?

9. **Comparing regimes.** Compare the classical bias–variance picture with the modern deep-learning picture. Which aspects remain useful, and which aspects become insufficient?

10. **Open problems.** Choose two unresolved issues from this chapter and explain why each is difficult. What kind of mathematical tools might help address them?

Chapter 34

Geometry and Dynamics of Learning

34.1 Parameter Space and Function Space

Training a neural network can be viewed from two closely related perspectives:

- **parameter space:** the space of weights and biases,
- **function space:** the space of input–output maps realized by the network.

If the network is parameterized by $\theta \in \mathbb{R}^d$, then training minimizes a loss

$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(x_i), y_i\big),$$

where f_θ is the function represented by the network.

Parameter-space view.

- A point θ represents one specific choice of parameters
- Optimization moves through a high-dimensional landscape $L(\theta)$

Function-space view.

- The object of real interest is not the parameter vector itself, but the function f_θ
- Different parameter vectors may represent similar or even identical functions

This distinction is important because neural networks are often highly redundant:

- permutations of hidden units can leave the realized function unchanged,
- rescalings in some architectures can produce equivalent predictions,
- large motions in parameter space can correspond to small motions in function space.

Insight. Optimization takes place in parameter space, but generalization is about the learned function. Much of modern theory studies how parameter dynamics induce trajectories in function space.

34.2 Loss Landscapes

The loss function defines a high-dimensional landscape over parameter space:

$$L(\theta) : \mathbb{R}^d \to \mathbb{R}.$$

Landscape interpretation.

- each point corresponds to a parameter configuration,
- the height of the landscape is the loss value,
- learning corresponds to moving downhill.

Although this picture is easy to draw in low dimensions, the true loss landscape of a neural network is extremely high-dimensional and far more intricate.

Critical points. A point θ^* is critical if

$$\nabla L(\theta^*) = 0.$$

Among critical points, one distinguishes:

- **local minima:** directions of nonnegative curvature,
- **local maxima:** directions of nonpositive curvature,
- **saddle points:** some directions curve upward and others downward.

In high dimensions, saddle points are often more prevalent than poor isolated local minima. This helps explain why optimization difficulties in deep learning are not well described simply as "getting stuck in bad local minima."

Flat and sharp regions.

- **Flat regions** have small curvature in many directions
- **Sharp regions** have large curvature, so loss changes rapidly under perturbations

This matters because optimization speed, stability, and sometimes generalization behavior are strongly influenced by local geometry.

Caution. In neural networks, geometric notions such as flatness can depend on parametrization, so they must be interpreted carefully. Still, the landscape picture remains a useful organizing idea.

34.3 Curvature, Hessians, and Optimization Geometry

To move beyond first-order descent intuition, one studies curvature.

34.3.1 Hessian Matrix

The local second-order geometry of the loss is captured by the Hessian:

$$H(\theta) = \nabla^2 L(\theta).$$

Interpretation of the Hessian.

- it measures how the gradient changes locally,
- it describes curvature in different directions,
- it helps distinguish minima, saddles, and flat directions.

For a direction $v \in \mathbb{R}^d$, the quadratic approximation gives

$$L(\theta + v) \approx L(\theta) + \nabla L(\theta)^\top v + \frac{1}{2} v^\top H(\theta) v.$$

Thus the quantity

$$v^\top H(\theta) v$$

describes the curvature of the loss in direction v.

34.3.2 Eigenvalue Intuition

The eigenvalues of the Hessian give local curvature scales.

- Large positive eigenvalues indicate steep upward curvature
- Small eigenvalues indicate nearly flat directions
- Negative eigenvalues indicate descent directions and hence saddle-like structure

Hessian intuition. Neural-network losses often exhibit:

- a few directions of strong curvature,
- many weakly curved or nearly flat directions,
- occasional negative-curvature directions away from minima.

This suggests that optimization takes place in a highly anisotropic geometry rather than an isotropic bowl.

34.3.3 Curvature and Optimization

Curvature matters because gradient descent behaves differently in different directions.

If one applies the update

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k),$$

then large curvature directions constrain the step size η. If η is too large relative to local curvature, the iteration can oscillate or diverge.

Consequences of curvature.

- steep directions slow stable optimization,
- flat directions lead to slow progress,
- anisotropy motivates momentum, preconditioning, and adaptive methods.

Second-order methods explicitly try to exploit Hessian information, but in large neural networks exact Hessian computations are often too expensive. Still, Hessian-based intuition is central even when one uses only first-order optimizers.

34.3.4 Geometry Beyond the Hessian

The Hessian is a local object. Deep learning also involves global geometric features:

- connected low-loss valleys,
- broad basins,
- ridges and plateaus,
- symmetries and degeneracies of parametrization.

Thus optimization geometry is not merely about one critical point, but about the structure of large low-loss regions and the trajectories connecting them.

34.4 Gradient Flow and Dynamical Systems

Optimization can be studied not only as an iterative algorithm but also as a dynamical system.

34.4.1 From Gradient Descent to Gradient Flow

Discrete-time gradient descent takes the form

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k).$$

When the step size η is small, one is led to the continuous-time limit

$$\frac{d\theta(t)}{dt} = -\nabla L(\theta(t)),$$

called **gradient flow**.

Interpretation. The parameter vector evolves in the direction of steepest instantaneous decrease of the loss.

34.4.2 Energy Dissipation

Along a gradient-flow trajectory, the loss decreases monotonically:

$$\frac{d}{dt} L(\theta(t)) = \nabla L(\theta(t))^\top \frac{d\theta(t)}{dt} = -\|\nabla L(\theta(t))\|^2 \leq 0.$$

This identity is fundamental. It shows that gradient flow is a dissipative dynamical system: the loss acts like an energy that decreases over time.

34.4.3 Fixed Points and Stability

A fixed point of gradient flow satisfies

$$\frac{d\theta(t)}{dt} = 0,$$

so necessarily

$$\nabla L(\theta) = 0.$$

Thus critical points of the loss correspond to equilibria of the flow.

Their stability depends on curvature:

- minima are locally attracting under suitable conditions,
- saddle points are unstable in directions of negative curvature,
- flat directions can produce slow or non-isolated behavior.

This dynamical-systems viewpoint explains why negative curvature helps optimization escape many saddles.

34.4.4 Stochastic Dynamics

Stochastic gradient descent introduces noise because gradients are estimated from minibatches. At a high level, one models this by a stochastic differential equation

$$d\theta(t) = -\nabla L(\theta(t))\,dt + \Sigma(\theta(t))^{1/2}\,dW_t,$$

where W_t is Brownian motion and Σ reflects gradient noise.

Interpretation. Training is no longer a deterministic flow, but a noisy dynamical process. This noise can:

- help escape unstable regions,
- bias trajectories toward some minima rather than others,
- influence both optimization and generalization.

Hence training dynamics are shaped by both geometry and stochasticity.

34.5 Feature Learning vs Kernel-Like Regimes

A central modern question is whether training mainly learns new features or behaves approximately like a linear model around initialization.

34.5.1 Linearization Around Initialization

Let θ_0 denote the initialization. For parameters θ near θ_0, one may linearize the network:

$$f_\theta(x) \approx f_{\theta_0}(x) + \nabla_\theta f_{\theta_0}(x)^\top(\theta - \theta_0).$$

This approximation treats the model as locally linear in parameters. If training remains close to initialization, the behavior of the network may be well approximated by this linearized model.

34.5.2 Neural Tangent Kernel

The associated kernel is the **neural tangent kernel** (NTK):

$$K(x, x') = \nabla_\theta f_{\theta_0}(x)^\top \nabla_\theta f_{\theta_0}(x').$$

High-level meaning of the NTK.

- it converts the network into a kernel method at initialization,
- in suitable wide-network regimes, training becomes approximately linear,
- predictions evolve similarly to kernel regression with kernel K.

In the infinite-width limit, this kernel often becomes deterministic, and the dynamics simplify considerably. This regime is sometimes called **lazy training**, because the parameters move little and the network behaves almost like its first-order Taylor expansion.

34.5.3 Kernel-Like Regime

In a kernel-like or lazy regime:

- features do not change much during training,
- learning happens mainly through reweighting fixed random features,
- analysis is simpler and often mathematically tractable.

This regime is important because it connects deep learning to classical kernel methods and yields rigorous results in some asymptotic settings.

34.5.4 Feature-Learning Regime

In practical deep learning, however, networks often do more than linearized fitting.

Feature learning means that internal representations evolve significantly during training. Hidden layers reshape the geometry of the data so that later layers can solve the task more easily.

In this regime:

- the tangent features themselves change,
- the network moves far enough from initialization that linearization becomes incomplete,
- representation learning becomes central to performance.

Key contrast.

- **kernel-like view:** learning acts on nearly fixed features,
- **feature-learning view:** learning changes the features themselves.

A major theme of modern theory is understanding when each regime dominates and how they interact.

34.6 Representation Geometry in Trained Networks

Deep learning is not only about reducing loss. It also changes the internal geometry of data representations.

34.6.1 Representation as Geometry

For an intermediate layer $h_\theta(x)$, inputs are mapped into a feature space. Training changes this map, and hence changes the geometry of the dataset in representation space.

Representation geometry asks:

- which examples become closer or farther apart,
- which directions are expanded or compressed,
- how class structure becomes more separable,
- how invariances are formed.

34.6.2 Separation and Flattening

Empirically, trained networks often transform data so that:

- examples from the same class cluster more tightly,
- examples from different classes become more separated,
- nuisance variations are suppressed,
- task-relevant directions are amplified.

This can be viewed as a geometric simplification of the learning problem.

34.6.3 Feature Learning Geometry

Feature learning geometry refers to how the network reshapes the data manifold during training.

At a high level, one may think of hidden layers as performing successive geometric operations:

- disentangling factors of variation,
- flattening or straightening class manifolds,
- aligning representations with decision boundaries,
- creating coordinates in which simple classifiers work well.

This viewpoint helps explain why deep networks can outperform fixed-feature methods: they do not merely fit outputs; they actively reorganize the input geometry.

34.6.4 Residual Networks and Dynamical Depth

Residual networks provide an especially suggestive bridge between geometry and dynamics. A residual block has the form

$$h_{k+1} = h_k + \Phi_k(h_k),$$

which resembles a discretized differential equation.

Thus depth can be interpreted as time in a representation flow. Under this view:

- layers evolve features gradually,
- deeper models implement longer geometric transformations,
- representation learning becomes a controlled dynamical process.

This is one reason dynamical-systems language is so natural in modern deep learning theory.

34.7 A Unifying Perspective

The geometry and dynamics of learning can be organized around several complementary viewpoints:

- **loss landscapes:** the global and local structure of $L(\theta)$,
- **curvature and Hessians:** second-order information shaping optimization,
- **gradient flow:** continuous-time evolution of parameters,
- **kernel-like regimes:** linearized training near initialization,
- **feature learning:** task-dependent evolution of representations,
- **representation geometry:** how the network restructures data internally.

Core idea. Understanding deep learning requires studying both where optimization moves in parameter space and how those motions change the function and representation geometry seen by the data.

Key insight. Modern learning theory is not only about approximation or statistics. It is also about geometry, dynamics, and the evolving structure of learned representations.

34.8 Outlook

This chapter developed a richer picture of learning by combining geometric and dynamical viewpoints:

- parameter space versus function space,
- loss landscapes and high-dimensional critical structure,
- curvature, Hessian intuition, and optimization geometry,
- continuous-time gradient flow and stochastic dynamics,
- NTK and lazy-training perspectives,
- feature learning and representation geometry.

Guiding principle. Learning is not merely the search for low loss. It is a structured dynamical process that moves through parameter space while reshaping the geometry of functions and representations.

Exercises

1. **Parameter space and function space.** Explain why two very different parameter vectors θ and θ' may correspond to similar functions f_θ and $f_{\theta'}$. Give at least two reasons this can happen in neural networks.

2. **Quadratic approximation.** Starting from the second-order Taylor approximation

$$L(\theta + v) \approx L(\theta) + \nabla L(\theta)^\top v + \frac{1}{2} v^\top H(\theta) v,$$

explain the meaning of the Hessian term. What does it tell us about curvature in the direction v?

3. **Eigenvalue interpretation.** Suppose the Hessian at a point has one negative eigenvalue, several large positive eigenvalues, and many near-zero eigenvalues. What does this suggest about the local geometry of the loss?

4. **Simple gradient flow.** Consider the one-dimensional loss

$$L(\theta) = \frac{1}{2}\theta^2.$$

Write the gradient-flow equation and solve it explicitly. Show that the loss decreases monotonically along the trajectory.

5. **Another gradient flow.** Consider

$$L(\theta) = \frac{1}{4}\theta^4.$$

Write the gradient-flow equation. Compare qualitatively its convergence near $\theta = 0$ with the quadratic case.

6. **Fixed points and stability.** Let gradient flow satisfy

$$\frac{d\theta}{dt} = -\nabla L(\theta).$$

Why are critical points of L fixed points of the flow? How does curvature help distinguish stable equilibria from unstable ones?

7. **Linearized training.** Derive the first-order Taylor approximation

$$f_\theta(x) \approx f_{\theta_0}(x) + \nabla_\theta f_{\theta_0}(x)^\top(\theta - \theta_0).$$

In what sense does this approximation lead to a kernel method?

8. **NTK versus feature learning.** Compare the following two views:

   - training as kernel-like regression with nearly fixed features,
   - training as feature learning in which internal representations evolve.

   What kinds of behavior can the second view explain that the first may miss?

9. **Representation geometry.** Describe how training might change the geometry of hidden representations in a classification network. What does it mean for class manifolds to become more separable?

10. **Residual networks and dynamics.** Consider the residual update

$$h_{k+1} = h_k + \Phi_k(h_k).$$

Explain why this resembles a discretized differential equation. What insight does this give into the role of depth?

11. **Flat and sharp regions.** Why might flatness be related to robustness of a minimum? Why should one nevertheless be cautious about using flatness as a definitive explanation of generalization?

12. **Synthesis.** Write a short essay comparing two pictures of deep learning:

   - the optimization of a complicated loss landscape in parameter space,
   - the gradual evolution of useful representations in function space.

   Explain why both pictures are needed.

Chapter 35

Practical System Design

35.1 Framing ML as a Systems Problem

Machine learning is often presented as a problem of choosing a model and minimizing a loss. In practice, however, useful machine learning systems are much larger than the model itself. They involve data collection, labeling, storage, training infrastructure, evaluation, deployment, monitoring, and continual maintenance.

Core viewpoint. A machine learning system is not just a function

$$x \mapsto \hat{y},$$

but a pipeline that connects:

- raw data,
- human decisions and labels,
- training and validation workflows,
- deployed inference services,
- monitoring and feedback loops.

Why this matters. A theoretically strong model can still fail in practice if:

- the data pipeline is unreliable,
- the evaluation setup is flawed,
- the system violates latency or memory constraints,
- deployment conditions differ from training conditions,
- failures are not detected after launch.

Thus practical machine learning must be framed as a *systems problem*, not merely an optimization problem.

Main system components.

- **Data infrastructure:** collection, storage, cleaning, labeling, versioning
- **Training infrastructure:** preprocessing, batching, training, validation, checkpointing
- **Experiment infrastructure:** metrics, logs, hyperparameters, comparisons
- **Serving infrastructure:** model packaging, inference, latency, scaling
- **Monitoring infrastructure:** drift detection, error analysis, feedback collection

Insight. In real-world ML, system quality is determined by the weakest link in the pipeline, not only by the sophistication of the learning algorithm.

35.2 Data Collection, Cleaning, and Labeling

Data is the foundation of a machine learning system. In many applications, the quality of the dataset matters more than small differences between model architectures.

35.2.1 Data Collection

The first step is to gather data relevant to the task.

Typical sources.

- sensors and devices,
- user interactions,
- logs and transaction records,
- public datasets,
- manually curated examples.

Key concerns.

- Does the data represent the target population?
- Does it cover the range of situations the model will face?
- Are rare but important cases included?

A common failure is *sampling mismatch*: the collected data is easier, cleaner, or less diverse than real deployment data.

35.2.2 Cleaning and Preprocessing

Raw data is rarely ready for immediate use. Practical systems require systematic preprocessing.

Examples.

- removing duplicates,
- handling missing values,
- correcting corrupted records,
- normalizing units and formats,
- standardizing text, images, or numerical features.

Feature preprocessing. Depending on the modality, one may apply:

- normalization or scaling,
- tokenization,
- image resizing and augmentation,
- categorical encoding,
- filtering and denoising.

Pipeline principle. Preprocessing should be automated and reproducible. Ad hoc manual transformations create hidden inconsistencies between training and deployment.

35.2.3 Labeling

Supervised learning depends on labels, and labeling quality is often a major bottleneck.

Label sources.

- expert annotation,
- crowd annotation,
- user-provided outcomes,
- weak or heuristic labels,
- labels inferred from downstream events.

Common labeling issues.

- inconsistent annotation standards,

- ambiguous examples,
- class imbalance,
- noisy labels,
- delayed or missing ground truth.

Because models learn the structure of labels, poor labeling policy often leads to systematic model failure.

35.2.4 Train, Validation, and Test Infrastructure

A robust system requires careful separation of datasets.

Training set. Used to fit model parameters.

Validation set. Used to tune hyperparameters, choose architectures, and make model-selection decisions.

Test set. Used only for final evaluation. It should remain isolated from repeated experimentation.

Why the split matters. If the validation or test data leaks into training decisions, performance estimates become overly optimistic.

Practical infrastructure concerns.

- prevent duplicate or near-duplicate examples across splits,
- ensure time-based splits when future prediction is the goal,
- preserve group boundaries when examples are correlated,
- version the exact train/val/test partitions.

A train/validation/test split is not only a statistical idea but also an engineering artifact that must be stored, reproduced, and audited.

35.2.5 Data Pipelines

A **data pipeline** transforms raw data into model-ready inputs and labels.

A concrete pipeline may include:

1. ingest raw records,
2. validate schema and basic integrity,
3. clean and standardize values,
4. join multiple sources,

5. generate features or encodings,
6. write processed data to storage for training or serving.

Batch versus streaming pipelines.

- **Batch pipelines** process data periodically in large jobs
- **Streaming pipelines** process examples continuously as they arrive

The correct choice depends on the application. Offline recommendation retraining may tolerate batch updates, while fraud detection may require continuous ingestion.

Insight. Reliable data pipelines are essential because every downstream stage depends on their correctness.

35.3 Training Pipelines and Experiment Management

Training a model in practice involves much more than running gradient descent. One needs infrastructure for repeatable experiments, efficient compute use, and trustworthy comparisons.

35.3.1 Training Pipelines

A training pipeline coordinates data loading, optimization, evaluation, and checkpointing.

Typical stages.

1. read processed training data,
2. construct batches,
3. compute forward pass,
4. evaluate loss,
5. compute gradients,
6. update parameters,
7. periodically evaluate on validation data,
8. save checkpoints and logs.

Operational requirements.

- training must resume after interruption,

- failed jobs should be recoverable,
- metrics should be stored continuously,
- model artifacts should be versioned.

35.3.2 Experiment Tracking

Modern model development requires many runs with different hyperparameters, architectures, and preprocessing choices.

Experiment tracking should record:

- code version,
- dataset version,
- train/val/test split,
- hyperparameters,
- optimizer settings,
- metrics and curves,
- final model artifact.

Without systematic experiment tracking, it becomes difficult to answer basic questions such as:

- Which change improved performance?
- Which dataset version was used?
- Can this result be reproduced?

Practical principle. Every important result should be traceable to a specific configuration of data, code, and compute.

35.3.3 Hyperparameter Search

Training quality often depends strongly on hyperparameters such as:

- learning rate,
- regularization strength,
- batch size,
- architecture depth or width,
- augmentation policy.

Common strategies include:

- grid search,
- random search,
- adaptive methods such as Bayesian optimization,
- multi-fidelity methods with early stopping.

Hyperparameter tuning is itself a systems task because it requires scheduler logic, job parallelism, budget control, and fair comparison criteria.

35.3.4 Checkpointing and Recovery

Long training jobs require checkpointing.

Why checkpointing matters.

- it prevents loss of progress,
- it enables later evaluation or fine-tuning,
- it allows rollback to earlier model states,
- it supports comparison of intermediate learning stages.

A checkpoint commonly stores:

- model parameters,
- optimizer state,
- scheduler state,
- training step or epoch count,
- metadata about the run.

35.3.5 Distributed Training

Large-scale models may exceed the capacity of a single device.

Two major patterns.

- **Data parallelism:** replicate the model across devices and split batches
- **Model parallelism:** split the model itself across devices

These choices affect throughput, communication cost, memory use, and engineering complexity.

Insight. Efficient training pipelines make experimentation faster, more reliable, and easier to interpret.

35.4 Serving, Inference, and Latency Constraints

A trained model is only useful if it can be deployed in a form compatible with real-world constraints.

35.4.1 Serving

Serving is the process of making model predictions available to downstream users or applications.

Deployment may occur in several ways:

- **online serving:** predictions are generated on request,
- **batch inference:** predictions are computed periodically for many items,
- **on-device inference:** the model runs locally on edge hardware.

Each setting imposes different requirements on memory, throughput, latency, and reliability.

35.4.2 Inference Constraints

At training time one often optimizes predictive accuracy. At serving time, additional constraints become central:

- inference latency,
- throughput,
- memory footprint,
- power consumption,
- hardware compatibility,
- cost per prediction.

A model with slightly better accuracy may be impractical if it is too slow or too expensive to serve.

35.4.3 Latency

Inference latency is the time required to produce a prediction after an input arrives.

Latency matters especially in:

- interactive systems,
- ranking and recommendation,

- autonomous systems,
- real-time anomaly or fraud detection.

Sources of latency.

- preprocessing time,
- feature retrieval,
- network communication,
- model execution,
- postprocessing and business logic.

Thus latency is a property of the entire system, not only of the model.

35.4.4 Deployment Optimizations

Common ways to reduce inference cost include:

- model pruning,
- quantization,
- distillation,
- caching frequent predictions,
- batching requests when possible,
- specialized hardware acceleration.

These methods trade off accuracy, complexity, and operational efficiency.

35.4.5 Training–Serving Skew

A major practical problem is **training–serving skew**: the model is trained on one data representation but receives a different one in deployment.

This can happen if:

- feature definitions differ between pipelines,
- missing values are handled differently,
- preprocessing code is duplicated inconsistently,
- online feature stores are out of sync with offline data.

A robust system should minimize the gap between training and serving pipelines.

Insight. Deployment success depends not only on predictive quality, but on whether the model fits operational constraints and receives the inputs it was designed for.

35.5 Monitoring Drift, Failures, and Feedback Loops

Deployment is not the end of the machine learning lifecycle. Models must be monitored continuously after release.

35.5.1 Why Monitoring Is Necessary

Real environments change over time:

- user behavior changes,
- sensors degrade,
- markets shift,
- interfaces change,
- upstream systems evolve.

A model that performed well at launch may deteriorate later if the data distribution changes.

35.5.2 Drift

A central monitoring concern is **drift**.

Covariate drift. The distribution of inputs changes:

$$P_{\text{train}}(x) \neq P_{\text{deploy}}(x).$$

Label or concept drift. The relationship between inputs and targets changes:

$$P_{\text{train}}(y \mid x) \neq P_{\text{deploy}}(y \mid x).$$

Both types of drift can degrade performance.

35.5.3 Operational Monitoring

A deployed system should monitor:

- input distributions,
- prediction distributions,
- latency and throughput,
- system errors and timeouts,
- downstream task metrics when ground truth becomes available.

Examples of warning signs.

- sudden change in average predicted score,
- missing or malformed input features,
- growing disagreement with human review,
- increased response time,
- drop in business or task outcomes.

35.5.4 Failures and Silent Errors

Some ML failures are obvious, but many are silent.

Examples of silent failures.

- shifted label semantics,
- broken feature extraction,
- incorrect join logic in pipelines,
- stale cached features,
- model serving the wrong version.

Because such errors may not crash the system, explicit monitoring is essential.

35.5.5 Feedback Loops

ML systems often influence the very data they later receive.

Examples.

- a recommendation system changes what users see and click,
- a ranking model changes exposure of content,
- a fraud model changes which transactions are investigated,
- a predictive system influences human decisions and labels.

This creates **feedback loops**, in which deployment changes future training data.

Feedback loops can create:

- selection bias,
- reinforcing errors,
- reduced exploration,
- instability over long horizons.

Therefore monitoring should include not only raw performance, but also how the model changes the environment in which it operates.

35.6 Engineering Tradeoffs in Real-World ML

Practical machine learning always involves tradeoffs rather than a single objective.

35.6.1 Accuracy versus Simplicity

A larger model may achieve slightly better validation accuracy, but a smaller model may be preferable if it is:

- easier to debug,
- easier to deploy,
- faster to serve,
- more stable across retraining runs.

35.6.2 Latency versus Quality

In real-time applications, one may sacrifice some predictive accuracy to satisfy strict latency budgets.

For example, a two-stage architecture may use:

- a fast model for candidate filtering,
- a more expensive model for final reranking.

35.6.3 Freshness versus Stability

Frequent retraining may adapt quickly to new data, but it can also introduce instability and make debugging harder.

Less frequent retraining is more stable, but may fail to adapt to drift.

35.6.4 Reproducibility versus Performance

Deterministic execution improves reproducibility, but can reduce throughput or hardware efficiency.

Thus engineering teams often choose a balance:

- maximize reproducibility in development and evaluation,
- allow some nondeterminism in large-scale production training when necessary.

35.6.5 Automation versus Human Oversight

Automation speeds up iteration, but some high-stakes tasks require human review, fallback rules, or manual escalation paths.

A strong ML system often combines:

- statistical predictions,
- business logic,
- rule-based safeguards,
- human intervention when uncertainty is high.

Debugging as a systems activity. Debugging in ML is broader than debugging code. One must diagnose:

- data bugs,
- label bugs,
- optimization failures,
- evaluation leakage,
- serving mismatches,
- monitoring blind spots.

Useful debugging tools include:

- training and validation curves,
- confusion matrices or error breakdowns,
- gradient and activation statistics,
- feature-distribution checks,
- ablation studies,
- inspection of individual failures.

Practical debugging principle. When a system fails, simplify the pipeline, isolate components, verify assumptions, and test each stage independently before adding complexity back.

Insight. Real-world ML design is governed by engineering constraints as much as by learning theory.

35.7 A Unifying Perspective

Practical machine learning systems integrate many layers of infrastructure:

- **data pipelines** for collection, cleaning, labeling, and versioning,
- **evaluation infrastructure** for train/validation/test separation,
- **training pipelines** for optimization, checkpointing, and scaling,
- **experiment tracking** for reproducibility and comparison,
- **serving systems** for inference under latency and cost constraints,
- **monitoring systems** for drift, failures, and feedback loops.

Core idea. A successful ML application is an end-to-end system whose components must work together reliably over time.

Key insight. The practical effectiveness of machine learning depends not only on the model, but on how data, infrastructure, deployment, and monitoring are designed as a coherent system.

35.8 Outlook

This chapter presented machine learning as an engineering and systems discipline:

- framing ML as a systems problem,
- building data pipelines and dataset splits,
- managing training workflows and experiments,
- designing serving systems under inference constraints,
- monitoring drift, failures, and feedback loops,
- navigating real-world engineering tradeoffs.

This concludes the book.

Final guiding principle. Machine learning succeeds in practice when mathematical models, data pipelines, and engineering systems are designed together rather than in isolation.

Exercises

1. **End-to-end pipeline design.** Choose one application:

   - spam detection,
   - medical image classification,
   - product recommendation,
   - fraud detection.

   Design a complete ML pipeline for the task, including:

   - data collection,
   - cleaning and preprocessing,
   - labeling,
   - train/validation/test splitting,
   - training and experiment tracking,
   - deployment,
   - monitoring after launch.

2. **Split design.** Explain why random splitting may be inappropriate in a time-dependent prediction task. Propose a better train/validation/test strategy.

3. **Label quality.** Give three examples of labeling noise or ambiguity. For each, explain how it could affect model behavior.

4. **Training–serving skew.** Describe a scenario in which the preprocessing used during serving differs from the preprocessing used in training. What failure might this create?

5. **Inference latency.** A model has excellent validation accuracy but is too slow for real-time use. List several ways to reduce latency and explain the tradeoffs.

6. **Experiment management.** Why is it not enough to record only final validation accuracy? What additional information should be tracked for each run?

7. **Monitoring drift.** Explain the difference between covariate drift and concept drift. Give an example of each.

8. **Feedback loops.** Describe how a recommendation system can create a feedback loop that changes future training data. Why can this make evaluation more difficult?

9. **Deployment failure analysis.** Consider a deployed classifier whose measured accuracy suddenly drops. List at least five possible failure points in the end-to-end system, not limited to the model itself.

10. **Debugging workflow.** Suppose training loss decreases, but validation performance is poor and unstable across runs. Give a systematic debugging plan that checks data, optimization, evaluation, and infrastructure.

11. **Engineering tradeoff essay.** Write a short essay on one of the following tradeoffs:

    - accuracy versus latency,
    - reproducibility versus training speed,
    - model complexity versus ease of deployment.

    Explain why there is no universally optimal choice.

12. **Real-world systems reflection.** Explain why practical ML should be viewed as a systems discipline rather than only a modeling discipline.

Chapter 36

Limits and Future Directions

36.1 What Current Machine Learning Still Struggles With

Modern machine learning has achieved remarkable successes in perception, language, control, and generative modeling. Yet these successes should not be confused with a complete solution to intelligence. Current systems remain highly capable in some regimes and surprisingly fragile in others.

Central tension. Many modern models are extremely effective at interpolation within the range of their training data, but they often struggle with robust extrapolation, causal understanding, and reliable reasoning under novel conditions.

Important current limitations.

- strong dependence on large amounts of data,
- sensitivity to changes in the input distribution,
- limited reliability under rare or adversarial conditions,
- difficulty separating causation from correlation,
- incomplete or unstable forms of reasoning,
- large computational and energy requirements,
- limited transparency and interpretability.

These limits do not negate the importance of current machine learning. Rather, they show that present paradigms are powerful but incomplete.

Limits of current paradigms. A useful way to summarize the situation is:

- **supervised learning** depends strongly on labeled data and benchmark design,
- **self-supervised learning** captures rich statistical structure but does not automatically yield robust world models,
- **deep scaling** improves many capabilities, but does not resolve all failure modes,
- **generative modeling** can produce realistic outputs without guaranteeing truth, grounding, or understanding.

Insight. The future of machine learning is shaped not only by extending current successes, but also by understanding where and why existing approaches fail.

36.2 Robustness, Uncertainty, and Distribution Shift

One of the clearest limitations of modern machine learning is fragility under perturbation and uncertainty.

36.2.1 Robustness

A model is **robust** if small or semantically irrelevant changes in input do not produce large or unjustified changes in output. Informally, one hopes that

$$\|x - x'\| \text{ small} \quad \Longrightarrow \quad f(x) \approx f(x').$$

This ideal is difficult to guarantee in high-dimensional settings.

Adversarial examples. A model may be highly accurate on standard data but vulnerable to carefully designed perturbations that cause incorrect predictions. Such perturbations may be small in norm and sometimes nearly imperceptible to humans.

Broader robustness issues. Fragility appears in many forms:

- adversarial perturbations,
- corruptions such as blur, noise, or occlusion,
- rare or unusual examples,
- changes in environment or measurement process.

36.2.2 Uncertainty

Accurate prediction is not enough. A reliable system should also know when it is uncertain.

Aleatoric uncertainty. This reflects irreducible noise in the data-generating process.

Epistemic uncertainty. This reflects uncertainty due to limited data, incomplete knowledge, or model misspecification.

Why uncertainty matters. In high-stakes tasks, a model should distinguish between:

- confident predictions on familiar data,
- uncertain predictions on rare or novel inputs.

Common tools include:

- Bayesian approaches,
- ensemble methods,
- predictive calibration,
- abstention or selective prediction.

36.2.3 Distribution Shift

A major challenge is that training and deployment distributions may differ.

Covariate shift. The input distribution changes:

$$P_{\text{train}}(x) \neq P_{\text{test}}(x).$$

Concept or conditional shift. The relationship between inputs and targets changes:

$$P_{\text{train}}(y \mid x) \neq P_{\text{test}}(y \mid x).$$

Out-of-distribution inputs. A model may encounter inputs unlike those seen during training, in which case its confidence estimates may become unreliable.

Core difficulty. Standard empirical risk minimization is typically designed for interpolation under approximately fixed distributions, not for arbitrary shifts in environment.

Insight. Robustness, uncertainty, and distribution shift are central because real-world intelligence requires dependable behavior outside ideal benchmark conditions.

36.3 Causality, Abstraction, and Reasoning

Statistical learning is powerful, but intelligence seems to require more than pattern recognition. It also involves causal structure, abstraction, and flexible reasoning.

36.3.1 Causality

Most standard machine learning methods are optimized to exploit correlations:

$$x \mapsto y.$$

But correlation does not imply causation.

Why causality matters. A purely correlational system may:

- rely on spurious features,
- fail under interventions,
- break when the environment changes,
- mispredict the effects of actions.

Causal reasoning asks questions such as:

- What would happen if we changed x?
- Which variables are causes rather than symptoms?
- Which relations remain stable across environments?

These questions are crucial in medicine, policy, science, and decision-making systems.

36.3.2 Abstraction

Intelligent systems often need to build higher-level concepts from raw data.

Abstraction refers to the formation of representations that capture essential structure while ignoring irrelevant detail. It is central to:

- transfer across tasks,
- compositional generalization,
- planning and symbolic reasoning,
- scientific understanding.

Although deep networks can learn rich representations, the extent to which they discover stable abstractions remains an open question.

36.3.3 Reasoning

Modern large models can display impressive forms of reasoning, yet their reliability remains uneven.

Current challenges in reasoning.

- multistep deduction may be inconsistent,
- reasoning can be brittle under rephrasing,
- models may produce convincing but incorrect chains of argument,
- abstract or compositional tasks remain difficult.

A central question. Do current neural architectures truly reason, or do they approximate reasoning through sophisticated statistical pattern completion?

The answer may depend on the task, the scale of the model, the training procedure, and the form of external tools available during inference.

Insight. Progress in machine learning may require deeper integration of statistical learning with causal models, abstraction mechanisms, and more reliable forms of reasoning.

36.4 Interpretability and Scientific Understanding

As models become larger and more capable, understanding how they work becomes increasingly important.

36.4.1 Interpretability

Interpretability is not a single property, but a collection of methods for understanding model behavior.

Major goals of interpretability.

- trust and transparency,
- debugging and failure analysis,
- scientific understanding of learned representations,
- safety and accountability.

Common approaches.

- feature attribution,
- saliency and attention-based analyses,

- probing internal representations,
- mechanistic analysis of circuits and subnetworks,
- surrogate or simplified models.

Core challenge. Explanations are often partial, non-unique, and sensitive to the method used. A model may admit many plausible interpretations without any one of them being definitive.

36.4.2 Scientific Machine Learning

Machine learning is increasingly used not only for prediction, but also for scientific discovery.

Scientific ML includes:

- learning surrogate models for physical systems,
- integrating differential equations with neural networks,
- using data-driven models in simulation,
- discovering hidden structure in biological, chemical, or physical data.

This area raises special demands:

- data may be scarce or expensive,
- physical constraints may be essential,
- interpretability and uncertainty are especially important,
- extrapolation beyond observed regimes is often required.

Thus scientific ML highlights both the promise and the limits of current paradigms.

Insight. Interpretability is not only about trust in deployed systems; it is also about using machine learning as a tool for understanding the world.

36.5 Energy, Data, and Compute Constraints

Machine learning progress is often associated with scale, but scale comes with costs.

36.5.1 Data Efficiency

Many modern systems require enormous datasets. This raises several issues:

- labeled data is expensive,
- high-quality data may be scarce,
- many domains cannot support internet-scale collection,
- data quality may matter more than data volume.

A major long-term goal is **data-efficient learning**: achieving strong performance from fewer examples, weaker supervision, or richer prior structure.

36.5.2 Compute Constraints

Training large models may require vast computational resources.

Consequences of compute-intensive paradigms.

- access becomes concentrated in well-resourced institutions,
- experimentation becomes expensive,
- reproducibility becomes harder,
- some research directions are favored over others because they scale well computationally.

36.5.3 Energy Constraints

Large-scale training and inference also consume significant energy.

This matters because:

- deployment at scale multiplies inference cost,
- environmental impact becomes non-negligible,
- hardware limitations constrain practical systems,
- real-time or edge applications require efficiency.

These concerns motivate work on:

- efficient architectures,
- sparsity and compression,
- distillation,
- low-precision computation,
- algorithmic efficiency improvements.

36.5.4 A Broader Limitation

A paradigm that improves mainly through more data and more compute may be powerful, but it may not be indefinitely sustainable.

Key question. How much future progress can come from scaling alone, and how much will require genuinely new ideas in representation, reasoning, memory, or learning dynamics?

Insight. Energy, data, and compute constraints are not only engineering details. They shape what kinds of intelligence can be built and who can build it.

36.6 Emerging Directions in Generative and Interactive AI

Recent years have seen rapid progress in generative models and interactive systems.

36.6.1 Generative AI

Generative models now produce realistic text, images, audio, video, and multimodal outputs. This has expanded the role of machine learning from classification and prediction to synthesis, simulation, and interaction.

Emerging themes in generative modeling.

- multimodal generation,
- controllable generation,
- long-context modeling,
- world-model-like simulation,
- integration of generation with reasoning and action.

At the same time, generative systems raise new issues:

- factual reliability,
- hallucination,
- controllability,
- evaluation difficulty,
- alignment with human goals.

36.6.2 Interactive and Agentic Systems

A growing direction is the construction of systems that do not merely predict or generate, but act over time.

Agentic systems are systems that:

- maintain goals over multiple steps,
- interact with tools or environments,
- plan or revise actions,
- incorporate feedback,
- sometimes coordinate multiple components such as retrieval, memory, and action selection.

These systems lie between classical machine learning, control, reinforcement learning, and human–computer interaction.

New challenges for agentic systems.

- error accumulation across long horizons,
- reliable planning,
- safe action under uncertainty,
- memory consistency,
- evaluation in open-ended environments.

36.6.3 Optimistic and Skeptical Views

There are at least two broad attitudes toward future progress.

Optimistic scenario. Scaling, multimodality, tool use, and better training methods may produce increasingly general-purpose systems with stronger reasoning, planning, and scientific usefulness.

Skeptical scenario. Current systems may remain limited by shallow causal understanding, brittle reasoning, enormous resource requirements, and dependence on statistical regularities rather than true world models.

The future may include elements of both views.

Insight. Emerging generative and interactive AI systems broaden what machine learning can do, but they also amplify questions about reliability, agency, and control.

36.7 Open Mathematical Questions

Many of the deepest open problems in machine learning are mathematical.

36.7.1 Generalization

Why do highly overparameterized models often generalize well? Existing theories explain parts of the phenomenon, but a complete predictive theory remains unavailable.

36.7.2 Optimization and Dynamics

Why do simple first-order methods work so well in nonconvex systems? What aspects of loss geometry, noise, and architecture determine the solutions actually found by training?

36.7.3 Representation and Abstraction

How do neural networks form useful internal representations? Under what conditions do they learn invariant, compositional, or causal structure rather than merely compressing correlations?

36.7.4 Reasoning and Computation

Can neural sequence models implement robust algorithmic reasoning in a principled sense? What are the right mathematical frameworks for studying reasoning in learned systems?

36.7.5 Robustness and Shift

Can one derive meaningful guarantees for robustness under realistic perturbations or distribution shifts? What assumptions are necessary for such guarantees to be non-vacuous?

36.7.6 Learning with Structure

How should one integrate symmetry, physics, geometry, logic, or causal structure into learning theory and algorithms in a mathematically principled way?

Open mathematical direction. Future theory will likely need to unify ideas from:

- probability and statistics,
- optimization,
- dynamical systems,
- information theory,

- geometry,
- control,
- scientific modeling.

Insight. The mathematics of learning is still developing, and many of its most important concepts may not yet have their final form.

36.8 A Final Perspective

This book has developed machine learning from both mathematical and practical perspectives. Across supervised learning, optimization, deep networks, generative models, and systems design, one theme has recurred:

> learning systems are shaped by the interaction of data, models, objectives, algorithms, and environments.

What has become clear.

- current machine learning is powerful but not complete,
- success on benchmarks does not eliminate fragility,
- intelligence requires more than raw pattern matching,
- future progress will depend on theory, systems, and scientific insight together.

Final insight. The future of AI will be determined not only by building larger models, but by building systems that are more robust, data-efficient, interpretable, causally informed, and mathematically understood.

36.9 Closing Remarks

Machine learning stands at an unusual and fertile intersection of mathematics, statistics, computer science, engineering, and the sciences.

Outlook.

- new theoretical frameworks may clarify generalization, reasoning, and representation,
- new architectures may improve efficiency, abstraction, and multimodal interaction,
- scientific ML may deepen the connection between machine learning and physical understanding,

- responsible development will remain central as systems become more capable and more widely deployed.

Final guiding principle. Progress in artificial intelligence should be judged not only by performance, but by understanding, reliability, efficiency, and the quality of the scientific and human purposes it serves.

Exercises

1. **Critical reflection on current paradigms.** Give three examples of tasks at which modern machine learning performs impressively and three examples of capabilities it still lacks. Explain why these do not contradict each other.

2. **Robustness versus accuracy.** A classifier performs extremely well on a benchmark test set but fails under small image corruptions. Explain why high in-distribution accuracy does not guarantee robustness.

3. **Uncertainty decomposition.** Explain the difference between aleatoric and epistemic uncertainty. Give one example of each in a real application.

4. **Distribution shift.** Describe a situation in which

$$P_{\text{train}}(x) \neq P_{\text{test}}(x)$$

but

$$P(y \mid x)$$

is approximately unchanged. Then describe a situation in which the conditional relation itself changes.

5. **Correlation and causation.** Give an example where a predictive model can succeed by exploiting correlation, but fails when an intervention changes the environment. Explain why causal reasoning would help.

6. **Reasoning and abstraction.** Write a short essay on the difference between pattern recognition and reasoning. To what extent can one emerge from the other?

7. **Interpretability critique.** Choose one interpretability method such as feature attribution, saliency, or probing. Explain what it can reveal and what it may fail to reveal.

8. **Scientific ML.** Why is scientific machine learning often harder than benchmark prediction? Discuss data scarcity, physical constraints, and extrapolation.

9. **Data efficiency.** Why is data efficiency important for the future of AI? Give examples of domains in which internet-scale data collection is unrealistic.

10. **Compute and energy.** Discuss how increasing model scale creates both scientific opportunities and practical limitations. Consider cost, access, reproducibility, and environmental impact.

11. **Agentic systems.** What distinguishes an agentic system from a one-shot predictor or generator? Give an example of a possible failure mode unique to long-horizon interactive systems.

12. **Optimistic future scenario.** Write a short essay describing an optimistic future for machine learning over the next decade. What breakthroughs or developments would support this view?

13. **Skeptical future scenario.** Write a short essay describing a skeptical future for machine learning over the next decade. What limitations or bottlenecks would support this view?

14. **Comparison of futures.** Compare the optimistic and skeptical scenarios from the previous two exercises. Which points are genuinely in conflict, and which could both be partly true?

15. **Open mathematical questions.** Choose one open mathematical question from this chapter and explain why it is difficult. What existing mathematical tools seem relevant, and where do they appear insufficient?